

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ СИСТЕМ И ТЕХНОЛОГИЙ

Кафедра «Информационные системы»

Дисциплина «Информационная безопасность»

Лабораторная работа № 1

Разработка программы разграничения полномочий пользователей
на основе парольной аутентификации с использованием
встроенных криптопровайдеров.

Выполнил:
студент гр. ПИБд-41
Белянин Н.Н.
Проверил:
Доцент, к.т.н.
Мытарев П. В.

Ульяновск, 2025г.

Лабораторная работа № 1

Разработка программы разграничения полномочий пользователей на основе парольной аутентификации с использованием встроенных криптопровайдеров.

Содержание задания

1. Программа должна обеспечивать работу в двух режимах: администратора (пользователя с фиксированным именем ADMIN) и обычного пользователя.
2. В режиме администратора программа должна поддерживать следующие функции (при правильном вводе пароля):
 - смена пароля администратора (при правильном вводе старого пароля);
 - просмотр списка имен зарегистрированных пользователей и установленных для них параметров (блокировка учетной записи, включение ограничений на выбираемые пароли, минимальная длина пароля, срок действия пароля) – всего списка целиком в одном окне или по одному элементу списка с возможностью перемещения к его началу или концу;
 - добавление уникального имени нового пользователя к списку с пустым паролем (строкой нулевой длины);
 - блокирование возможности работы пользователя с заданным именем;
 - включение или отключение ограничений на выбираемые пользователем пароли (в соответствии с индивидуальным заданием, определяемым номером варианта);
 - завершение работы с программой.
3. Для каждого пользователя должны устанавливаться минимальная длина пароля (по умолчанию – 0, т.е. пароль может быть не установлен) и срок действия пароля в месяцах (значение 0 означает бессрочный пароль).
4. В режиме обычного пользователя программа должна поддерживать только функции смены пароля пользователя (при правильном вводе старого пароля) и завершения работы, а все остальные функции должны быть заблокированы.
5. После своего запуска программа должна запрашивать у пользователя в специальном окне входа ввод его имени и пароля. При вводе пароля его символы всегда должны на экране заменяться символом ‘*’.
6. При отсутствии введенного в окне входа имени пользователя в списке зарегистрированных администратором пользователей программа должна выдавать соответствующее сообщение и предоставлять пользователю возможность повторного ввода имени или завершения работы с программой.
7. При неправильном вводе пароля программа должна выдавать соответствующее сообщение и предоставлять пользователю возможность повторного ввода. При трехкратном вводе неверного пароля работа программы должна завершаться.
8. При первоначальном вводе пароля (обязательном при первом входе администратора или пользователя с зарегистрированным ранее администратором именем) и при дальнейшей замене пароля программа должна просить пользователя подтвердить введенный пароль путем его повторного ввода.
9. Если выбранный пользователем пароль не соответствует требуемым ограничениям (при установке соответствующего параметра учетной записи пользователя), то программа должна выдавать соответствующее сообщение и предоставлять пользователю возможность ввода другого пароля, завершения работы с программой (при первом входе данного пользователя) или отказа от смены пароля.
10. Информация о зарегистрированных пользователях, их паролях, отсутствии блокировки их работы с программой, а также включении или отключении ограничений на

выбираемые пароли должна сохраняться в специальном файле или базе данных(предпочтительнее). При первом запуске программы, в случае файла, он должен создаваться автоматически и содержать информацию только об администраторе, имеющем пустой пароль.

11. Файл или база данных с учетными данными зарегистрированных пользователей должен быть защищен криптографическими средствами от несанкционированного доступа.
12. При запуске программы файл с учетными данными должен расшифровываться во временный файл (или в файл в оперативной памяти), который после завершения работы программы должен быть снова зашифрован для отражения возможных изменений в учетных записях пользователей. «Старое» содержимое файла учетных записей при этом стирается.
13. Пароль в файл(базу данных) записывается в виде хэш-кода. При вводе пароля идет сравнение хэш-кода введенного пароля с хэш-кодом пароля, сохраненного в файле.
14. После ввода парольной фразы при запуске программы, генерации ключа расшифрования и расшифрования файла с учетными данными зарегистрированных пользователей правильность введенной парольной фразы определяется по наличию в расшифрованном файле учетной записи администратора программы.
15. При вводе неправильной парольной фразы или отказе от ее ввода работа программы должна завершаться с выдачей соответствующего сообщения.
16. Временный файл на диске с расшифрованными учетными данными после завершения работы программы удаляется.
17. Варианты использования алгоритмов шифрования и хеширования выбираются в соответствии с выданным преподавателем заданием.
18. Интерфейс с программой должен быть организован на основе меню, обязательной частью которого должно являться подменю «Справка» с командой «О программе». При выборе этой команды должна выдаваться информация об авторе программы и выданном индивидуальном задании. Интерфейс пользователя программы может также включать панель управления с дублирующими команды меню графическими кнопками и строку состояния.
19. Для реализации указанных в пунктах 2-3 функций в программе должны использоваться специальные диалоговые формы, позволяющие пользователю (администратору) вводить необходимую информацию.
20. Отчет по лабораторной работе должен содержать листинги программы с подробными комментариями, скрин-шоты экранов, описание структуры файла или базы данных с пользовательской информацией, подробное описание функций, связанных с хешированием и шифрованием, описание мест хранения зашифрованных данных, в т.ч. в момент их шифрования (расшифрования).

Вариант – 6

Индивидуальные варианты заданий (ограничения на выбираемые пароли) Отсутствие повторяющихся символов.

Индивидуальные варианты заданий на использование криптографических методов

№	Используемый режим шифрования алгоритма DES для шифрования файла	Добавление к ключу случайного значения	Используемый алгоритм хеширования пароля
6	CBC	Нет	SHA

Ссылка на репозиторий: <https://github.com/Nikbeli/Information-Security>

Оглавление

Ход выполнения лабораторной работы	5
<i>Теоретическая часть выполнения</i>	5
<i>Основные понятия</i>	5
<i>Принципы авторизации безопасного управления доступом</i>	7
<i>Угрозы защите программной системы</i>	7
Реализация программной системы	8
<i>Стек технологий</i>	9
<i>Структура слоёв веб-сервера</i>	10
user/utills/ DesFileCipher.java (Шифрование и расшифровка файлов с записью)	14
user/config/DatabaseCryptoSyncManager.java	15
<i>Реализация JWT</i>	22
JwtProperties для конфигурации токена	22
JwtService сервис токена	23
JwtFilter.java	24
SecurityConfiguration.java реализация защиты	26
DTO модели	28
MapperStruct для маппинга	29
UserService.java – сервис бизнес-логики	30
UserController.java – контроллер (эндпоинты)	33
AuthService.java – сервис аутентификации	35
AuthController.java – контроллер аутентификации	38
PasswordHashUtil.java – хэширование пароля	40
OtpService.java бизнес-логика писем и проверки	41
EmailService.java – класс формирования письма	42
Демонстрация работы программной системы	43
<i>Clients.ts – обращения к API</i>	54
<i>LoginPage.tsx – страница входа</i>	57
<i>WelcomePage.tsx – информация о студенте</i>	59
<i>UserManager.tsx – панель администрирования пользователей</i>	60
Заключение	69
Приложение. Листинг кода	72

Ход выполнения лабораторной работы

Теоретическая часть выполнения

Современные информационные системы требуют надёжной защиты от несанкционированного доступа, в частности от атак и взломов злоумышленников, которые несут угрозу, чтобы нарушить безопасность. Информационная безопасность становится неотъемлемой частью на протяжении всего жизненного цикла программного обеспечения (идеи, разработки, тестирования, эксплуатации). Одними из элементов обеспечения защиты являются аутентификация пользователей и разграничение полномочий. Парольная аутентификация остаётся наиболее распространённым способом идентификации, а использование встроенных криптопровайдеров позволяет повысить надёжность обработки и хранения учётных данных.

Цель работы: Разработка программы и изучение механизмов информационной безопасности: аутентификации, авторизации и разграничения полномочий пользователя, а также освоение практических навыков использования встроенных криптопровайдеров для защиты учётных данных (хэширование и шифрование)

Основные понятия

- **Информационная безопасность** – это защищённость информации и поддерживающей инфраструктуры от случайных или преднамеренных воздействий естественного или искусственного характера, которые могут нанести неприемлемый ущерб субъектам информационных отношений, в том числе владельцам и пользователям информации и поддерживающей инфраструктуры.

- **Защита информации** – это комплекс мероприятий, направленных на обеспечение информационной безопасности.

И защита информации от несанкционированного доступа, разрушения, модификации или раскрытия – это очень важно особенно для больших компаний.

- **Аутентификация** – это процесс проверки подлинности субъекта (пользователя, программы или устройства) с установленным или известным уровнем гарантии, направленный на подтверждение, что предъявленные данные действительно принадлежат указанному субъекту.

Существует несколько видов аутентификации:

1. **Знание (knowledge-based)** – пароль, PIN-код, секретный вопрос. Данный вид может так же подразделяться на однофакторную и двухфакторную аутентификацию (2FA-метод проверки личности пользователя, требующий двух разных способов подтверждения, к примеру: пароль + смс).

2. **Владение (possession-based)** – токен¹, ключ USB, смарт-карта.

3. **Присущие признаки (inherence-based)** – биометрия (отпечаток пальца, сетчатка глаза, голос).

- **Авторизация** – это процесс предоставления субъектам определённых прав и полномочий после успешной аутентификации на выполнение определённых действий.

- **Идентификация** – это присвоение субъектам и объектам доступа идентификаторов с целью удобного распознавания. Цель — обеспечить легитимным пользователям безопасный доступ к ресурсам, данным или услугам, а злоумышленникам — не перехватить доступ и действовать от чужого имени. Обычный способ *идентификации* - ввод имени пользователя при входе в систему. Стандартное средство проверки подлинности (*аутентификации*) пользователя - пароль.

¹Токен (аппаратный токен, USB-ключ, криптографический токен) – это дополнительный способ аутентификации пользователей и авторизации доступа к цифровым ресурсам. Токен безопасности содержит уникальные данные, которые делают доступ к системе более защищенным.

Принципы авторизации безопасного управления доступом

При проектировании и эксплуатации информационных систем критически важно соблюдать *принципы безопасного управления доступом* рассмотрим два следующих фундаментальных принципа информационной безопасности:

1. *Принцип наименьших привилегий* – пользователь или процесс должен обладать только теми правами, которые необходимы для выполнения определённых функций и задач. Это минимизирует потенциальный ущерб в случае компрометации учётной записи.

2. *Принцип разделения обязанностей* – не допускается доступ к критическим важным операциям и ресурсам, которые должны находиться в руках разных пользователей. Такой подход снижает риски злоупотребления полномочиями и повышает прозрачность контроля.

Угрозы защите программной системы

Важно при конструировании программного обеспечения предусмотреть защиту системы. Сейчас парольная аутентификация широко распространена и в то же время очень уязвима к целому ряду атак. Одной из серьёзных угроз является перехват и кража паролей, которые происходят различными способами:

- *Перехват сети (Network Sniffing)*

Аутентификационные данные могут передаваться по незащищённому каналу (HTTP), в таком случае злоумышленник может использовать снифферы (Wireshark, tcpdump) для перехвата трафика и извлечения паролей в открытом виде. Для предотвращения этого стоит всю передачу данных между сервером и клиентом защитить по TLS² (HTTPS). Это гарантирует

² **TLS (Transport Layer Security)** — криптографический протокол, который обеспечивает безопасную передачу данных по сети.

конфиденциальность и целостность передаваемых учётных данных пользователя.

- *Вредоносное программное обеспечение (Keyloggers, трояны, фишинг)*

Клиентская сторона уязвима программами, перехватывающими ввод с клавиатуры, буфера обмена или другими подменяющими интерфейсами входа в систему. Особенно актуально для Desktop-приложений. Предотвратить это можно на клиенте, с помощью минимизации времени хранения пароля в памяти и очистка полей ввода, а на сервере можно реализовать двухфакторную аутентификацию (2FA) как дополнительный барьер.

- *Кража из файлов или базы данных*

Хранение паролей в формате plain-text³ будет крайне сильной уязвимостью. Достаточно получить доступ к базе данных (через SQL-инъекцию или утечку резервной копии), чтобы скомпрометировать всех пользователей. Для решения этой проблемы нужно никогда не хранить пароли в открытом виде. В системе применять хэширование с солью, когда каждому пользователю генерируется уникальная соль, которая хранится в отдельном поле рядом с хэшем пароля.

Реализация программной системы

Приступим к выполнению практической части лабораторной работы, где нужно реализовать программную систему в соответствии с техническим заданием. Во время реализации программы необходимо предусмотреть различные технологии обеспечения безопасности в архитектуре кода и при работе с ней. В ходе разработки было принято решение реализовать многослойную модель MVC (Model-View-Controller) проекта на клиент-

³ **Plain-text** (простой текст) — формат электронных писем, который содержит только текст без форматирования, изображений, кнопок и других сложных элементов.

серверной архитектуре⁴. Реализованы одна серверная часть, строго разделённая на сервисы, модели, репозиторий и контроллеры (end-points). Клиентская часть представлена одним независимым клиентом веб-приложением.

Стек технологий

- ***Серверная часть (Backend)*** – реализован на языке программирования Java (v21) с использованием фреймворка Spring Boot (v3.5.6). Для валидации используется Hibernate Validator, миграция – Flyway, а для ORM (Object-Relational Mapping, объектно-реляционное отображение) Spring Data JPA и PostgreSQL. Для получения документации API используется OpenAPI (Swagger UI).

Важной частью безопасности при разработке системы с разграничением прав является использование Spring Security + JWT ⁵(json web token). Также для повышения безопасности программной системы было принято решение добавить двухфакторную аутентификацию (2FA) с помощью внедрения почтового сервиса и отправления письма с кодом доступа. Данная функция доступна опционально по выбору админа для пользователей.

- ***Веб-клиент (Frontend или клиентская часть)*** – реализовано на фреймворке React (компонетно-ориентированный подход с хуками), используемый язык программирования для типизации TypeScript. Сборка осуществляется на Vite и применяется линтинг ESLint (единые правила код-стайла, предотвращение ошибок). Разделение на компоненты, страницы и api обращения. Для последних используется axios, чтобы успешно взаимодействовать с REST API.

⁴ **Клиент-серверная архитектура** — это модель организации вычислительных систем, в которой задачи распределены между *клиентами* и *серверами*. Клиент запрашивает услуги или ресурсы у сервера, а сервер отвечает на запросы, предоставляя необходимые данные или функциональность.

⁵ **JWT** - открытый стандарт (RFC 7519) для создания токенов доступа, основанный на формате JSON. Позволяет безопасно передавать информацию между клиентом и сервером с помощью шифрования)

• **База данных (PostgreSQL)** – в качестве системы управления базы данных используется PgAdmin4, для миграции и переноса версий (версионирования SQL-скриптов) используется Flyway. Основные таблицы: users.

Структура слоёв веб-сервера

Controller – Rest-эндпоинты (@RestController), обрабатывают HTTP-запросы, валидируются входные данные и вызываются сервисы.

Service – бизнес-логика (аутентификация, управление пользователями, политики паролей, отправка писем).

Repository – интерфейсы наследуют JpaRepository, взаимодействие с базой данных.

Entity – JPA-сущности, отображаются таблицы базы данных.

В базе данных и в веб-сервере API используется всего одна сущность пользователей Users. Схема модели в базе данных:

1. **UUID Id** – идентификатор пользователя.
2. **String email** – логин и почта пользователя (для входа в систему и отправки письма на почту).
3. **String password_hash** – пароль, который хранится в виде хэша, созданного алгоритмом SHA.
4. **String salt** – для усиления криптостойкости, было принято решение использовать *соль*⁶
5. **Role role** – поддержка ролевой системы (осуществляется с помощью enum Role: USER, ADMIN).

⁶ **Соль (Salt) в контексте информационной безопасности (ИБ)** — это случайные данные, которые добавляются к паролю перед хешированием для защиты от атак.

6. **Boolean accountLocked** – блокировка аккаунта.

7. **Boolean passwordRestrictions** – ограничения на пароль по вводу (отсутствие повторяющихся символов).

8. **Int minPasswordLength** – ограничение на длину пароля.

9. **Int passwordExpirationMonths** - срок действия пароля в месяцах.

10. **LocalDateTime passwordLastChanged** – дата изменения пароля.

11. **Int failedAttempts** – счётчик неудачных попыток входа.

12. **Boolean emailConfirmed** – обязательное подтверждение через отсылку кода доступа на почту пользователя.

13. **Boolean firstLogin** – флаг первого входа.

Примечание: большинство полей не nullable и должны быть заполнены, исключения пароль, соль, дата пароля и длина пароля. Использование типа UUID вместо автоинкрементальных ID позволяет повысить безопасность и предотвратить обычный перебор. Хранение salt отдельно от хэша позволяет гибко менять алгоритмы хэширования.

```
package com.cryptoauth.infosecurity.user.entity;

import java.time.LocalDateTime;
import java.util.UUID;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(updatable = false, nullable = false)
    private UUID id;

    @Column(unique = true, nullable = false)
    private String email;
```

```
// По ТЗ: пароль в базе хранится как хэш (алгоритм — SHA)
```

```
@Column(name = "password_hash", length = 512)
```

```
private String password_hash;
```

```
private String salt;
```

```
@Column(nullable = false)
```

```
@Enumerated(EnumType.STRING)
```

```
private Role role;
```

```
// блокировка учётки
```

```
@Column(nullable = false)
```

```
private boolean accountLocked;
```

```
// включены ли ограничения
```

```
@Column(nullable = false)
```

```
private boolean passwordRestrictions;
```

```
// срок действия пароля в месяцах (0 = бессрочно)
```

```
@Column(nullable = false)
```

```
private int passwordExpirationMonths;
```

```
// дата смены пароля
```

```
private LocalDateTime passwordLastChanged;
```

```
// минимальная длина пароля (0 = разрешён пустой пароль)
```

```
private int minPasswordLength;
```

```
// счётчик неуспешных попыток входа
```

```
@Column(nullable = false)
```

```
private int failedAttempts;
```

```
// первый вход
```

```
@Column(nullable = false)
```

```
private boolean firstLogin;
```

```
// нужно ли подтверждение из почты
```

```
@Column(nullable = false)
```

```
private boolean emailConfirmed;
```

```
}
```

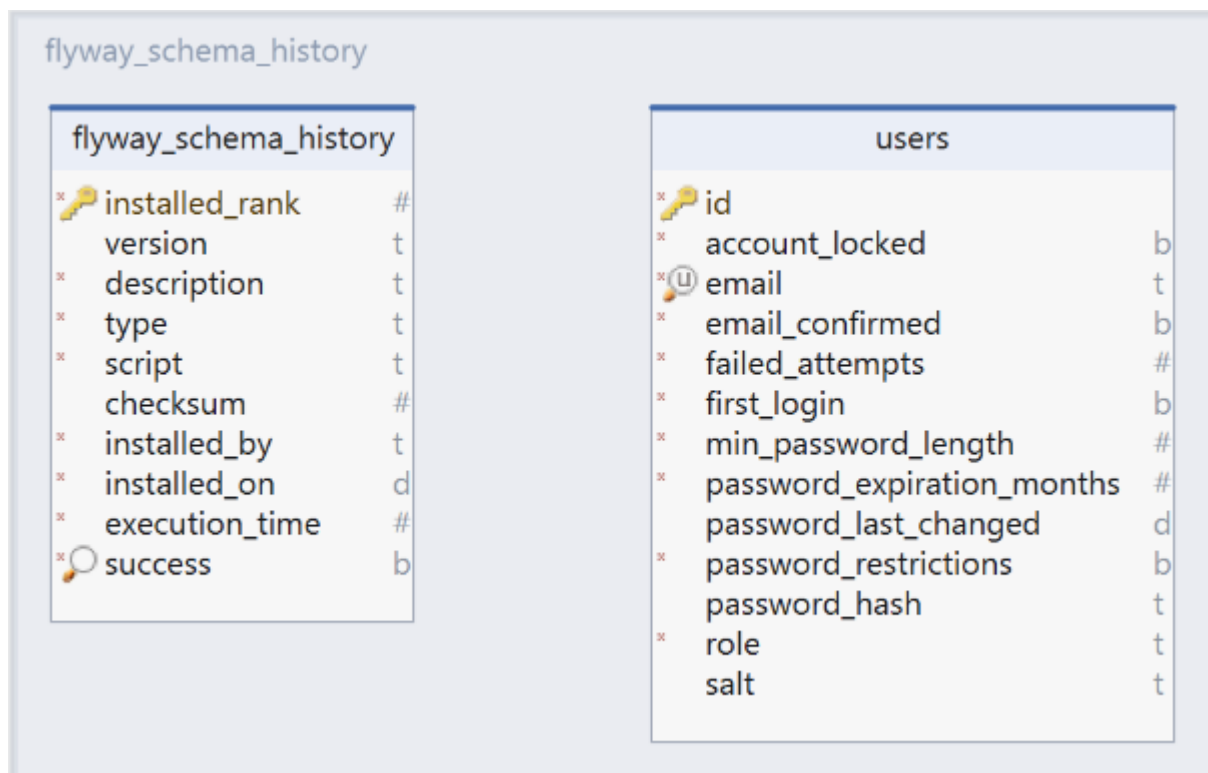


Рисунок 1. Сущности в базе данных.

При инициализации и запуске проекта веб-сервера ключевым и важным аспектом безопасности в данной программной системе является создание зашифрованного файла, которая представляет собой снимок базы данных в формате json. Как это работает?

Первый запуск создаёт файл и записывает в него админа всей системе. При повторном запуске, зашифрованный файл userdata.enc расшифровывается в userdata.json и все данные переносятся в базу данных PostgreSQL, там данные о пользователях хранятся до тех пор, пока работа с программой не будет прекращена. При выключении программной системы, добавленные данные в базу данных перенесутся в json-файл (наш снимок) и зашифруются. При этом при каждом запуске система запрашивает парольную фразу, чтобы предоставить доступ и расшифровать файл. База данных тем временем удалится и пропадёт. В коде представлены два метода **EncryptFile**, **DecryptFile** их логика похожа, потому что шифрование и

расшифровка похожи между собой (XOR операция). Фрагменты кода, отвечающие за синхронизацию базы, шифрование и парольную фразу:

user/utils/ DesFileCipher.java (Шифрование и расшифровка файлов с записью)

```
package com.cryptoauth.infosecurity.user.utils;

import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

import javax.crypto.*;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

import java.io.*;

public class DesFileCipher {

    private static final String TRANSFORMATION = "DES/CBC/PKCS5Padding";

    // фиксированный IV (8 байт), выполняем по тз.
    private static final byte[] FIXED_IV = new byte[] { 0, 0, 0, 0, 0, 0, 0, 1 };

    private DesFileCipher() {}

    // Генерация ключа DES из парольной фразы (берём первые 8 байт SHA-1 хэша)
    public static SecretKeySpec keyFromPassphrase(String passphrase) {
        try {
            MessageDigest messageDigest = MessageDigest.getInstance("SHA");
            byte[] digest =
messageDigest.digest(passphrase.getBytes(java.nio.charset.StandardCharsets.UTF_8));

            // DES требует 8-байтный ключ -> берём первые 8 байт хэша
            byte[] keyBytes = new byte[8];
            System.arraycopy(digest, 0, keyBytes, 0, 8);

            return new SecretKeySpec(keyBytes, "DES");
        } catch (Exception e) {
            throw new RuntimeException("Cannot generate key from passphrase", e);
        }
    }

    public static void processFile(int mode, File inFile, File outFile, String passphrase) {
        try {
            Cipher cipher = Cipher.getInstance(TRANSFORMATION);

            cipher.init(mode, keyFromPassphrase(passphrase), new IvParameterSpec(FIXED_IV));

            if (!inFile.exists() && mode == Cipher.DECRYPT_MODE)
                throw new FileNotFoundException("Encrypted file not found: " + inFile.getAbsolutePath());
```

```

        try (FileInputStream fis = new FileInputStream(inFile);
            FileOutputStream fos = new FileOutputStream(outFile);
            CipherOutputStream cos = new CipherOutputStream(fos, cipher)) {

            byte[] buffer = new byte[8192];
            int r;

            while ((r = fis.read(buffer)) != -1)
                cos.write(buffer, 0, r);
        } catch (IOException | NoSuchAlgorithmException | NoSuchPaddingException
            | InvalidKeyException | InvalidAlgorithmParameterException e) {
            throw new RuntimeException((mode == Cipher.ENCRYPT_MODE ? "Encryption" :
"Decryption") + " failed", e);
        }
    }

    public static void encryptFile(File inFile, File outFile, String passphrase) {
        processFile(Cipher.ENCRYPT_MODE, inFile, outFile, passphrase);
    }

    public static void decryptFile(File inFile, File outFile, String passphrase) {
        processFile(Cipher.DECRYPT_MODE, inFile, outFile, passphrase);
    }

    // Перезапись данных из базы данных в файл
    public static void overwriteAndDelete(File file) {
        if (file == null || !file.exists()) return;

        try (RandomAccessFile raf = new RandomAccessFile(file, "rw")) {
            long size = raf.length();
            raf.seek(0);
            byte[] zeros = new byte[8192];
            long written = 0;

            while (written < size) {
                int toWrite = (int) Math.min(zeros.length, size - written);
                raf.write(zeros, 0, toWrite);
                written += toWrite;
            }
        } catch (IOException ignored) {}
        file.delete();
    }
}

```

user/config/DatabaseCryptoSyncManager.java (Синхронизация и парольная фраза)

```

package com.cryptoauth.infosecurity.user.config;

import com.cryptoauth.infosecurity.user.entity.Role;
import com.cryptoauth.infosecurity.user.entity.User;
import com.cryptoauth.infosecurity.user.repository.UserRepository;
import com.cryptoauth.infosecurity.user.utils.DesFileCipher;
import com.cryptoauth.infosecurity.user.utils.PasswordHashUtil;
import com.fasterxml.jackson.databind.ObjectMapper;

```

```

import com.fasterxml.jackson.databind.SerializationFeature;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.event.ContextClosedEvent;
import org.springframework.stereotype.Component;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.context.event.EventListener;
import org.springframework.transaction.annotation.Transactional;

import java.io.File;
import java.nio.file.Files;
import java.time.LocalDateTime;
import java.util.*;

@Component
@RequiredArgsConstructor
@Slf4j
public class DatabaseCryptoSyncManager implements CommandLineRunner {

    private final UserRepository userRepository;

    private final JdbcTemplate jdbcTemplate;

    // Сохраним passphrase при старте, чтобы использовать при закрытии
    private volatile String passphrase;

    private final ObjectMapper objectMapper = new ObjectMapper().registerModule(new
JavaTimeModule())
        .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);

    private static final String JSON_FILE = "userdata.json";
    private static final String ENC_FILE = "userdata.enc";

    private final File jsonFile = new File(JSON_FILE);
    private final File encFile = new File(ENC_FILE);

    @Override
    public void run(String... args) throws Exception {
        File jsonFile = new File(JSON_FILE);
        File encFile = new File(ENC_FILE);

        // == Запрос парольной фразы == //
        Scanner scanner = new Scanner(System.in);
        System.out.print("Введите парольную фразу для доступа к данным (Enter = Init!): ");
        this.passphrase = scanner.nextLine().trim();

        // Расшифровка или создание нового файла ==
        if (encFile.exists()) {
            try {
                DesFileCipher.decryptFile(encFile, jsonFile, passphrase);
                log.info("Файл userdata.enc расшифрован для работы");
            } catch (Exception e) {
                log.error("Ошибка расшифровки userdata.enc: неверный пароль или повреждён файл", e);
                return;
            }
        } else {

```



```

        log.warn("Файл userdata.enc не найден. Создаётся новый JSON с администратором по умолчанию");
        createDefaultAdmin(jsonFile);
        DesFileCipher.encryptFile(jsonFile, encFile, passphrase);
        Files.deleteIfExists(jsonFile.toPath());
        log.info("Создан новый зашифрованный файл userdata.enc");
    }

```

```

// === Синхронизация JSON → PostgreSQL === //
if (jsonFile.exists() && Files.size(jsonFile.toPath()) > 0) {
    try {
        User[] usersFromFile = objectMapper.readValue(jsonFile, User[].class);

        for (User u : usersFromFile) {
            u.setId(null); // JPA сгенерирует новый UUID
        }
    }
}

```

```

// Полностью перезаписываем таблицу
userRepository.deleteAll();
userRepository.saveAll(Arrays.asList(usersFromFile));
log.info("Все пользователи из JSON перезаписаны в базе данных");

```

```

    } catch (Exception e) {
        log.error("Ошибка загрузки пользователей из JSON", e);
    }
} else {
    log.warn("JSON-файл отсутствует или пуст. Тогда создаётся администратор по умолчанию");
    if (userRepository.count() == 0) {
        createDefaultAdmin(jsonFile);
        DesFileCipher.encryptFile(jsonFile, encFile, passphrase);
        Files.deleteIfExists(jsonFile.toPath());
        log.info("Создан администратор по умолчанию и зашифрованный файл.");
    }
}
}

```

```

// === Экспорт PostgreSQL -> JSON -> ENC при завершении === //
Runtime.getRuntime().addShutdownHook(new Thread(() -> {
    try {
        List<User> allUsers = userRepository.findAll();
        objectMapper.writerWithDefaultPrettyPrinter().writeValue(jsonFile, allUsers);
        DesFileCipher.encryptFile(jsonFile, encFile, passphrase);
        Files.deleteIfExists(jsonFile.toPath());
        log.info("Данные PostgreSQL экспортированы и зашифрованы в userdata.enc");
    } catch (Exception e) {
        log.error("Ошибка при экспорте PostgreSQL -> ENC", e);
    }
}));
}

```

```

@EventListener(ContextClosedEvent.class)
@Transactional
public void onContextClosed(ContextClosedEvent ev) {
    // безопасность, если passphrase не установлен — ничего не делаем
    if (this.passphrase == null) {
        log.warn("Passphrase отсутствует — пропускаем экспорт/очистку.");
        return;
    }
}

```

```

try {
    // Экспортируем snapshot из БД в JSON
    List<User> allUsers = userRepository.findAll();
    objectMapper.writerWithDefaultPrettyPrinter().writeValue(jsonFile, allUsers);
    log.info("Экспортировано {} пользователей в {}", allUsers.size(), jsonFile.getAbsolutePath());

    // Шифруем JSON -> userdata.enc
    DesFileCipher.encryptFile(jsonFile, encFile, this.passphrase);
    Files.deleteIfExists(jsonFile.toPath());
    log.info("JSON зашифрован в {}", encFile.getAbsolutePath());

    // Очищаем таблицу через JPA
    userRepository.deleteAll();
    log.warn("Все записи пользователей удалены");

    clearDatabase();
    log.warn("Таблица users сброшена");

} catch (Exception ex) {
    log.error("Ошибка при экспорте/шифровании/очистке при закрытии контекста", ex);
}

// === Очистка базы данных === //
private void clearDatabase() {
    try {
        // Очистка таблицы пользователей
        userRepository.deleteAll();
        log.warn("Все пользователи в базе данных удалены (выполнена очистка PostgreSQL)");

        jdbcTemplate.execute("DROP TABLE IF EXISTS users CASCADE");
        // jdbcTemplate.execute("TRUNCATE TABLE users RESTART IDENTITY CASCADE");
        log.warn("Таблица 'users' удалена и очищена из базы данных.");
    } catch (Exception e) {
        log.error("Ошибка при очистке базы данных", e);
    }
}

// === Вспомогательный метод для создания админа === //
private void createDefaultAdmin(File jsonFile) {
    try {
        String salt = PasswordHashUtil.generateSalt();
        String hash = PasswordHashUtil.sha1Hex("admin", salt);

        User admin = User.builder()
            .email("admin@example.com")
            .password_hash(hash)
            .salt(salt)
            .role(Role.ADMIN)
            .accountLocked(false)
            .failedAttempts(0)
            .passwordRestrictions(false)
            .passwordExpirationMonths(0)
            .minPasswordLength(0)
            .emailConfirmed(false)
            .firstLogin(false)
            .passwordLastChanged(LocalDateTime.now())
    
```

```

        .build();

        objectMapper.writerWithDefaultPrettyPrinter().writeValue(jsonFile, List.of(admin));
        log.info("Создан JSON с администратором по умолчанию");
    } catch (Exception e) {
        log.error("Ошибка при создании администратора по умолчанию", e);
    }
}
}
}

```

Демонстрация работы функционала шифрования и дешифрования файла, синхронизации с базой данных.

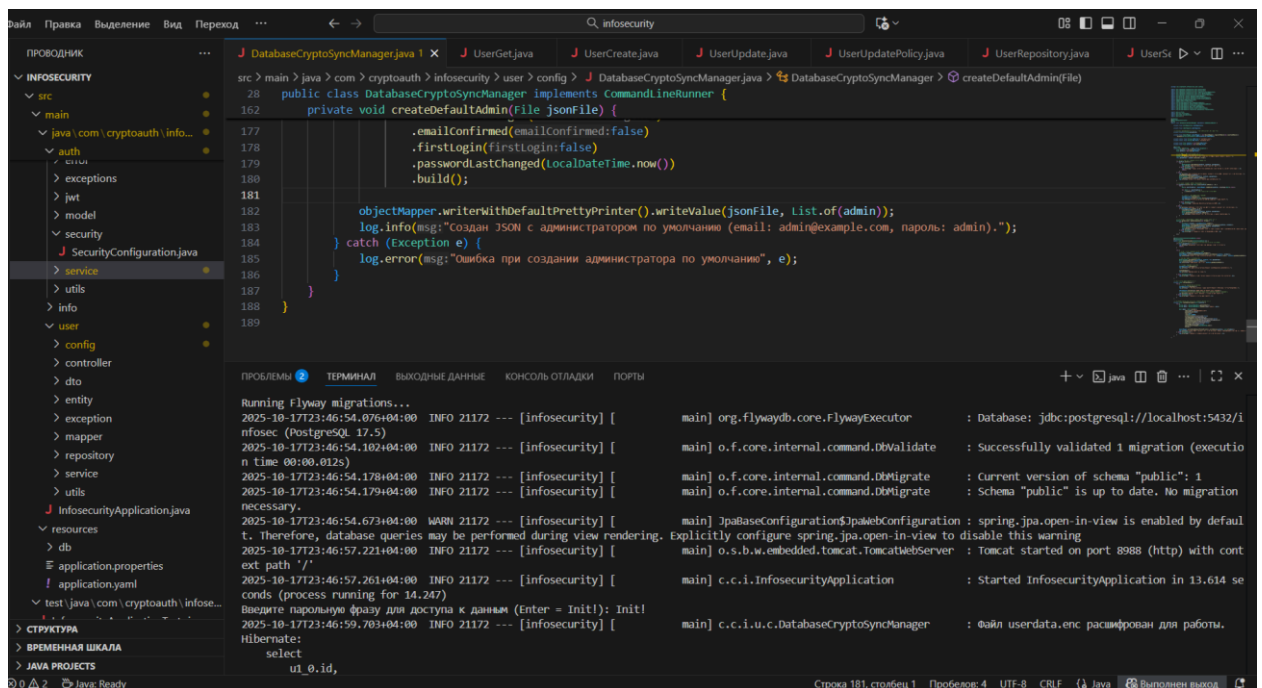


Рисунок 2. Парольная фраза (1)

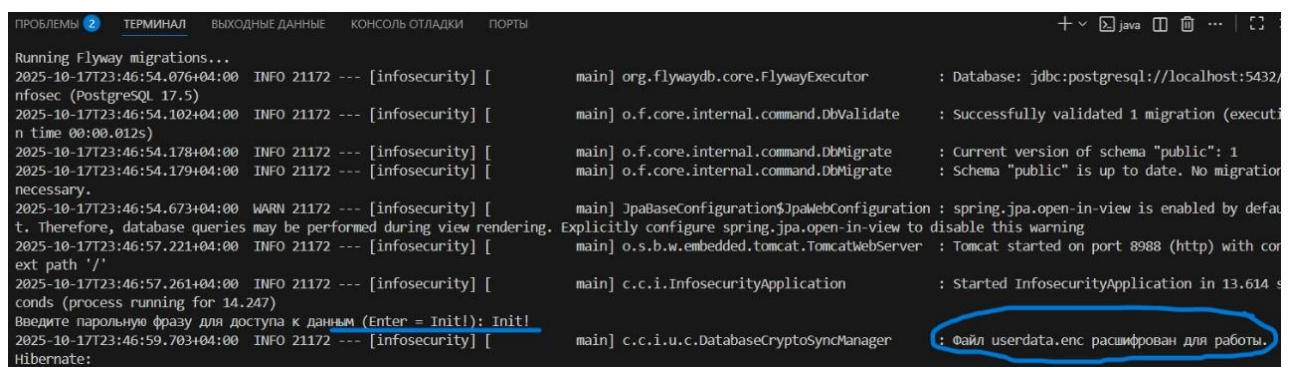


Рисунок 3. Парольная фраза (2)

На данных изображениях показывается работа парольной фразы для расшифровки файла с дальнейшим использованием. Далее демонстрируется зашифрованный файл после выключения программной системы.

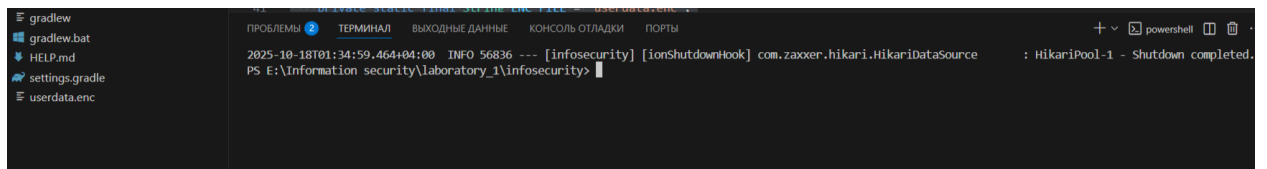


Рисунок 3. Выключенное состояние.

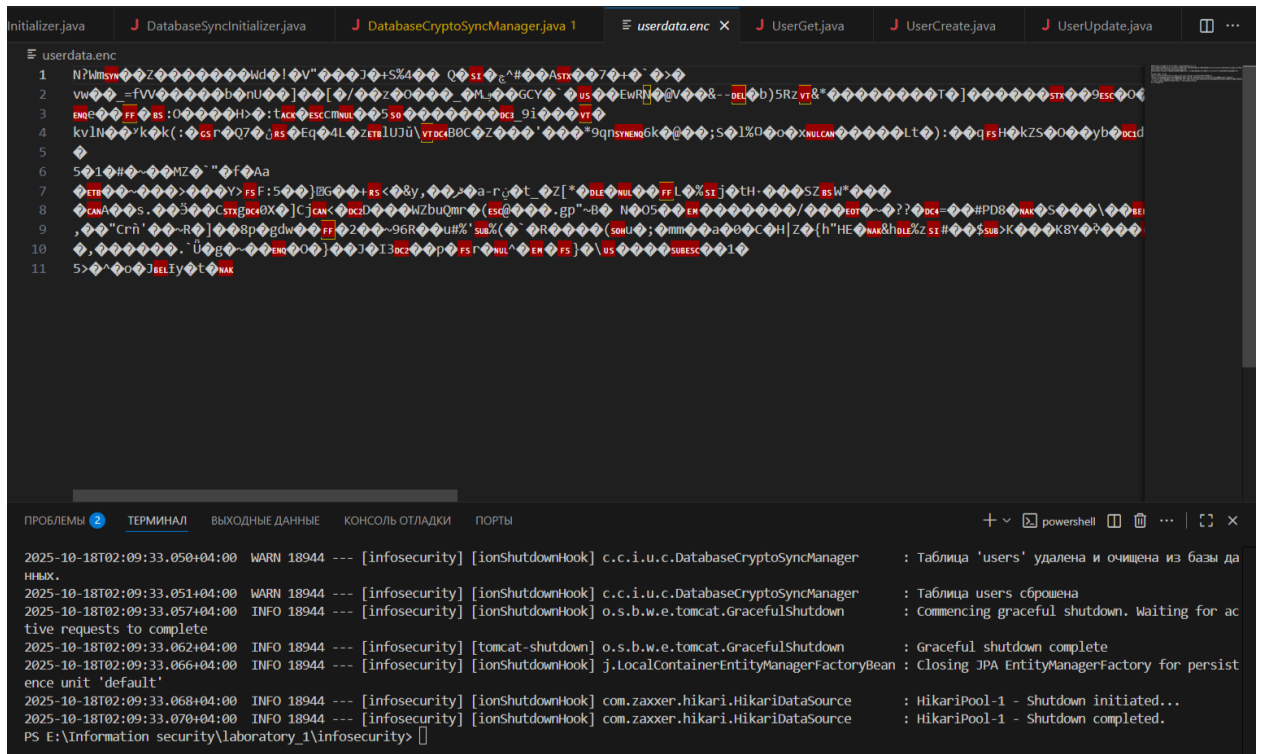


Рисунок 4. Зашифрованный файл.

При этом все изменения в базе данных отображаются в файле локальном и при выключении системы, шифруется, тем самым у нас всегда есть актуальные данные в зашифрованном и спрятанном виде, а база данных пуста.

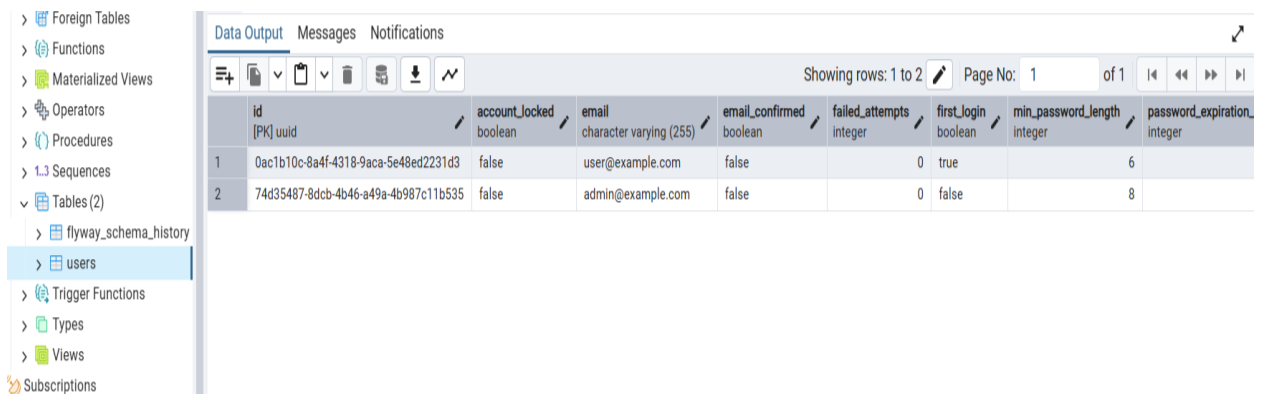


Рисунок 5. Актуальные данные в базе данных PostgreSQL.

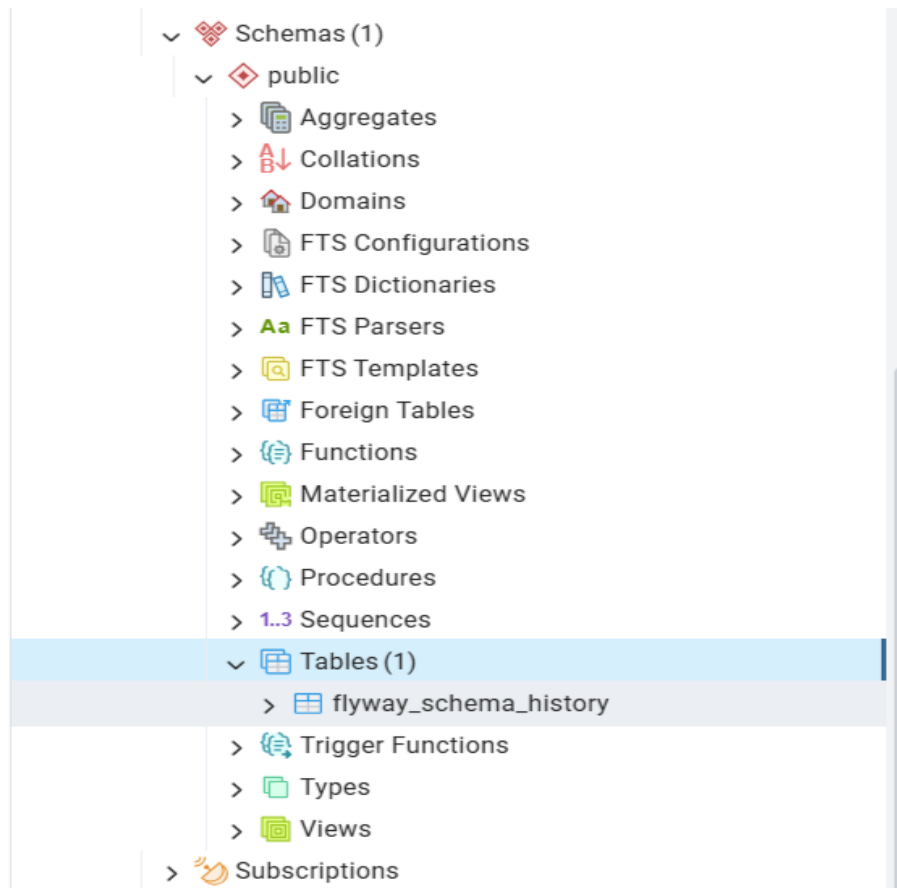


Рисунок 6. Таблица пользователей отсутствует.

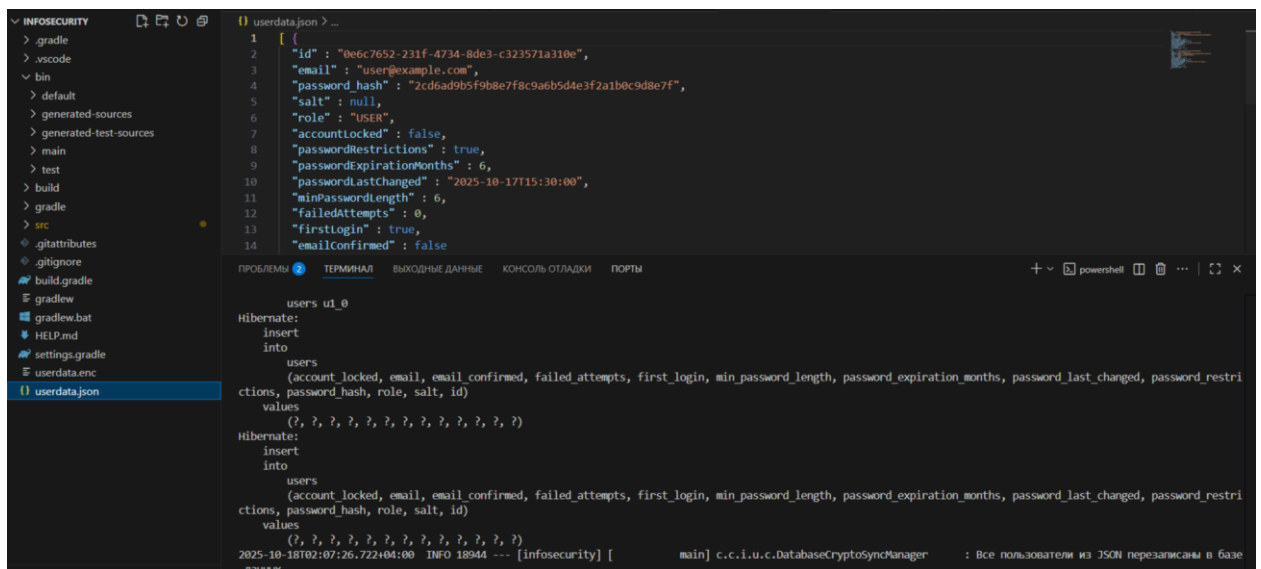


Рисунок 7. Расшифровка файла и синхронизация.

Для обеспечения надёжной аутентификации и авторизации в системе используется **stateless-подход**⁷ на основе JWT в связке с Spring Security. Это

⁷ **Stateless** (без сохранения состояния) — архитектура ПО, которая не сохраняет информацию о предыдущих состояниях или сеансах.

позволяет избежать хранения сессий на сервере, упрощает масштабирование и обеспечивает единый механизм доступа как для веба, так и для десктоп-клиентов.

Аутентификация происходит следующим образом:

Пользователь отправляет учётные данные (email + password) на эндпоинт. Сервер проверяет корректность данных, учитывая политики безопасности: подтверждён ли email, не заблокирован ли аккаунт. При успешной проверке генерируется **JWT access token**, подписанный с использованием HS256 и секретного ключа из переменных окружения.

```
src > main > resources > ! application.yml
1  jwt:
2    secret: VGhpcy1pc0Etc2VjcmV0S2V5LXlvdS1zaG91bGQtdXNlIQ==
```

Рисунок 8. Токен.

Ключевые меры безопасности в реализации JWT. Секретный ключ никогда не хранится в коде — только в application.yml. В токен не включаются чувствительные данные (пароли, соль, внутренние ID сессий). Срок жизни токена ограничен (рекомендуется 15–30 минут). В моей реализации токен 10 часов стоит, просто в рамках лабораторной работы.

Реализация JWT

JwtProperties для конфигурации токена

```
package com.cryptoauth.infosecurity.auth.jwt;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.context.annotation.Configuration;

import lombok.Getter;
import lombok.Setter;

@Configuration
@ConfigurationProperties(prefix = "jwt", ignoreInvalidFields = true)
@Getter
@Setter
public class JwtProperties {
    private String key = "";
```

```

// Base64 encoded secret (256-bit min for HS256)
private String secret;

// Время истечения срока действия токена в миллисекундах
private long expirationMs = 1000 * 60 * 60 * 10; // 10 часов
}

```

JwtService сервис токена

```

package com.cryptoauth.infosecurity.auth.jwt;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;

import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;
import java.util.function.Function;

@Service
@RequiredArgsConstructor
public class JwtService {
    private final JwtProperties jwtProperties;

    private SecretKey secretKey;

    @PostConstruct
    public void init() {
        byte[] keyBytes = Decoders.BASE64.decode(jwtProperties.getSecret());
        this.secretKey = Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateToken(String email, String role){
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
        claims.put("jti", UUID.randomUUID().toString());

        return Jwts.builder()
            .claims()
            .add(claims)
            .subject(email)
            .id(UUID.randomUUID().toString())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtProperties.getExpirationMs()))
            .and()
            .signWith(secretKey)
            .compact();
    }

    private <T> T extractClaim(String token, Function<Claims, T> claimResolver){
        final Claims claims = extractAllClaim(token);
        return claimResolver.apply(claims);
    }
}

```

```

    public Claims extractAllClaim(String token){
        return Jwts.parser()
            .verifyWith(secretKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public boolean validateToken(String token, UserDetails userDetails) {
        final String email = extractEmail(token);
        return (email.equals(userDetails.getUsername()) && !isTokenExpired(token));
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public String extractEmail(String token) {
        return extractClaim(token, Claims::getSubject);
    }
}

```

JwtFilter.java

```

package com.cryptoauth.infosecurity.auth.jwt;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import com.cryptoauth.infosecurity.auth.service.MyUserDetailsService;

import java.io.IOException;
import java.util.List;

@Slf4j
@Component
@RequiredArgsConstructor
public class JwtFilter extends OncePerRequestFilter {

    private static final String AUTHORIZATION = "Authorization";
    private static final String TOKEN_PREFIX = "Bearer ";

    private final JwtService jwtService;
    private final MyUserDetailsService userDetailsService;

```



```

// Пути, которые не нужно фильтровать
private static final List<String> EXCLUDE_URLS = List.of(
    "/swagger-ui", "/swagger-ui.html", "/v3/api-docs", "/v3/api-docs/swagger-config",
    "/h2-console", "/api/auth/initialize", "/api/auth/login"
);

@Override
protected void doFilterInternal(
    @NonNull HttpServletRequest request,
    @NonNull HttpServletResponse response,
    @NonNull FilterChain filterChain
) throws ServletException, IOException {

    String path = request.getRequestURI();

    // Если путь публичный — пропускаем JWT фильтр
    if (EXCLUDE_URLS.stream().anyMatch(path::startsWith) ||
"OPTIONS".equalsIgnoreCase(request.getMethod())) {
        filterChain.doFilter(request, response);
        return;
    }

    try {
        String token = getTokenFromRequest(request);

        if (StringUtils.hasText(token) &&
            SecurityContextHolder.getContext().getAuthentication() == null) {

            String username = jwtService.extractEmail(token);

            if (username != null) {
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);

                if (userDetails != null && jwtService.validateToken(token, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails,
                            null,
                            userDetails.getAuthorities()
                        );

                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(authToken);

                    log.debug("Пользователь {} аутентифицирован", username);
                } else {
                    log.warn("JWT токен не прошёл валидацию");
                }
            }
        }
    } catch (Exception ex) {
        log.error("Ошибка при обработке JWT токена: {}", ex.getMessage());
    }

    filterChain.doFilter(request, response);
}

private String getTokenFromRequest(HttpServletRequest request) {
    String bearer = request.getHeader(AUTHORIZATION);
    if (StringUtils.hasText(bearer) && bearer.startsWith(TOKEN_PREFIX)) {
        return bearer.substring(TOKEN_PREFIX.length());
    }
    return null;
}

```

```
| }  
| }
```

Конфигурация безопасности в программной системе реализована через класс `SecurityConfiguration`, который настраивает цепочку фильтров `Spring Security`. Все запросы защищены через ***SecurityFilterChain*** с отключённым ***CSRF (stateless-архитектура)***. Также для обеспечения безопасности со стороны клиента (веб-сервера) организована политика безопасности ***CORS (Cross-Origin Resource Sharing)***, которая защищает нашу API (веб-сервер) от обращений с других сайтов. В веб-среде браузеры по умолчанию блокируют AJAX-запросы с одного origin на другой, но для легитимных клиентов (например, фронтенда на <https://example.com>) необходимо явно разрешить взаимодействие с API.

Поскольку система использует JWT для аутентификации, сессии на сервере не хранятся, а каждый запрос проверяется независимо на основе токена. Это требует механизма защиты от межсайтовой подделки запроса (CSRF). Браузер не может подделать его без доступа к клиентскому коду, поэтому `csrf().disable()` является обоснованным и безопасным решением. Все HTTP-запросы по умолчанию защищены и разрешены только те эндпоинты, где установлен `permitAll()`. Любой другой запрос требует валидного JWT-токена. Запросы перехватываются `AuthenticationEntryPoint`, который возвращает ошибки с кодом 401, 403. Разрешены необходимые HTTP-методы (GET, POST, PUT, DELETE, OPTION, PATCH). Так `SecurityConfiguration` формирует **многоуровневую защиту** на уровне сети (HTTPS), CORS, аутентификации (JWT), авторизации (URL + методы) и обработки защиты.

SecurityConfiguration.java реализация защиты

```
package com.cryptoauth.infosecurity.auth.security;  
  
import com.cryptoauth.infosecurity.auth.jwt.JwtFilter;  
import lombok.RequiredArgsConstructor;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.http.HttpMethod;  
import org.springframework.security.authentication.AuthenticationManager;
```

```
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import org.springframework.web.filter.CorsFilter;
```

```
import java.util.List;
```

```
// Конфигурация Spring Security для системы управления доступом
```

```
@Configuration
```

```
@EnableWebSecurity
```

```
@EnableMethodSecurity(securedEnabled = true)
```

```
@RequiredArgsConstructor
```

```
public class SecurityConfiguration {
```

```
    private final JwtFilter jwtFilter;
```

```
    @Bean
```

```
    SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
```

```
        // Отключаем CSRF (используем JWT)
```

```
        httpSecurity.csrf(AbstractHttpConfigurer::disable);
```

```
        // H2 Console frames
```

```
        httpSecurity.headers(headers ->
```

```
            headers.frameOptions(frame -> frame.sameOrigin())
```

```
        );
```

```
        // CORS конфигурация
```

```
        httpSecurity.cors(cors -> cors.configurationSource(request -> {
```

```
            CorsConfiguration config = new CorsConfiguration();
```

```
            config.setAllowCredentials(true);
```

```
            config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173"));
```

```
            config.setAllowedHeaders(List.of("*"));
```

```
            config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
```

```
            config.setMaxAge(3600L);
```

```
            return config;
```

```
        }));
```

```
        // Stateless сессии (JWT)
```

```
        httpSecurity.sessionManagement(session ->
```

```
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

```
        );
```

```
        // Правила авторизации
```

```
        httpSecurity.authorizeHttpRequests(requests -> requests
```

```
            // Preflight запросы
```

```
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
```

```
            // Swagger и API Docs публичны
```

```
            .requestMatchers(
```

```
                "/swagger-ui/**",
```

```
                "/v3/api-docs/**",
```

```
                "/swagger-ui.html"
```

```
            ).permitAll()
```

```
            // Публичные эндпоинты (инициализация и логин)
```

```

        .requestMatchers(
            "/api/v1/auth/**",
            "/api/v1/info/about",
            "/h2-console/**"
        ).permitAll()

        // Все остальные требуют аутентификации
        .anyRequest().authenticated()
    );

    // Добавляем JWT фильтр
    httpSecurity.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);

    return httpSecurity.build();
}

@Bean
public CorsFilter corsFilter() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return new CorsFilter(source);
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}
}

```

Для передачи данных между слоями и клиентом используются DTO в виде Java record. Они иммутабельны, что исключает случайные изменения после создания, вдобавок они могут содержать только те поля, которые необходимо менять и автоматически реализуют equals(), hashCode(), toString() и прочие реализации Getter/Setter без кода.

DTO модели

```

package com.cryptoauth.infosecurity.user.dto;
public record UserCreate(String email) {}

package com.cryptoauth.infosecurity.user.dto;
public record UserChangePassword(String oldPassword, String newPassword, String confirmPassword) {}

package com.cryptoauth.infosecurity.user.dto;
import java.time.LocalDateTime;
import java.util.UUID;
import com.cryptoauth.infosecurity.user.entity.Role;

public record UserGet(UUID id,

```

```
String email, Role role, String password_hash, String salt, Boolean accountLocked,
Boolean passwordRestrictions, Integer passwordExpirationMonths, LocalDateTime passwordLastChanged,
Integer minPasswordLength, Integer failedAttempts, Boolean firstLogin, Boolean emailConfirmed
) {}
```

```
package com.cryptoauth.infosecurity.user.dto;
import java.time.LocalDateTime;
import com.cryptoauth.infosecurity.user.entity.Role;
```

```
public record UserUpdate(String email, Role role, String password_hash, String salt, Boolean accountLocked,
Boolean passwordRestrictions, Integer passwordExpirationMonths, LocalDateTime passwordLastChanged,
Integer minPasswordLength, Integer failedAttempts, Boolean firstLogin, Boolean emailConfirmed
) {}
```

Преобразование сущностей в DTO выполняется с помощью MapStruct. Генерация маппинга происходит на этапе компиляции — нет рефлексии, нет риска NullPointerException. Можно явно определить, какие поля копировать, что предотвращает утечку конфиденциальных данных при добавлении новых полей в сущность. Mapper интегрируется как Spring-компонент, что упрощает внедрение зависимостей и тестирование.

MapperStruct для маппинга

```
package com.cryptoauth.infosecurity.user.mapper;

import java.util.List;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;
import org.mapstruct.MappingConstants;

import com.cryptoauth.infosecurity.user.dto.UserCreate;
import com.cryptoauth.infosecurity.user.dto.UserGet;
import com.cryptoauth.infosecurity.user.dto.UserUpdate;
import com.cryptoauth.infosecurity.user.entity.User;

@Mapper(componentModel = MappingConstants.ComponentModel.SPRING)
public interface UserMapper {
    @Mapping(target = "passwordRestrictions", source = "passwordRestrictions") // исправление имени
    @Mapping(target = "firstLogin", source = "firstLogin")
    @Mapping(target = "accountLocked", source = "accountLocked")
    @Mapping(target = "minPasswordLength", source = "minPasswordLength")
    @Mapping(target = "passwordExpirationMonths", source = "passwordExpirationMonths")
    @Mapping(target = "passwordLastChanged", source = "passwordLastChanged")
    @Mapping(target = "role", source = "role")
    @Mapping(target = "id", source = "id")
    @Mapping(target = "email", source = "email")
    UserGet toDto(User user);

    List<UserGet> toDtoList(List<User> users);

    @Mapping(target = "passwordRestrictions", ignore = true)
    @Mapping(target = "firstLogin", ignore = true)
    @Mapping(target = "accountLocked", ignore = true)
```

```

    @Mapping(target = "minPasswordLength", ignore = true)
    @Mapping(target = "passwordExpirationMonths", ignore = true)
    @Mapping(target = "passwordLastChanged", ignore = true)
    @Mapping(target = "role", ignore = true)
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "password_hash", ignore = true)
    @Mapping(target = "salt", ignore = true)
    @Mapping(target = "failedAttempts", ignore = true)
    @Mapping(target = "emailConfirmed", ignore = true)
    User toEntity(UserCreate userCreate);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "passwordRestrictions", ignore = true)
    void updateEntity(@MappingTarget User user, UserUpdate dto);
}

```

Используются аннотации вроде `@NotBlank`, `@Email`, `@Size` позволяет некорректные запросы отклонять до попадания в бизнес-логику. Это защищает от базовых атак (например, переполнения буфера через очень длинный пароль). Сервис `UserService` реализует бизнес-логику с учётом базовых принципов безопасности. Проводится проверка на уникальность и запрет регистрации зарезервированных адресов – это снижает риски обхода проверок и предотвращения коллизии.

UserService.java – сервис бизнес-логики

```

package com.cryptoauth.infosecurity.user.service;

import java.time.LocalDateTime;
import java.util.UUID;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

import com.cryptoauth.infosecurity.user.dto.UserCreate;
import com.cryptoauth.infosecurity.user.dto.UserGet;
import com.cryptoauth.infosecurity.user.dto.UserResponseProfile;
import com.cryptoauth.infosecurity.user.dto.UserUpdate;
import com.cryptoauth.infosecurity.user.entity.Role;
import com.cryptoauth.infosecurity.user.entity.User;
import com.cryptoauth.infosecurity.user.exception.UserNotFoundException;
import com.cryptoauth.infosecurity.user.mapper.UserMapper;
import com.cryptoauth.infosecurity.user.repository.UserRepository;
import com.cryptoauth.infosecurity.user.utils.PasswordHashUtil;

import org.springframework.transaction.annotation.Transactional;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class UserService {

```

```

private final UserRepository userRepository;

private final UserMapper userMapper;

// --- создание нового пользователя ---
@Transactional()
public UserGet createUser(UserCreate userCreate) {
    String email = userCreate.email().toLowerCase().trim();

    if (email.isBlank())
        throw new IllegalArgumentException("Email is required");

    if ("admin".equalsIgnoreCase(email) || "admin@example.com".equalsIgnoreCase(email)) {
        throw new IllegalArgumentException("Cannot create reserved ADMIN user");
    }

    if (userRepository.existsByEmail(email)) {
        throw new IllegalArgumentException("User already exists");
    }

    User user = userMapper.toEntity(userCreate);
    user.setRole(Role.USER);
    user.setFailedAttempts(0);
    user.setAccountLocked(false);
    user.setPasswordExpirationMonths(0);
    user.setMinPasswordLength(0);
    user.setPasswordRestrictions(false);

    return userMapper.toDto(userRepository.save(user));
}

@Transactional(readOnly = true)
public Page<UserGet> getAllUsers(String search, Pageable pageable) {
    Page<User> page = (search == null || search.isBlank())
        ? userRepository.findAll(pageable)
        : userRepository.findByEmailContainingIgnoreCase(search, pageable);

    return page.map(userMapper::toDto);
}

@Transactional(readOnly = true)
public UserGet getUser(UUID id) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("User not found"));
    return userMapper.toDto(user);
}

@Transactional(readOnly = true)
public UserGet getUserByEmail(String email) {
    return userMapper.toDto(userRepository.findByEmail(email)
        .orElseThrow(() -> new UserNotFoundException(email)));
}

@Transactional
public UserGet updateUser(UUID id, UserUpdate userUpdate) {
    User user = userRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("User not found: " + id));

    // Обновление email (если задан)
    if (userUpdate.email() != null && !userUpdate.email().isBlank()) {
        String email = userUpdate.email().toLowerCase().trim();
        if (!email.equals(user.getEmail()) && userRepository.existsByEmail(email)) {
            throw new IllegalArgumentException("Email already in use: " + email);
        }
    }
}

```

```

    }
    user.setEmail(email);
}

// Обновление роли (если задана)
if (userUpdate.role() != null) {
    user.setRole(userUpdate.role());
}

if (userUpdate.password_hash().length() < user.getMinPasswordLength()) {

    throw new IllegalArgumentException("Пароль слишком короткий. Минимальная длина: " +
user.getMinPasswordLength() + " символов.");
}

// Обновление пароля (если задан)
if (userUpdate.password_hash() != null && !userUpdate.password_hash().isBlank()) {
    String password = userUpdate.password_hash();

    // Проверка ограничений на пароль. Если включены ограничения на пароль
    if (user.isPasswordRestrictions()) {

        // Если включены ограничения на пароль
        if (user.isPasswordRestrictions()) {
            // Проверяем отсутствие повторяющихся символов
            for (int i = 0; i < password.length(); i++) {
                for (int j = i + 1; j < password.length(); j++) {
                    if (password.charAt(i) == password.charAt(j)) {
                        throw new IllegalArgumentException(
                            "Пароль содержит повторяющиеся символы. Задайте более сложный пароль.");
                    }
                }
            }
        }

        // Хешируем новый пароль
        String salt = PasswordHashUtil.generateSalt();
        String hashedPassword = PasswordHashUtil.sha1Hex(password, salt);

        user.setSalt(salt);
        user.setPassword_hash(hashedPassword);

        user.setPasswordLastChanged(LocalDate.now());
    }

    // Прочие обновления
    if (userUpdate.accountLocked() != null) {
        user.setAccountLocked(userUpdate.accountLocked());
    }

    if (userUpdate.failedAttempts() != null) {
        user.setFailedAttempts(userUpdate.failedAttempts());
    }

    if (userUpdate.passwordExpirationMonths() != null) {
        user.setPasswordExpirationMonths(userUpdate.passwordExpirationMonths());
    }

    if (userUpdate.minPasswordLength() != null) {
        user.setMinPasswordLength(userUpdate.minPasswordLength());
    }
}

```



```

    }

    if (userUpdate.passwordRestrictions() != null) {
        user.setPasswordRestrictions(userUpdate.passwordRestrictions());
    }

    if (userUpdate.firstLogin() != null) {
        user.setFirstLogin(userUpdate.firstLogin());
    }

    if (userUpdate.emailConfirmed() != null) {
        user.setEmailConfirmed(userUpdate.emailConfirmed());
    }

    return userMapper.toDto(userRepository.save(user));
}

@Transactional
public void deleteUser(String email) {
    if (!userRepository.existsByEmail(email)) {
        throw new UserNotFoundException(email);
    }
    userRepository.deleteByEmail(email);
}

@Transactional(readOnly = true)
public UserResponseProfile getMe(String email) {
    UserGet userGet = userMapper.toDto(userRepository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("User not found with email: " + email)));

    UserResponseProfile userResponseProfile = new UserResponseProfile(userGet.id(), userGet.email(),
        userGet.role());
    return userResponseProfile;
}
}

```

Контроллер UserController следует REST-нотации и в целом не раскрывает внутренние детали системы. Эндпоинт получения информации о пользователе корректно использует объект Authentication для извлечения email из JWT-токена, что исключает необходимость передачи идентификатора вручную.

UserController.java – контроллер (эндпоинты)

```

package com.cryptoauth.infosecurity.user.controller;

import java.util.UUID;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

```

```

import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.security.core.Authentication;

import com.cryptoauth.infosecurity.user.dto.UserCreate;
import com.cryptoauth.infosecurity.user.dto.UserGet;
import com.cryptoauth.infosecurity.user.dto.UserResponseProfile;
import com.cryptoauth.infosecurity.user.dto.UserUpdate;
import com.cryptoauth.infosecurity.user.service.UserService;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1/user")
public class UserController {

    private final UserService userService;

    @GetMapping("/users")
    public Page<UserGet> getAllUsers(@RequestParam(value = "search", required = false) String search, Pageable
pageable) {
        return userService.getAllUsers(search, pageable);
    }

    @PostMapping("/register")
    public UserGet registerUser(@RequestBody UserCreate userCreate){
        return userService.createUser(userCreate);
    }

    @GetMapping("/users/{email}")
    public UserGet findUser(@PathVariable String email) {
        return userService.getUserByEmail(email);
    }

    @GetMapping("/users/user/{email}/id")
    public UserGet getIdUser(@PathVariable UUID id) {
        return userService.getUser(id);
    }

    @PutMapping("/users/{id}")
    public ResponseEntity<UserGet> updateUser(@PathVariable("id") UUID id, @RequestBody UserUpdate
userUpdate) {
        return ResponseEntity.ok(userService.updateUser(id, userUpdate));
    }

    @DeleteMapping("/users/{email}")
    public ResponseEntity<Void> deleteUser(@PathVariable String email) {
        userService.deleteUser(email);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/me")
    public UserResponseProfile getMe(Authentication authentication) {
        // Вытаскивается из токена
        String username = authentication.getName();
        return userService.getMe(username);
    }
}

```

| }

Сервис AuthService реализует многоэтапную аутентификацию с учётом состояния пользователя: проверка блокировки, срока действия пароля, первого входа, подтверждения email. Сервисы в целом следуют принципам информационной безопасности: проверяют email, блокируют аккаунты после неудачных попыток.

AuthService.java – сервис аутентификации

```
package com.cryptoauth.infosecurity.auth.service;

import com.cryptoauth.infosecurity.auth.dto.*;
import com.cryptoauth.infosecurity.auth.jwt.JwtService;
import com.cryptoauth.infosecurity.user.entity.User;
import com.cryptoauth.infosecurity.user.repository.UserRepository;
import com.cryptoauth.infosecurity.user.utils.PasswordHashUtil;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.Date;
import java.util.HashSet;
import java.util.Set;

@Slf4j
@Service
@RequiredArgsConstructor
public class AuthService {

    private final JwtService jwtService;
    private final OtpService otpService;
    private final UserRepository userRepository;

    public LoginResult login(UserLogin userLogin) {

        // Проверка email и существования пользователя
        if (userLogin.email() == null || userLogin.email().isBlank()) {
            throw new RuntimeException("Email cannot be empty");
        }

        User user = userRepository.findByEmail(userLogin.email())
            .orElseThrow(() -> new RuntimeException("Пользователь не найден!"));

        if (user.isAccountLocked()) {
            throw new RuntimeException("Аккаунт заблокирован");
        }

        log.info("Попытка входа пользователя: {}", user.getEmail());

        // Проверка срока действия пароля (опционально)
        if (user.getPasswordLastChanged() != null && user.getPasswordExpirationMonths() > 0) {
            long monthsSinceChange = java.time.temporal.ChronoUnit.MONTHS.between(
                user.getPasswordLastChanged(), LocalDateTime.now());
            if (monthsSinceChange >= user.getPasswordExpirationMonths()) {
                throw new RuntimeException("Пароль истёк");
            }
        }
    }
}
```

```

        log.warn("Срок действия пароля истёк для: {}", user.getEmail());
        // Можно отправлять уведомление или требовать сброс пароля
    }
}

```

```

// Первый вход — пароль не проверяется, сразу токен
if (user.isFirstLogin()) {
    user.setFirstLogin(false);
    userRepository.save(user);
    String token = jwtService.generateToken(user.getEmail(), user.getRole().name());
    return new LoginResult(false, user.getEmail(), token);
}

```

```

// Проверка пароля
boolean passwordOk = PasswordHashUtil.matches(
    userLogin.password(),
    user.getSalt(),
    user.getPassword_hash());

```

```

if (!passwordOk) {
    int failedAttempts = user.getFailedAttempts() + 1;
    user.setFailedAttempts(failedAttempts);
    if (failedAttempts >= 3) {
        user.setAccountLocked(true);
        log.warn("Аккаунт заблокирован: {}", user.getEmail());
        userRepository.save(user);
        throw new RuntimeException("Учётная запись заблокирована из-за 3 неудачных попыток входа.");
    }
    userRepository.save(user);
    throw new RuntimeException("Неверный пароль или логин");
}

```

```

// Сбрасываем счётчик неудачных попыток при успешном входе
if (user.getFailedAttempts() > 0) {
    user.setFailedAttempts(0);
}

```

```

userRepository.save(user);

```

```

// Определяем логику OTP
if (user.isEmailConfirmed()) {
    // Генерируем OTP и сообщаем контроллеру, что нужен OTP
    otpService.generateAndSendOtp(user.getEmail());
    log.info("OTP отправлен пользователю {}", user.getEmail());
    return new LoginResult(true, user.getEmail(), null);
} else {
    // Email не подтверждён — сразу выдаём токен
    String token = jwtService.generateToken(user.getEmail(), user.getRole().name());
    return new LoginResult(false, user.getEmail(), token);
}
}

```

```

public String verifyAndGenerateToken(OtpVerifyRequest request) {
    User user = userRepository.findByEmail(request.email())
        .orElseThrow(() -> new RuntimeException("User not found"));
}

```

```

if (!user.isEmailConfirmed()) {
    log.info("Email не подтверждён — OTP не требуется");
    return jwtService.generateToken(user.getEmail(), user.getRole().name());
}

```

```

boolean verified = otpService.verifyOtp(request.email(), request.otp());
if (!verified)

```

```
throw new RuntimeException("Invalid or expired OTP");
```

```
log.info("OTP успешно подтверждён — выдаём токен для {}", user.getEmail());  
return jwtService.generateToken(user.getEmail(), user.getRole().name());  
}
```

```
public void resetPassword(ResetPasswordRequest request) {  
    User user = userRepository.findByEmail(request.email())  
        .orElseThrow(() -> new RuntimeException("User not found"));
```

```
    // Проверяем старый пароль  
    String oldHash = PasswordHashUtil.sha1Hex(request.oldPassword(), user.getSalt());  
    if (!user.getPassword_hash().equals(oldHash)) {  
        throw new RuntimeException("Старый пароль введён неверно.");  
    }
```

```
    // Проверяем, что новый пароль отличается от старого  
    if (request.newPassword().equals(request.oldPassword())) {  
        throw new RuntimeException("Новый пароль не должен совпадать со старым.");  
    }
```

```
    // Проверка на минимальность пароля!  
    if (request.newPassword().length() < user.getMinPasswordLength()) {  
        throw new IllegalArgumentException("Пароль слишком короткий. Минимальная длина: " +  
user.getMinPasswordLength() + " символов.");  
    }
```

```
    // Проверка уникальности символов  
    if (user.isPasswordRestrictions()) {  
        Set<Character> chars = new HashSet<>();  
        for (char c : request.newPassword().toCharArray()) {  
            if (!chars.add(c)) {  
                throw new IllegalArgumentException(  
                    "Пароль содержит повторяющиеся символы. Задайте более сложный пароль.");  
            }  
        }  
    }
```

```
    String newSalt = PasswordHashUtil.generateSalt();  
    String newHash = PasswordHashUtil.sha1Hex(request.newPassword(), newSalt);
```

```
    user.setSalt(newSalt);  
    user.setPassword_hash(newHash);  
    user.setPasswordLastChanged(LocalDateTime.now());  
    user.setPasswordLastChanged(LocalDateTime.now());  
    user.setFailedAttempts(0);  
    user.setAccountLocked(false);  
    userRepository.save(user);
```

```
    log.info("Пароль успешно обновлён для пользователя {}", user.getEmail());  
}
```

```
public void validToken(String token) {  
    try {  
        var claims = jwtService.extractAllClaim(token);  
        if (claims.getExpiration().before(new Date())) {  
            throw new RuntimeException("Token expired");  
        }  
        log.info("Токен действителен для {}", claims.getSubject());  
    } catch (Exception e) {  
        throw new RuntimeException("Invalid token", e);  
    }
```

```
}
```

```
// ---- Дополнительные методы для контроллера ----
```

```
public void repeatOtpCode(ResendOtpRequest request) {  
    otpService.generateAndSendOtp(request.email());  
    log.info("Повтор OTP для {}", request.email());  
}
```

```
public void forgotPassword(ForgotPasswordRequest request) {  
    otpService.generateAndSendOtp(request.email());  
    log.info("Отправлен OTP для восстановления пароля {}", request.email());  
}
```

```
public void verifyOtpForgotPassword(OtpVerifyRequest request) {  
    boolean verified = otpService.verifyOtp(request.email(), request.otp());  
    if (!verified)  
        throw new RuntimeException("Invalid OTP for password reset");  
    log.info("OTP успешно подтверждён для восстановления {}", request.email());  
}  
}
```

В совокупности архитектура демонстрирует подход к безопасности: разделяются слои ответственности, транзакционность.

AuthController.java – контроллер аутентификации

```
package com.cryptoauth.infosecurity.auth.controller;
```

```
import com.cryptoauth.infosecurity.auth.dto.*;  
import com.cryptoauth.infosecurity.auth.service.AuthService;
```

```
import lombok.RequiredArgsConstructor;  
import lombok.extern.slf4j.Slf4j;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;
```

```
@Slf4j  
@RestController  
@RequiredArgsConstructor  
@RequestMapping("/api/v1/auth")  
public class AuthController {  
    private final AuthService authService;
```

```
    @PostMapping("/login")
```

```
    public ResponseEntity<LoginResult> loginAndSendOtp(@RequestBody UserLogin userLogin) {  
        LoginResult result = authService.login(userLogin);  
        return ResponseEntity.ok(result);  
    }
```

```
    @PostMapping("/resend-otp")
```

```
    public ResponseEntity<String> repeatOtp(@RequestBody ResendOtpRequest request) {  
        authService.repeatOtpCode(request);  
        return ResponseEntity.ok("OTP repeat");  
    }
```

```
    @PostMapping("/forgot-password")
```

```
    public ResponseEntity<String> forgotPassword(@RequestBody ForgotPasswordRequest request) {  
        authService.forgotPassword(request);  
    }
```

```

        return ResponseEntity.ok("OTP sent to email");
    }

    @PostMapping("/verify-otp-forgot-password")
    public ResponseEntity<String> verifyOtpForgotPassword(@RequestBody OtpVerifyRequest request) {
        authService.verifyOtpForgotPassword(request);
        return ResponseEntity.ok("OTP verify success!");
    }

    @PostMapping("/reset-password")
    public ResponseEntity<String> resetPassword(@RequestBody ResetPasswordRequest request) {
        authService.resetPassword(request);
        return ResponseEntity.ok("Password has been reset successfully");
    }

    @PostMapping("/token")
    public ResponseEntity<String> getToken(@RequestBody OtpVerifyRequest request) {
        String token = authService.verifyAndGenerateToken(request);
        return ResponseEntity.ok(token);
    }

    @GetMapping("/valid-token")
    public ResponseEntity<String> validToken(@RequestParam("token") String token) {
        authService.validToken(token);
        return ResponseEntity.ok("Token is valid");
    }
}

```

Класс PasswordHashUtil предназначен для хэширования паролей с использованием соли и последующей проверки соответствия введённого пароля сохранённому хэшу. При создании нового пользователя: генерируется уникальная соль — случайная последовательность из 16 байт с помощью криптографически стойкого генератора SecureRandom. Эта соль кодируется в строку формата Base64 и сохраняется в базе данных. Пароль пользователя объединяется с солью, и к полученной комбинации применяется хэш-функция SHA-1 (в Java вызывается как "SHA"). Результат хэширования преобразуется в шестнадцатеричную строку и также сохраняется в базе данных.

При попытке входа, система извлекает из базы данных соль и сохранённый хэш. Введённый пользователем пароль хэшируется повторно с той же солью с помощью той же функции sha1Hex. Полученный хэш сравнивается со значением из базы (matches). Если строки совпадают — пароль считается верным.

PasswordHashUtil.java – хэширование пароля.

```
package com.cryptoauth.infosecurity.user.utils;

import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

public class PasswordHashUtil {

    private PasswordHashUtil() {}

    // Генерация соли (16 байт)
    public static String generateSalt() {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        return Base64.getEncoder().encodeToString(salt);
    }

    // Хэширование с солью
    public static String sha1Hex(String input, String salt) {
        try {
            // "SHA" - SHA-1
            MessageDigest messageDigest = MessageDigest.getInstance("SHA");
            messageDigest.update(salt.getBytes(java.nio.charset.StandardCharsets.UTF_8));
            byte[] digest = messageDigest.digest(input.getBytes(java.nio.charset.StandardCharsets.UTF_8));

            StringBuilder stringBuilder = new StringBuilder();
            for (byte b : digest) {
                stringBuilder.append(String.format("%02x", b & 0xff));
            }

            return stringBuilder.toString();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    public static boolean matches(String raw, String salt, String hashed) {
        if (raw == null && hashed == null) return true;
        if (raw == null || hashed == null) return false;
        return sha1Hex(raw, salt).equals(hashed);
    }
}
```

Опционально в системе реализован механизм **двухфакторной аутентификации (2FA)** на основе одноразового кода (OTP), доставляемого по электронной почте. Когда требуется подтверждение личности пользователя, вызывается метод `generateAndSendOtp(String email)`. Этот метод генерирует 6-значный числовой код с помощью криптографически стойкого генератора `SecureRandom` и устанавливает срок действия кода — 5 минут от момента генерации. Появляется связка `email`, `otp`, время действия и хранится во внутреннем хранилище `ConcurrentHashMap` — это позволяет

безопасно работать в многопоточной среде. Email Service формирует письмо и отправляет на почту через Spring JavaMail (JavaMailSender)

OtpService.java бизнес-логика писем и проверки

```
package com.cryptoauth.infosecurity.auth.service;

import org.springframework.stereotype.Service;
import com.cryptoauth.infosecurity.auth.model.OtpInfo;

import lombok.RequiredArgsConstructor;

import java.security.SecureRandom;
import java.time.Instant;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

@Service
@RequiredArgsConstructor
public class OtpService {

    private final EmailService emailService;

    private final Map<String, OtpInfo> otpStorage = new ConcurrentHashMap<>();
    private static final long EXPIRATION_TIME_MS = 5 * 60 * 1000;
    private static final SecureRandom random = new SecureRandom();

    public void generateAndSendOtp(String email) {
        String otp = String.format("%06d", random.nextInt(1_000_000));
        long expiration = Instant.now().toEpochMilli() + EXPIRATION_TIME_MS;

        otpStorage.put(email, new OtpInfo(email, otp, expiration));
        emailService.sendOtpEmail(email, otp);
    }

    public boolean verifyOtp(String email, String otp) {
        OtpInfo info = otpStorage.get(email);

        if (info == null) return false;

        boolean valid = info.otp().equals(otp) && Instant.now().toEpochMilli() < info.expirationTime();

        if (valid) {
            otpStorage.remove(email);
        }

        return valid;
    }
}

package com.cryptoauth.infosecurity.auth.model;

public class OtpInfo {

    private final String email;

    private final String otp;
    private final long expirationTime;
```

```

public OtpInfo(String email, String otp, long expirationTime) {
    this.email = email;
    this.otp = otp;
    this.expirationTime = expirationTime;
}

public String email() { return email; }
public String otp() { return otp; }
public long expirationTime() { return expirationTime; }
}

```

EmailService.java – класс формирования письма

```

package com.cryptoauth.infosecurity.auth.service;

import java.time.LocalDateTime;

import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;
import lombok.RequiredArgsConstructor;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;

@Service
@RequiredArgsConstructor
public class EmailService {
    private final JavaMailSender mailSender;

    public void sendOtpEmail(String toEmail, String otpCode) {
        try {
            MimeMessage message = mailSender.createMimeMessage();
            MimeMessageHelper helper = new MimeMessageHelper(message, true, "UTF-8");

            // От кого письмо (имя + email)
            helper.setFrom("Информационная безопасность! <infosecurity@gmail.com>");
            // Почта поддержки
            helper.setReplyTo("infosecurity@gmail.com");
            // Кому
            helper.setTo(toEmail);

            // Тема письма — информативная и ненавязчивая
            helper.setSubject("Ваш уникальный код подтверждения входа");

            // Текст письма — с приветствием, объяснением, деталями и подписью
            StringBuilder emailText = new StringBuilder();

            emailText.append("Здравствуйте! 🖐️ \n\n");
            emailText.append(
                "Мы получили запрос на отправку кода подтверждения для доступа к вашему аккаунту. 📧\n\n");
            emailText.append("Пожалуйста, используйте следующий код для подтверждения вашей личности:\n\n");

            emailText.append(" □ >>> ").append(otpCode).append(" <<< □ \n\n");
            emailText.append("□ Этот код действителен в течение 5 минут. Пожалуйста, не сообщайте его никому.\n\n");

            emailText.append(
                "Если вы не запрашивали этот код, пожалуйста, проигнорируйте это письмо — это может быть угрозой вашей безопасности.\n\n");

```

```

emailText.append(
    "🔒 Для вашей безопасности не пересылайте этот код и не вводите его на подозрительных сайтах.\n\n");

emailText.append(
    "Если у вас возникли вопросы или проблемы, пожалуйста, свяжитесь с нашей службой поддержки:\n");
emailText.append("✉ support@infosecurity@gmail.com\n\n");
emailText.append("С уважением,\n");
emailText.append("Команда infosecurity@gmail.com 👤\n\n");
emailText.append("🕒 Дата и время отправки: ").append(LocalDate.now()).append("\n");

helper.setText(emailText.toString(), false);

mailSender.send(message);
} catch (MessagingException e) {
    throw new RuntimeException("Не удалось отправить email: " + e.getMessage(), e);
}
}
}

```

```

mail:
  host: smtp.gmail.com
  port: 587
  # username: furniturecoup73@gmail.com
  # password: hwba frfa mmbo bpji
  username: clinicapayment@gmail.com
  password: tpwj qnut rehj kfif
  from: clinicapayment@gmail.com

```

Рисунок 9. Конфигурации для MailSender

Демонстрация работы программной системы

При входе в веб-приложение нас встречает окно справочник, которое информирует о том, какой студент предоставляет лабораторную работу на проверку. По кнопке далее мы переходим на окно авторизации, где доступна авторизация по логину и пароли, а также с подтверждением пользователя по коду, отправленному на почту. Админ по роли при прохождении авторизации попадает на панель работы с пользователями: может их добавлять, редактировать у них политики, пароль, а также может удалять. Опционально предусмотрен поиск. Также на формах представлены ошибки связанные с входом в систему (блокировка, неверный пароль и логин).

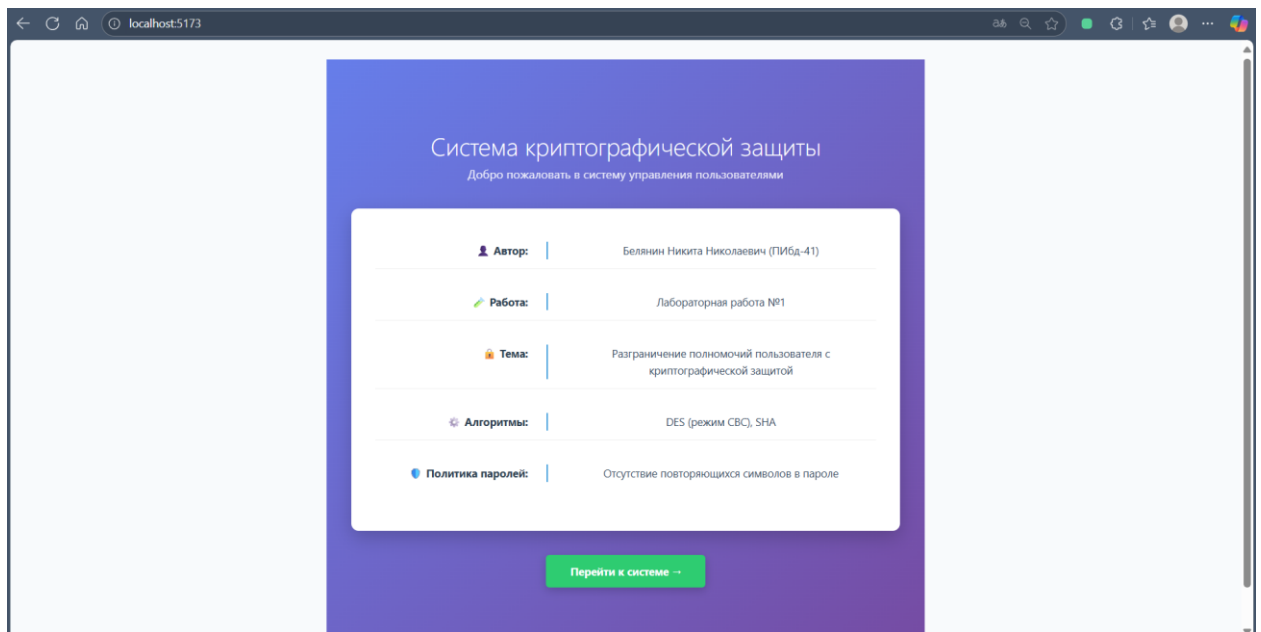


Рисунок 10. Справочная информация студента.

Страница входа, где необходимо ввести логин и пароль. При неверных вводах данных вызывается ошибка авторизации. После 3х попыток аккаунт блокируется.

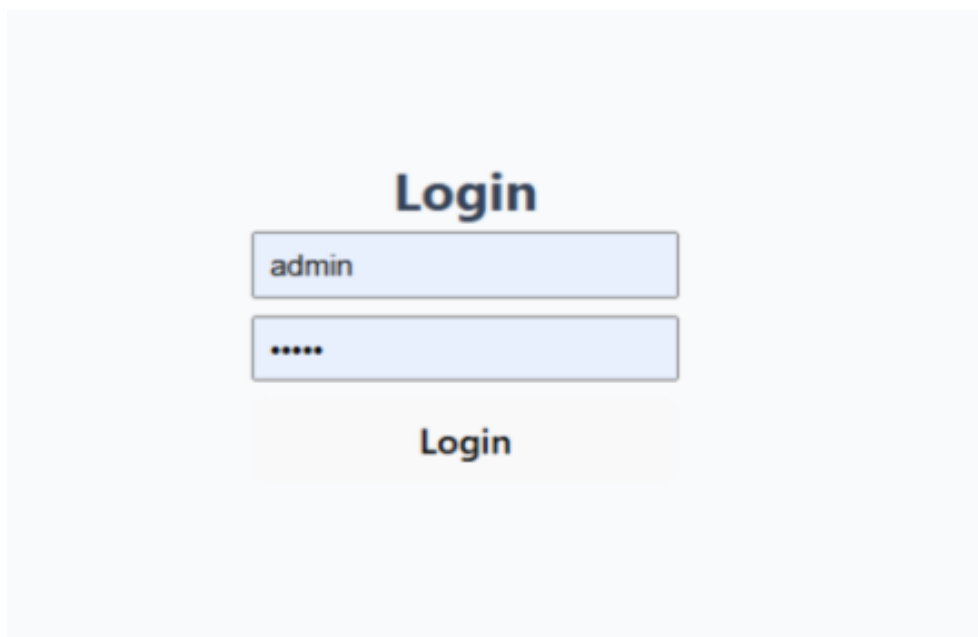


Рисунок 11. Страница авторизации.

Ошибка, связанная с неверным логином не в том формате.



Рисунок 12. Страница авторизации (ошибка с mail).

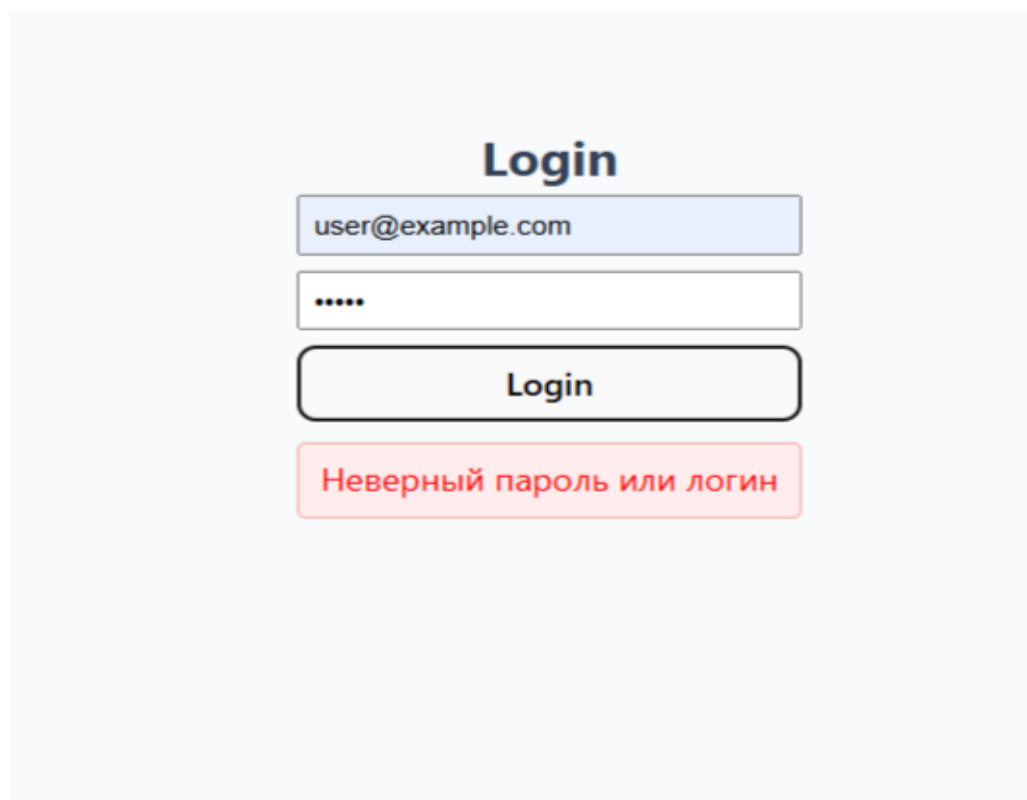


Рисунок 12. Страница авторизации (ошибка входа по паролю и email).

После 3х попыток аккаунт блокируется и об этом сообщается пользователю. При последующих входах, сообщается о блокировке профиля и пройти авторизацию не удаётся.

The image shows a login interface with a light blue background. At the top, the word "Login" is displayed in a bold, dark blue font. Below it, there are two input fields: the first contains the email address "user@example.com" and the second contains five dots representing a password. A rounded rectangular button with a black border and the text "Login" is positioned below the password field. At the bottom, a red rectangular box with rounded corners contains the text "Учётная запись заблокирована из-за 3 неудачных попыток входа." in red.

Рисунок 13. Страница авторизации (ошибка с mail).

The image shows a login interface with a light blue background. At the top, the word "Login" is displayed in a bold, dark blue font. Below it, there are two input fields: the first contains the email address "user@example.com" and the second contains five dots representing a password. A rounded rectangular button with a black border and the text "Login" is positioned below the password field. At the bottom, a red rectangular box with rounded corners contains the text "Аккаунт заблокировн" in red.

Рисунок 14. Страница авторизации (ошибка с mail).

Ниже представлена панель управления пользователями, в ней можно добавить пользователя по email и изменить его пароль вместе с политиками безопасности пароля. Все ошибки приведены ниже. Также можно заблокировать и разблокировать пользователя, а также удалить профиль. Также выводят сообщения ошибки о неверных действиях.

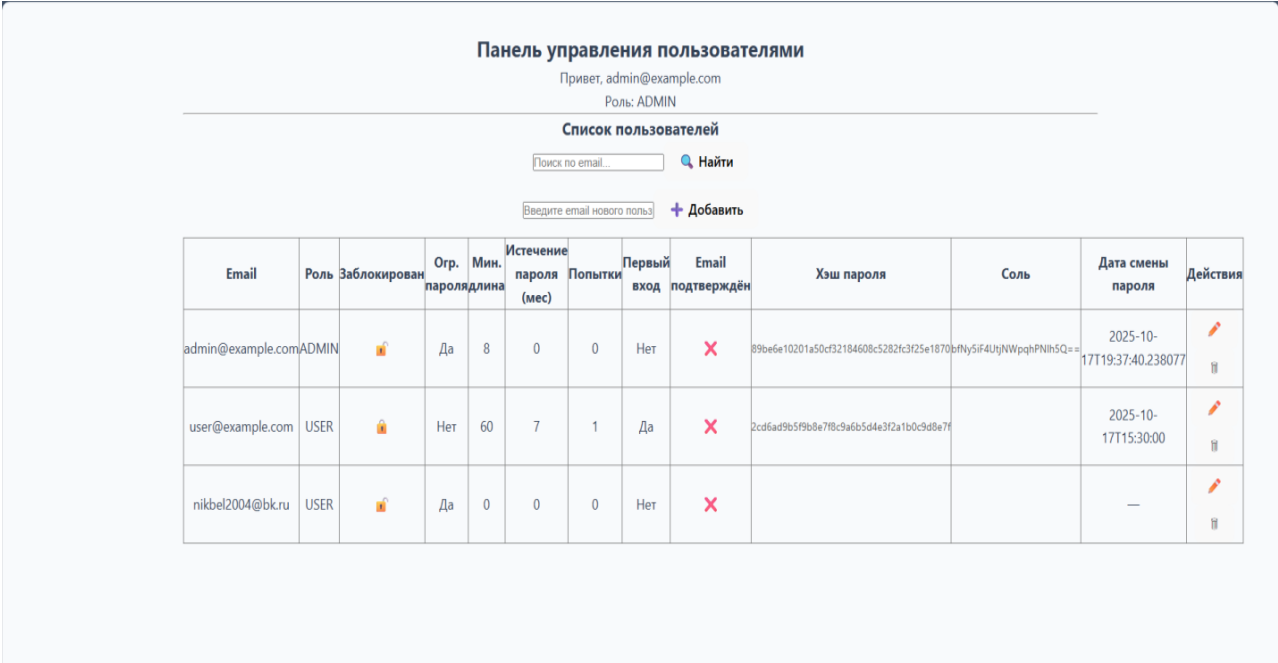


Рисунок 15. Панель управления.

В панель администрирования может попасть только админ. При авторизации проверяется роль пользователя и по ней предоставляются права для админа на операции с пользователями. При редактировании пользователя в таблице разблокируются поля для внесения изменений и на них накладываются проверки.

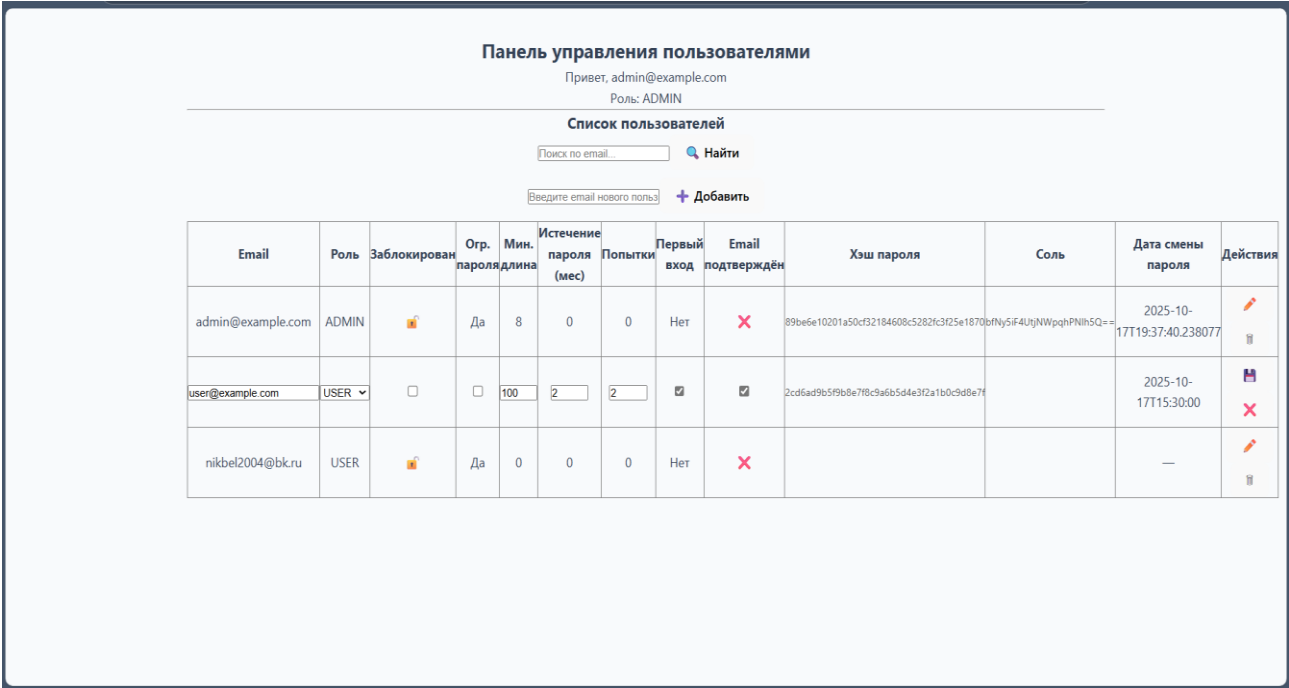


Рисунок 16. Панель управления (редактирования политик).

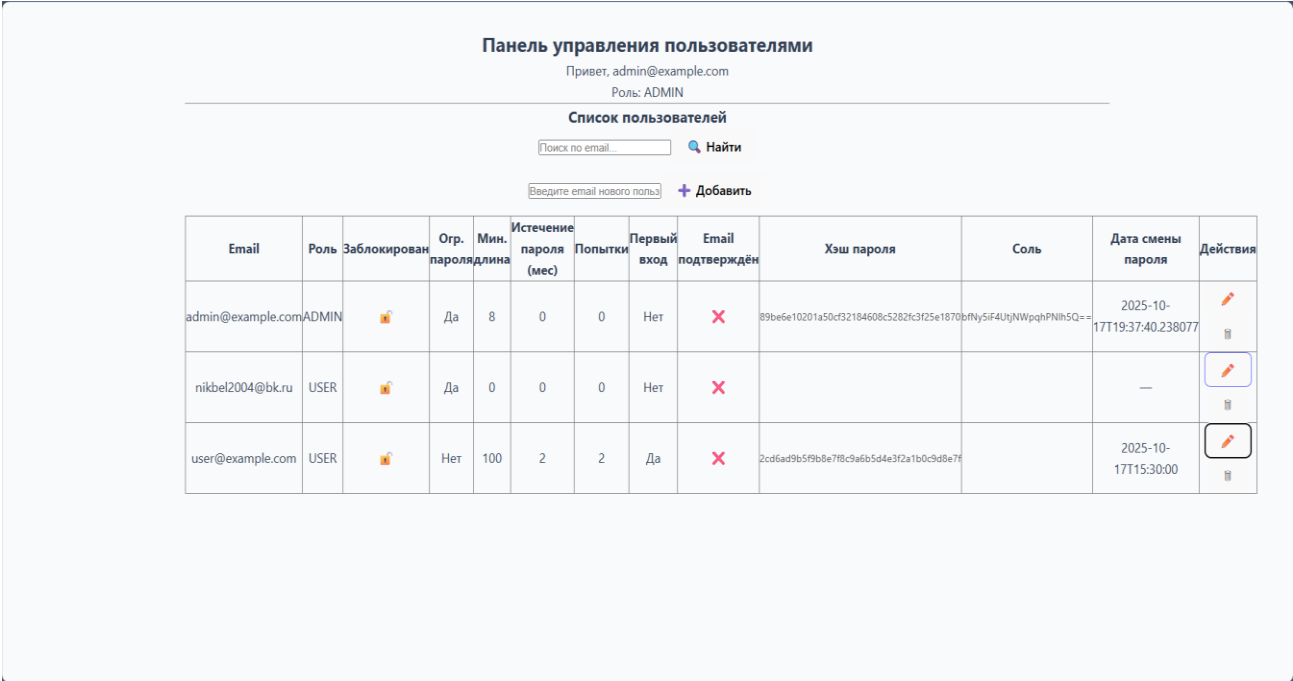


Рисунок 17. Панель управления (результат изменений).

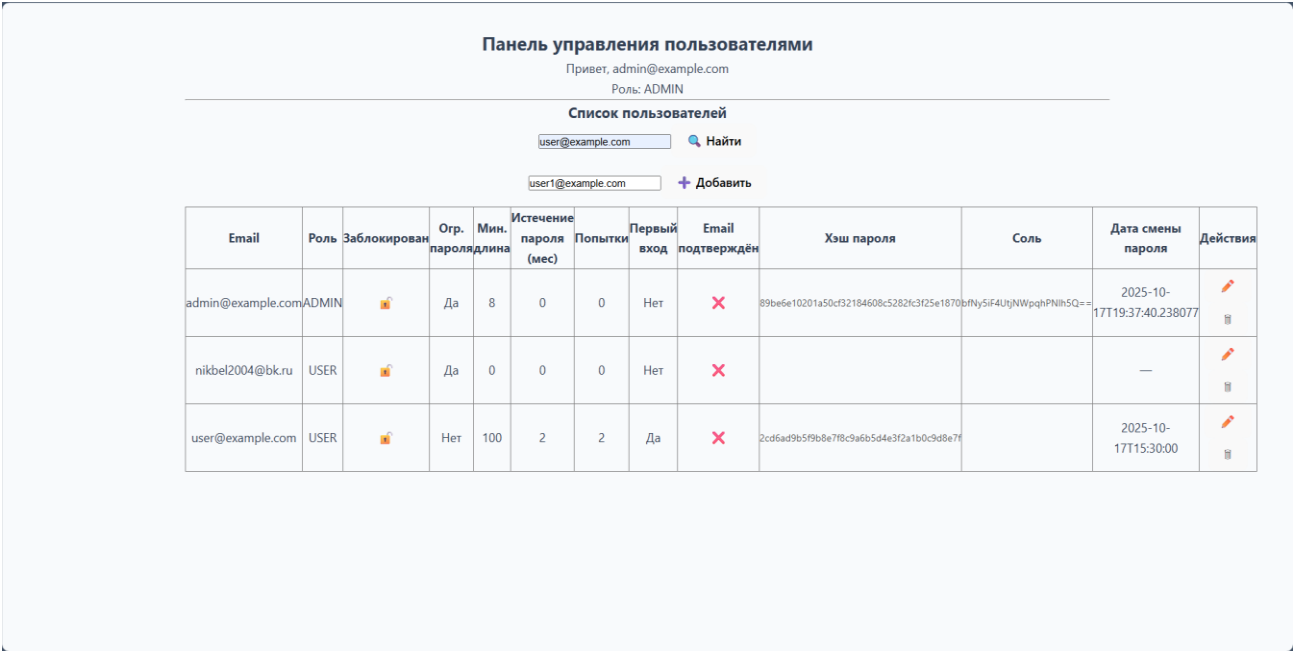


Рисунок 18. Панель управления (добавление пользователя).

user@example.com	USER		Нет	3	2	3	Нет	✗	1c4accfa047dcbdb2b6948b740a6a600b453fb09	fVjs4cay3QOT6t7XRR49lw==	2025-10-19T14:34:43.856772	
------------------	------	--	-----	---	---	---	-----	---	--	--------------------------	----------------------------	--

Рисунок 19. Панель управления (результат блокировки).

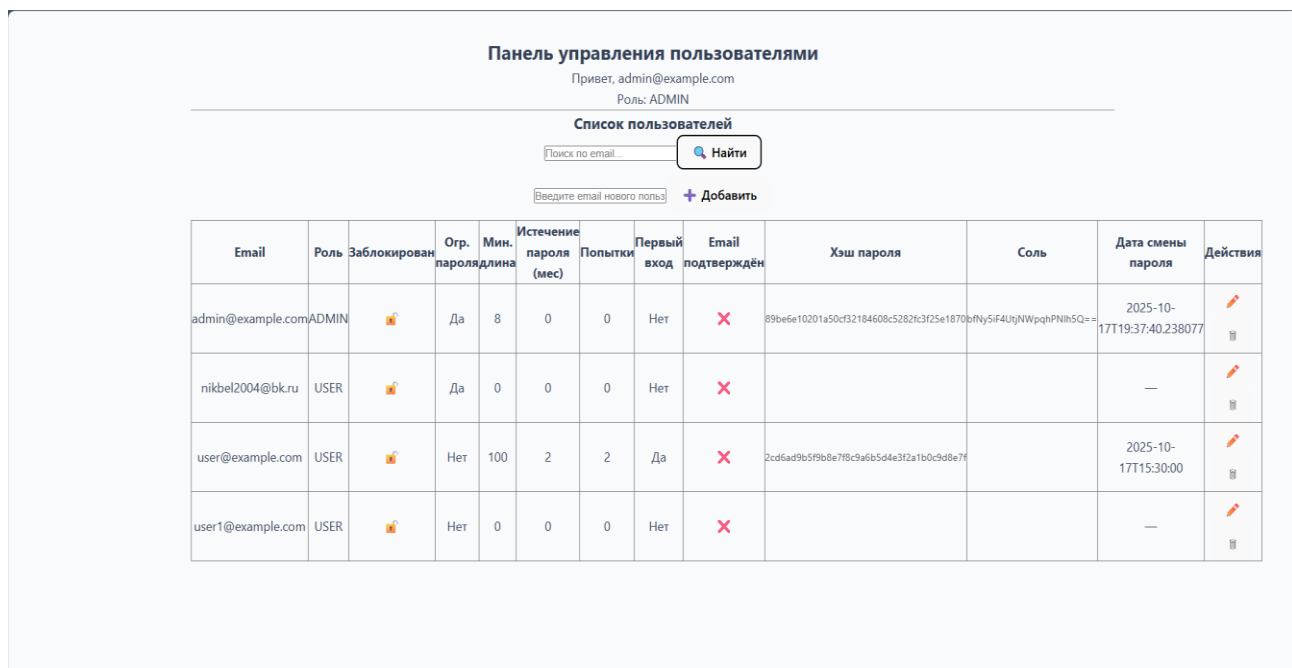


Рисунок 20. Панель управления (результат добавление пользователя).

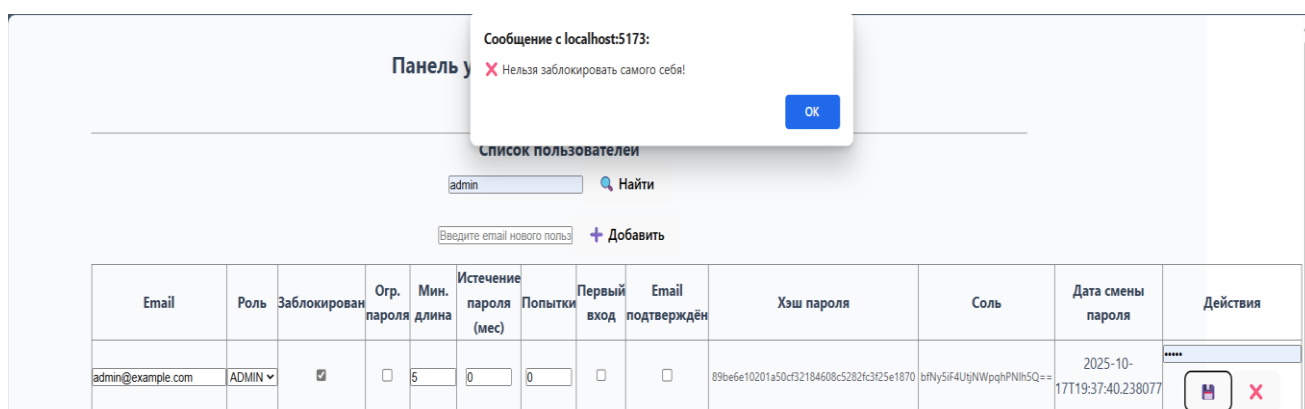


Рисунок 21. Панель управления (админ не блокирует себя).

Внесение политик для профиля. Ограничение на длину пароля, установлено 7 символов, а введено 5. Также наложено ограничение на повторяющиеся символы, однако для действия этого ограничения нужно от лица администратора эти ограничения разрешить.

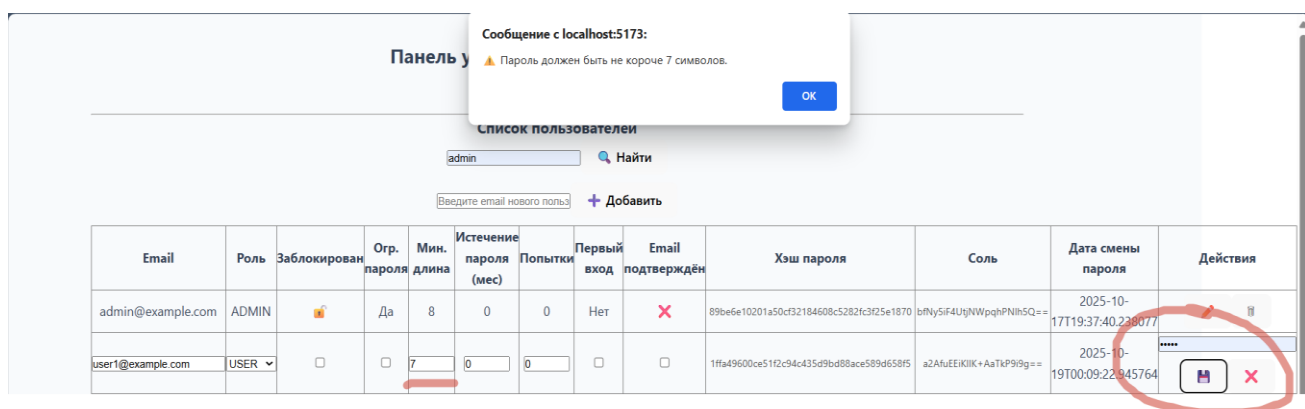


Рисунок 22. Панель управления (минимальный пароль).

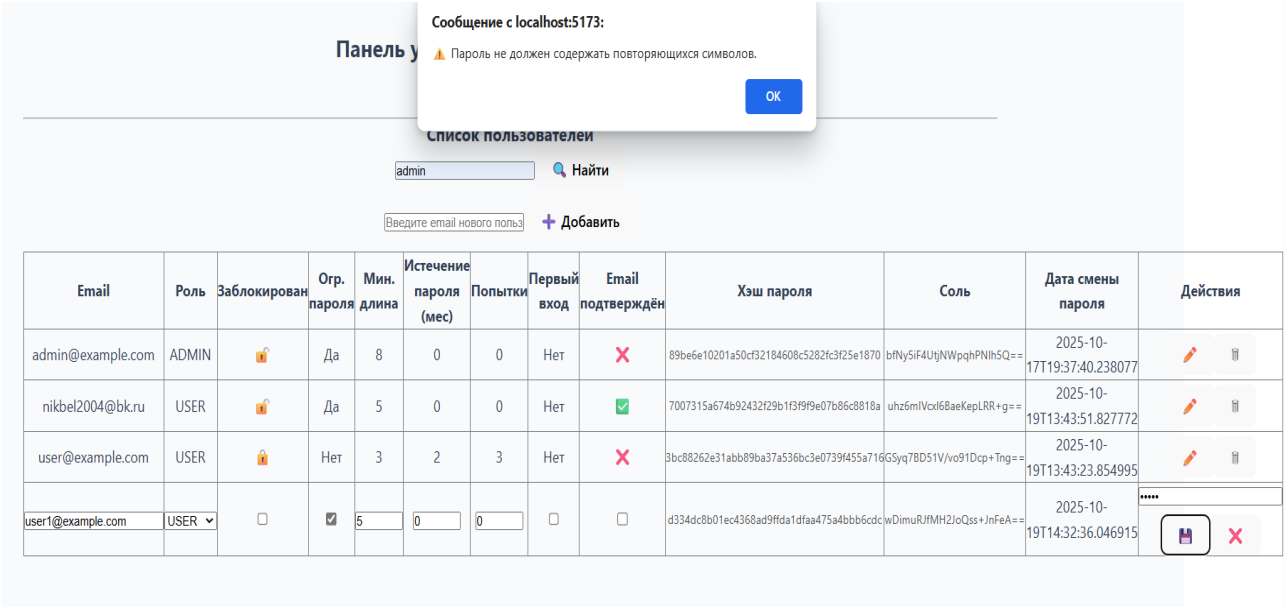


Рисунок 23. Панель управления (политики безопасности пароля) Отсутствие повторяющихся чисел.

Email	Роль	Заблокирован	Огр. пароля	Мин. длина	Истечение пароля (мес)	Попытки	Первый вход	Email подтверждён	Хэш пароля	Соль	Дата смены пароля	Действия
admin@example.com	ADMIN	🔒	Да	8	0	0	Нет	❌	89be6e10201a50cf32184608c5282fc3f25e1870	bfNy5iF4UtgNWpqhPNIh5Q==	2025-10-17T19:37:40.238077	✏️ 🗑️
user@example.com	USER	🔒	Нет	100	2	2	Да	❌	2cd6ad9b5f9b8e7f8c9a6b5d4e3f2a1b0c9d8e7f		2025-10-17T15:30:00	✏️ 🗑️
user1@example.com	USER	🔒	Нет	5	0	0	Нет	❌	1ffa49600ce51f2c94c435d9bd88ace589d658f5	a2AfuEEiKlIK+AAtkP9i9g==	2025-10-19T00:09:22.945764	✏️ 🗑️
nikbel2004@bk.ru	USER	🔒	Да	5	0	0	Нет	✅	e546df91267e9c6eec89d80e811b045f5b15d070	dP993IezStxZApFV6eHjXg==	2025-10-19T11:58:12.338162	✏️ 🗑️

Рисунок 24. Панель управления (результат добавления пароля user1).

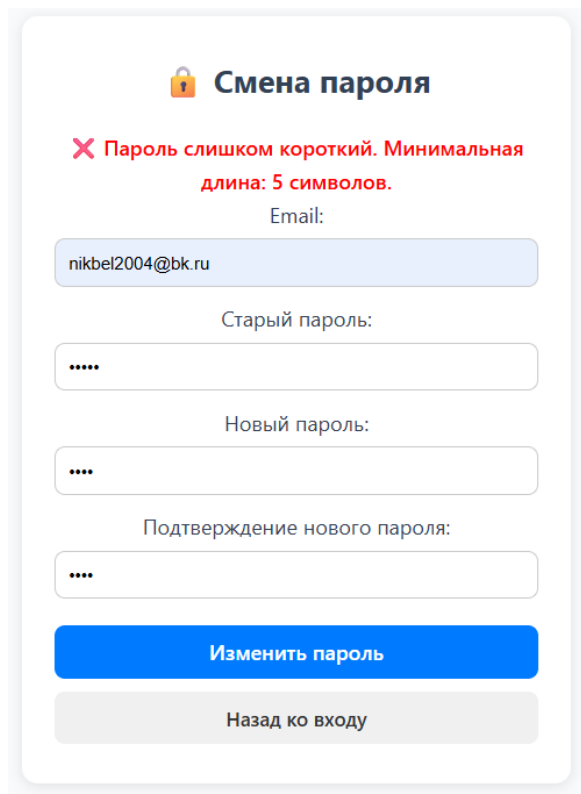
Далее представлены страницы смены пароля доступные обычным пользователям. После регистрации и авторизации от лица пользователя предлагается возможность сменить пароль, для этого система запрашивает предоставить корректный email, для повышения безопасность, ввести старый действительный пароль и ввести новый с последующим подтверждением.

The image shows two identical UI mockups of a password change form, side-by-side. Each form has a title 'Смена пароля' with a lock icon. The first form displays a red error message: '✗ Старый пароль введён неверно.' (Old password entered incorrectly). The second form displays a red error message: '✗ Пароль содержит повторяющиеся символы. Задайте более сложный пароль.' (Password contains repeating symbols. Set a more complex password). Both forms include an email field with 'nikbel2004@bk.ru', and three password fields: 'Старый пароль:', 'Новый пароль:', and 'Подтверждение нового пароля:'. The password fields contain masked characters (dots). At the bottom of each form are two buttons: a blue 'Изменить пароль' (Change password) button and a grey 'Назад ко входу' (Back to login) button.

Рисунок 25. Страница смены пароля (представлены ошибки с неверно введённым старым паролем и с содержанием повторяющихся символов).

The image shows a single UI mockup of a password change form. It has a title 'Смена пароля' with a lock icon. A red error message reads: '✗ Пароли не совпадают.' (Passwords do not match). The form includes an email field with 'nikbel2004@bk.ru', and three password fields: 'Старый пароль:', 'Новый пароль:', and 'Подтверждение нового пароля:'. The password fields contain masked characters (dots). At the bottom are two buttons: a blue 'Изменить пароль' (Change password) button and a grey 'Назад ко входу' (Back to login) button.

Рисунок 26. Страница смены пароля (представлена ошибка, связанная с тем, что не совпадает новый пароль с подтверждённым паролем).



 **Смена пароля**

✗ Пароль слишком короткий. Минимальная длина: 5 символов.

Email:

nikbel2004@bk.ru

Старый пароль:

.....

Новый пароль:

....

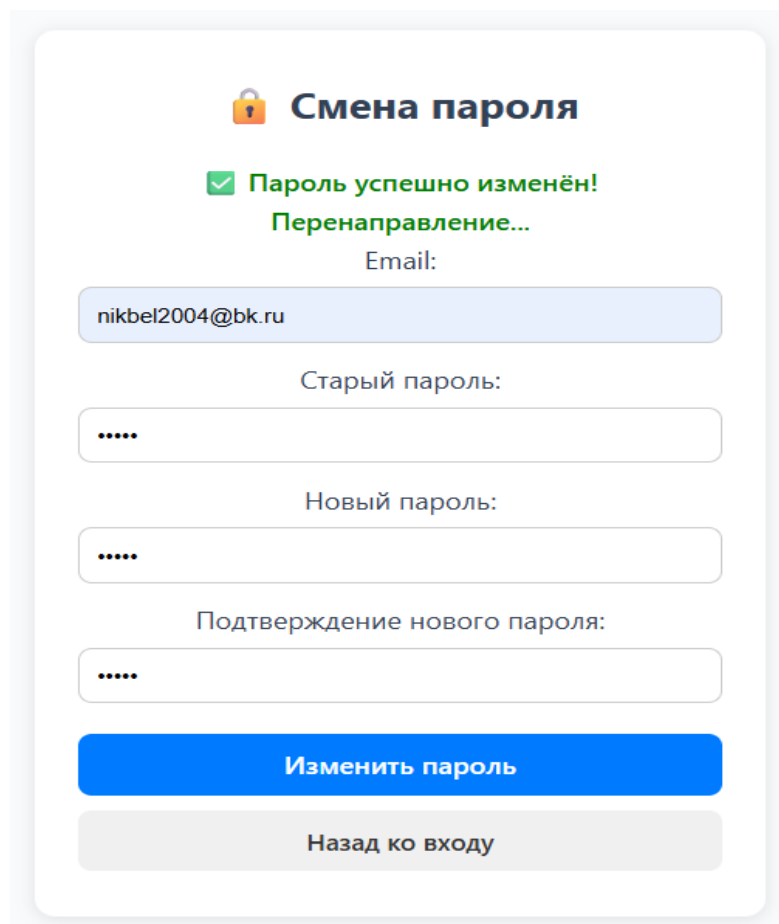
Подтверждение нового пароля:

....

Изменить пароль

Назад ко входу

Рисунок 27. Страница смены пароля (представлена ошибка, связанная с тем, новый пароль не содержит минимальное число цифр, которое должно содержаться в пароле).



 **Смена пароля**

✓ Пароль успешно изменён! Перенаправление...

Email:

nikbel2004@bk.ru

Старый пароль:

.....

Новый пароль:

.....

Подтверждение нового пароля:

.....

Изменить пароль

Назад ко входу

Рисунок 28. Страница смены пароля (успешная смена пароля).

Опционально добавлена возможность усиления профиля. Админ ставить галочку возле подтверждения по email и тут уже на почту летит код с подтверждением, после пользователь может работать с профилем. Это необязательная опция, однако, весомая в безопасности.

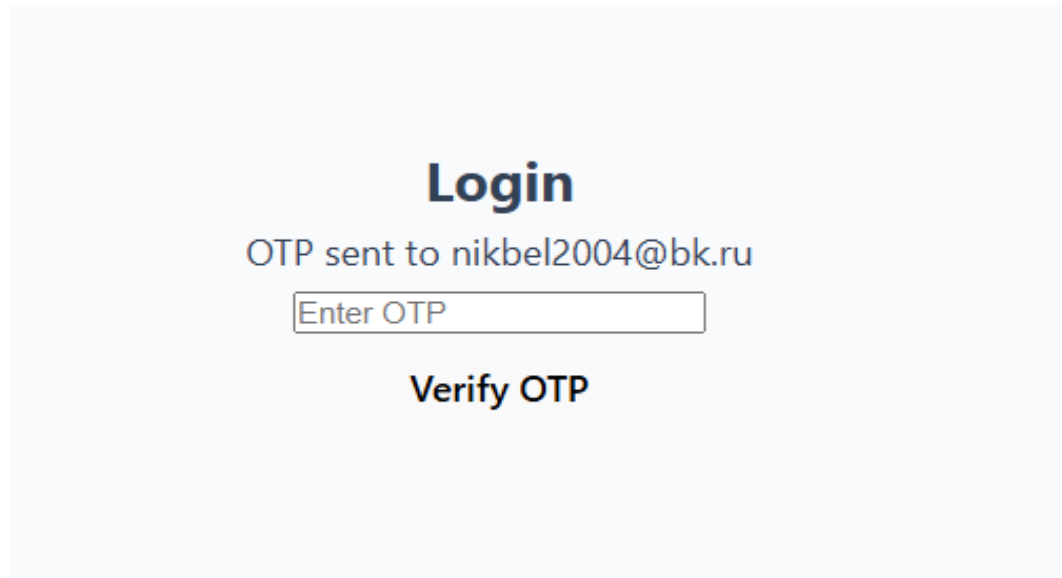


Рисунок 29. Страница с вводом кода подтверждения.

Ниже представлено письмо, которое приходит пользователю при отправке кода подтверждения. Код действителен в течение 5 минут, в случае повторного прохождения авторизации, отправляется новый действующий код, а старый уже не будет корректным. При вводе неверного кода подтверждения появляется ошибка, и пользователь не может попасть внутрь системы.

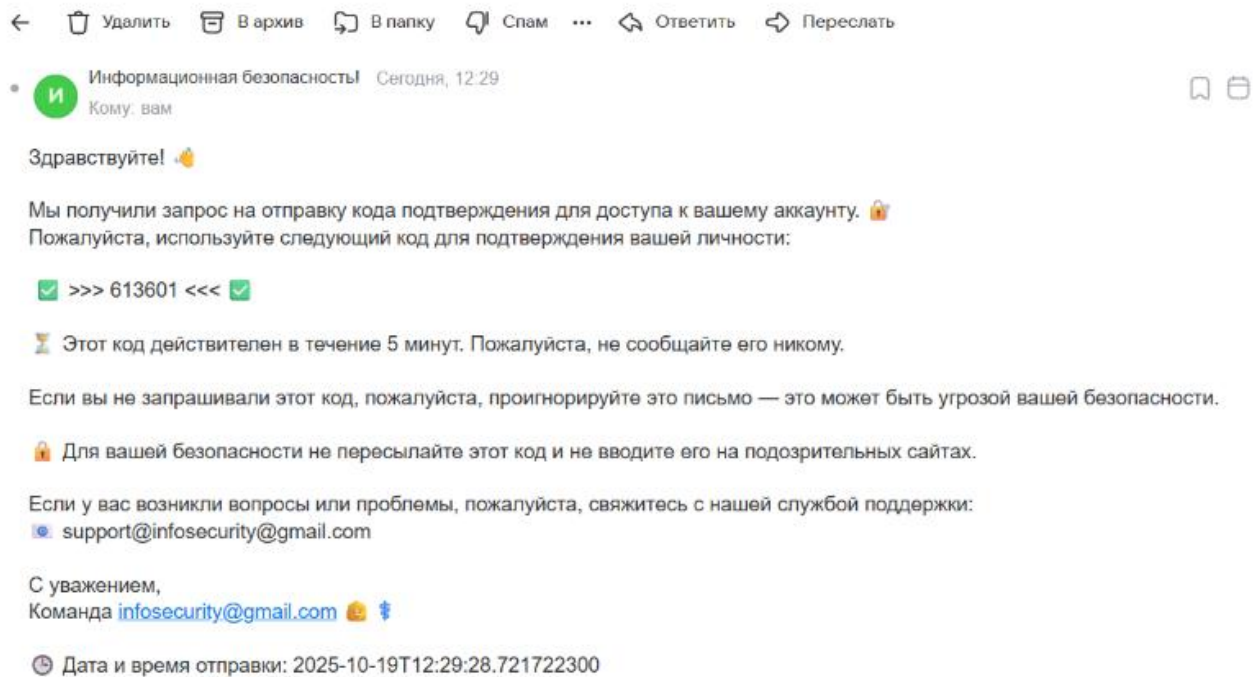


Рисунок 30. Страница с вводом кода подтверждения.

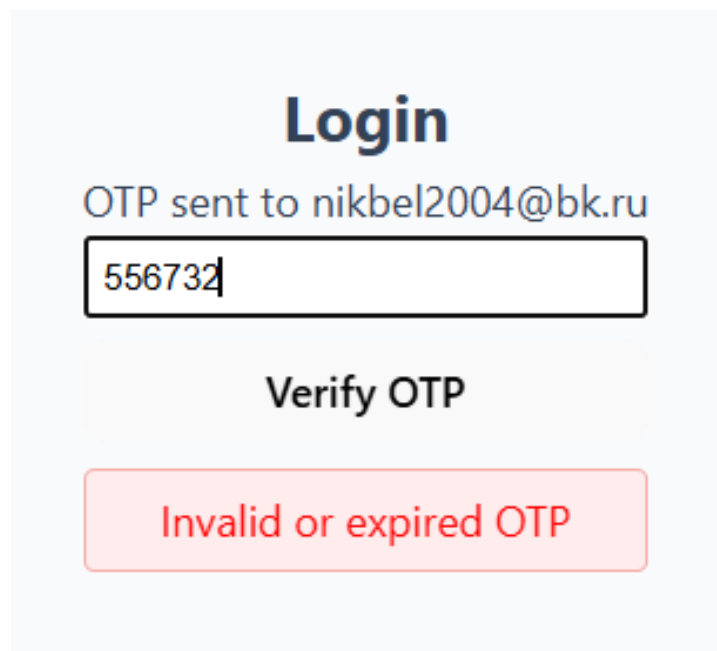


Рисунок 31. Страница с вводом неверного кода подтверждения.

На самом клиенте, есть только обработка ошибок, поля отображение, а вся логика работает на backend сервере, при обращении от frontend. Логика как таковой на Frontend нет. Фрагменты кода основных реализаций представлены ниже:

Clients.ts – обращения к API

```
| import axios, { type AxiosInstance } from 'axios';
```

```
import type { InfoResponse } from '../types/info';
import type { UserCreate, UserGet, UserUpdate } from '../types/user';
import type { AuthResponse, LoginResult, OtpVerifyRequest, UserLogin } from '../types/auth';

const API_BASE = 'http://localhost:8988/api/v1';
```

```
// === Axios-инстансы ===
```

```
export const infoApi = axios.create({ baseURL: `${API_BASE}/info`, timeout: 10000 });
export const userApi = axios.create({ baseURL: `${API_BASE}/user`, timeout: 10000 });
export const authApi = axios.create({ baseURL: `${API_BASE}/auth`, timeout: 10000 });
```

```
// === INFO SERVICE ===
```

```
export const infoService = {
  getAbout: async (): Promise<InfoResponse> => {
    const response = await infoApi.get('/about');
    return response.data;
  },
};
```

```
// === USER SERVICE ===
```

```
export const userService = {
  getAllUsers: async (search?: string, pageable?: any): Promise<any> => {
    const params: any = {};
    if (search) params.search = search;
    if (pageable) Object.assign(params, pageable);
    const response = await userApi.get('/users', { params });
    return response.data;
  },
```

```
  registerUser: async (userData: UserCreate): Promise<UserGet> => {
    const response = await userApi.post('/register', userData);
    return response.data;
  },
```

```
  getUserByEmail: async (email: string): Promise<UserGet> => {
    const response = await userApi.get(`/users/${email}`);
    return response.data;
  },
```

```
  getUserById: async (id: string): Promise<UserGet> => {
    const response = await userApi.get(`/users/user/${id}/id`);
    return response.data;
  },
```

```
  updateUser: async (id: string, userData: UserUpdate): Promise<UserGet> => {
    const response = await userApi.put(`/users/${id}`, userData);
    return response.data;
  },
```

```
// Добавляем этот метод
```

```
getMe: async (token?: string) => {
  const authToken = token || localStorage.getItem('authToken');
  if (!authToken) throw new Error('Token not found');
```

```
  const response = await userApi.get('/me', {
    headers: {
      Authorization: `Bearer ${authToken}`,
    },
  });
```

```
  return response.data; // возвращаем данные пользователя
};
```

```

// === AUTH SERVICE ===
export const authService = {
  login: async (credentials: UserLogin): Promise<LoginResult> => {
    const response = await authApi.post('/login', credentials);
    // маппим backend-ответ точно под LoginResult
    return {
      email: response.data.email,
      requiresOtp: response.data.requiresOtp,
      token: response.data.token ?? null,
    };
  },

  // получение токена через OTP
  getToken: async (otpData: OtpVerifyRequest): Promise<AuthResponse> => {
    const response = await authApi.post('/token', otpData);
    const token = response.data;
    const userStr = localStorage.getItem('user');
    const user = userStr ? JSON.parse(userStr) : null;

    if (token && user) {
      localStorage.setItem('authToken', token);
      localStorage.setItem('user', JSON.stringify(user));
      return { token, user };
    }
    throw new Error('Token or user data missing');
  },

  // Проверка токена на валидность
  validateToken: async (token?: string): Promise<boolean> => {
    const authToken = token || localStorage.getItem('authToken');
    if (!authToken) throw new Error('Token not found');
    const response = await authApi.get(`/valid-token?token=${authToken}`);
    return response.data === 'Token is valid';
  },

  // Регистрация
  register: async (userData: UserCreate): Promise<AuthResponse> => {
    const response = await authApi.post('/register', userData);
    return response.data;
  },

  resetPassword: async (payload: { email: string; oldPassword: string; newPassword: string; confirmPassword: string; }) => {
    const response = await authApi.post('/reset-password', payload);
    return response.data;
  },

  // Выход
  logout: async (): Promise<void> => {
    try { await authApi.post('/logout'); } finally {
      localStorage.removeItem('authToken');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
  },
};

// === Интерцепторы ===
const setupAuthInterceptor = (instance: AxiosInstance) => {
  instance.interceptors.request.use((config: any) => {
    const token = localStorage.getItem('authToken');
    if (token) config.headers.Authorization = `Bearer ${token}`;
  });
};

```



```

    return config;
  });
};

const setupErrorInterceptor = (instance: AxiosInstance) => {
  instance.interceptors.response.use(
    (response) => response,
    (error) => {
      if (error.response?.status === 401) {
        localStorage.removeItem('authToken');
        localStorage.removeItem('user');
        window.location.href = '/login';
      }
      return Promise.reject(error);
    }
  );
};

// === Применяем интерцепторы ===
[infoApi, userApi, authApi].forEach(api => {
  setupAuthInterceptor(api);
  setupErrorInterceptor(api);
});

```

LoginPage.tsx – страница входа

```

import React, { useState } from 'react';
import { authService } from '../api/clients';
import type { UserLogin, OtpVerifyRequest, LoginResult } from '../types/auth';

const LoginPage: React.FC = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [otp, setOtp] = useState('');
  const [loginResult, setLoginResult] = useState<LoginResult | null>(null);
  const [error, setError] = useState<string | null>(null);

  // === Универсальный обработчик ошибок от Spring AdviceController ===
  const extractErrorMessage = (err: any): string => {
    if (!err) return 'Неизвестная ошибка';

    const backendError = err.response?.data ?? err.message ?? err.toString();
    if (typeof backendError === 'string') return backendError;
    if (backendError.message) return backendError.message;
    if (backendError.error) return backendError.error;
    if (backendError.cause && backendError.cause.className)
      return `Ошибка: ${backendError.cause.className} → ${backendError.cause.method}`;

    try {
      return JSON.stringify(backendError);
    } catch {
      return 'Не удалось обработать ошибку';
    }
  };

  // === Авторизация по email + password ===
  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault();
    setError(null);

    try {
      const result = await authService.login({ email, password } as UserLogin);
      console.log('Login response:', result);

      setLoginResult({
        email: result.email,
        requiresOtp: result.requiresOtp,

```

```

    token: result.token ?? null,
  });

  // если OTP не требуется → сразу заходим
  if (!result.requiresOtp && result.token) {
    localStorage.setItem('authToken', result.token);
    localStorage.setItem('user', JSON.stringify({ email: result.email }));
    window.location.href = '/usermanager';
  }
} catch (err: any) {
  console.error('Login error:', err);
  setError(extractErrorMessage(err));
}
};

// === Проверка OTP ===
const handleVerifyOtp = async () => {
  if (!loginResult) return;
  setError(null);

  const request: OtpVerifyRequest = { email: loginResult.email, otp };

  try {
    const response = await authService.getToken(request);
    const token = response.token;

    if (!token) throw new Error('Token not received');

    localStorage.setItem('authToken', token);
    localStorage.setItem('user', JSON.stringify({ email: loginResult.email }));
    window.location.href = '/changepassword';
  } catch (err: any) {
    console.error('OTP error:', err);
    setError(extractErrorMessage(err));
  }
};

return (
  <div style={{ maxWidth: 400, margin: '50px auto', textAlign: 'center' }}>
    <h2>Login</h2>

    {/* === Форма email + password === */}
    {!loginResult?.requiresOtp && (
      <>
        <form onSubmit={handleLogin}>
          <input
            type="email"
            placeholder="Email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            style={{ marginBottom: 8, width: '100%', padding: 6 }}
          />
          <br />
          <input
            type="password"
            placeholder="Password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            style={{ marginBottom: 8, width: '100%', padding: 6 }}
          />
          <br />
          <button type="submit" style={{ padding: 8, width: '100%' }}>
            Login
          </button>
        </form>
      </>
    )}
  )

```

```

    { /* === Форма для OTP === */
    {loginResult?.requiresOtp && (
    <div>
      <p>OTP sent to {loginResult.email}</p>
      <input
        type="text"
        placeholder="Enter OTP"
        value={otp}
        onChange={(e) => setOtp(e.target.value)}
        style={{ marginBottom: 8, width: '100%', padding: 6 }}
      />
      <br />
      <button onClick={handleVerifyOtp} style={{ padding: 8, width: '100%' }}>
        Verify OTP
      </button>
    </div>
    )}

    {error && (
    <p
      style={{
        color: 'red',
        backgroundColor: '#ffecec',
        border: '1px solid #f5aca6',
        padding: '6px 10px',
        borderRadius: 4,
        margin: '10px 0',
      }}
    >
      {error}
    </p>
    )}
  </div>
);
};

export default LoginPage;

```

WelcomePage.tsx – информация о студенте

```

import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import type { InfoResponse } from '../types/info';
import { infoService } from '../api/clients';
import './WelcomePage.css';

const WelcomePage: React.FC = () => {
  const [info, setInfo] = useState<InfoResponse | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string>('');
  const navigate = useNavigate();

  useEffect(() => {
    loadInfo();
  }, []);

  const loadInfo = async () => {
    try {
      setLoading(true);
      const data = await infoService.getAbout();
      setInfo(data);
    } catch (err: any) {
      setError(err.response?.data?.message || err.message || 'Ошибка загрузки');
    } finally {
      setLoading(false);
    }
  };

  const handleNext = () => {

```

```

    navigate('/login');
  };

  if (loading) {
    return (
      <div className="welcome-container">
        <div className="loading">Загрузка информации...</div>
      </div>
    );
  }

  if (error) {
    return (
      <div className="welcome-container">
        <div className="error">
          <h3>Ошибка загрузки</h3>
          <p>{error}</p>
          <button onClick={loadInfo} className="retry-button">
            Повторить попытку
          </button>
        </div>
      </div>
    );
  }

  return (
    <div className="welcome-container">
      <div className="welcome-header">
        <h1>Система криптографической защиты</h1>
        <p>Добро пожаловать в систему управления пользователями</p>
      </div>

      {info && (
        <div className="info-panel">
          <InfoRow label="👤 Автор:" value={` ${info.author} (${info.group})` } />
          <InfoRow label="📄 Работа:" value={info.labNumber} />
          <InfoRow label="🔒 Тема:" value={info.topic} />
          <InfoRow label="📋 Алгоритмы:" value={info.algorithms} />
          <InfoRow label="🔐📄 Политика паролей:" value={info.passwordPolicy} />
        </div>
      )}

      <div className="actions">
        <button
          className="next-button"
          onClick={handleNext}
          disabled={!info}
        >
          Перейти к системе →
        </button>
      </div>
    </div>
  );
};

const InfoRow: React.FC<{ label: string; value: string }> = ({ label, value }) => (
  <div className="info-row">
    <div className="info-label">{label}</div>
    <div className="info-value">{value}</div>
  </div>
);

export default WelcomePage;

```

UserManager.tsx – панель администрирования пользователей

```

import React, { useEffect, useState } from 'react';
import { userService } from '../api/clients';

```

```

import { useNavigate } from 'react-router-dom';

const UserManager: React.FC = () => {
  const navigate = useNavigate();
  const [user, setUser] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [users, setUsers] = useState<any[]>([]);
  const [search, setSearch] = useState("");
  const [newEmail, setNewEmail] = useState("");
  const [editingUser, setEditingUser] = useState<any | null>(null);

  useEffect(() => {
    const token = localStorage.getItem('authToken');
    if (!token) {
      navigate('/login');
      return;
    }

    userService
      .getMe(token)
      .then((userData) => {
        if (userData.role !== 'ADMIN') {
          navigate('/login');
          return;
        }
        if (userData.role === 'USER') {
          navigate('/changepassword');
          return;
        }
        setUser(userData);
        loadUsers();
      })
      .catch((err) => {
        console.error('Ошибка получения профиля:', err);
        localStorage.removeItem('authToken');
        localStorage.removeItem('user');
        navigate('/login');
      })
      .finally(() => setLoading(false));
  }, [navigate]);

  const loadUsers = async () => {
    try {
      const data = await userService.getAllUsers(search);
      setUsers(Array.isArray(data) ? data : data.content || data.users || []);
    } catch (e) {
      console.error('Ошибка загрузки пользователей:', e);
    }
  };

  const handleAddUser = async () => {
    if (!newEmail.trim()) return;
    try {
      await userService.registerUser({ email: newEmail });
      setNewEmail("");
      await loadUsers();
    } catch (e) {
      console.error('Ошибка добавления пользователя:', e);
      alert('Ошибка при добавлении пользователя');
    }
  };
}

```

```

* СБРОС/УСТАНОВКА пароля админом + обновление остальных полей
* Если у пользователя включены passwordRestrictions, проверяем правило:
* — все символы пароля должны быть уникальны (отсутствие повторяющихся символов).
*/
const handleSaveEdit = async (id: string) => {
  try {
    if (!editingUser) return;

    const {
      email,
      role,
      accountLocked,
      passwordRestrictions,
      passwordExpirationMonths,
      minPasswordLength,
      failedAttempts,
      firstLogin,
      emailConfirmed,
      newPassword, // новое поле пароля
    } = editingUser;

    // 🔍 Проверки пароля
    if (newPassword && newPassword.length > 0) {
      if (passwordRestrictions) {
        const chars = newPassword.split("");
        const unique = new Set(chars);
        if (unique.size !== chars.length) {
          alert('❌ Пароль не должен содержать повторяющихся символов.');
```

```

const handleDeleteUser = async (email: string) => {
  if (!window.confirm('Удалить пользователя ' + email + '?')) return;
  try {
    await userService.deleteUser(email);
    await loadUsers();
  } catch (e) {
    console.error('Ошибка удаления пользователя:', e);
    alert('Ошибка при удалении пользователя');
  }
};

if (loading) return <p>Загрузка...</p>;
if (!user) return null;

return (
  <div>
    <h2>Панель управления пользователями</h2>
    <p>Привет, {user.email}</p>
    <p>Роль: {user.role}</p>

    <hr />
    <h3>Список пользователей</h3>

    <div style={{ marginBottom: 10 }}>
      <input
        type="text"
        placeholder="Поиск по email..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      <button onClick={loadUsers}>🔍 Найти</button>
    </div>

    <div style={{ marginBottom: 10 }}>
      <input
        type="email"
        placeholder="Введите email нового пользователя"
        value={newEmail}
        onChange={(e) => setNewEmail(e.target.value)}
      />
      <button onClick={handleAddUser}>➕ Добавить</button>
    </div>

    <table border={1} cellPadding={6} style={{ borderCollapse: 'collapse', width: '100%' }}>
      <thead>
        <tr>
          <th>Email</th>
          <th>Роль</th>
          <th>Заблокирован</th>
          <th>Огр. пароля</th>
          <th>Мин. длина</th>
          <th>Истечение пароля (мес)</th>
          <th>Попытки</th>
          <th>Первый вход</th>
          <th>Email подтверждён</th>
          <th>Хэш пароля</th>
          <th>Соль</th>
          <th>Дата смены пароля</th>
          <th>Действия</th>
        </tr>
      </thead>
      <tbody>
        {users.length === 0 && (

```

```

<tr>
  <td colspan={13} style={{ textAlign: 'center' }}>Нет пользователей</td>
</tr>
)}
{users.map((u) => (
  <tr key={u.id}>
    <td>
      {editingUser?.id === u.id ? (
        <input
          type="email"
          value={editingUser.email}
          onChange={(e) =>
            setEditingUser({ ...editingUser, email: e.target.value })
          }
        />
      ) : (
        u.email
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <select
          value={editingUser.role}
          onChange={(e) =>
            setEditingUser({ ...editingUser, role: e.target.value })
          }
        >
          <option value="ADMIN">ADMIN</option>
          <option value="USER">USER</option>
        </select>
      ) : (
        u.role
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="checkbox"
          checked={!!editingUser.accountLocked}
          onChange={(e) =>
            setEditingUser({ ...editingUser, accountLocked: e.target.checked })
          }
        />
      ) : (
        u.accountLocked ? '🔒' : '🔓'
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="checkbox"
          checked={!!editingUser.passwordRestrictions}
          onChange={(e) =>
            setEditingUser({ ...editingUser, passwordRestrictions: e.target.checked })
          }
        />
      ) : (
        u.passwordRestrictions ? 'Да' : 'Нет'
      )}
    </td>

```



```

<td>
  {editingUser?.id === u.id ? (
    <input
      type="number"
      value={editingUser.minPasswordLength ?? 0}
      onChange={(e) =>
        setEditingUser({ ...editingUser, minPasswordLength: +e.target.value })
      }
      style={{ width: 60 }}
    />
  ) : (
    u.minPasswordLength
  )}
</td>

```

```

<td>
  {editingUser?.id === u.id ? (
    <input
      type="number"
      value={editingUser.passwordExpirationMonths ?? 0}
      onChange={(e) =>
        setEditingUser({
          ...editingUser,
          passwordExpirationMonths: +e.target.value,
        })
      }
      style={{ width: 60 }}
    />
  ) : (
    u.passwordExpirationMonths
  )}
</td>

```

```

<td>
  {editingUser?.id === u.id ? (
    <input
      type="number"
      value={editingUser.failedAttempts ?? 0}
      onChange={(e) =>
        setEditingUser({ ...editingUser, failedAttempts: +e.target.value })
      }
      style={{ width: 60 }}
    />
  ) : (
    u.failedAttempts
  )}
</td>

```

```

<td>
  {editingUser?.id === u.id ? (
    <input
      type="checkbox"
      checked={!editingUser.firstLogin}
      onChange={(e) =>
        setEditingUser({ ...editingUser, firstLogin: e.target.checked })
      }
    />
  ) : (
    u.firstLogin ? 'Да' : 'Нет'
  )}
</td>

```

```

<td>
  {editingUser?.id === u.id ? (
    <input
      type="checkbox"
      checked={!editingUser.emailConfirmed}
      onChange={(e) =>
        setEditingUser({ ...editingUser, emailConfirmed: e.target.checked })
      }
    />
  ) : (
    u.emailConfirmed ? '☑' : '☐'
  )}
</td>

<td style={{ fontSize: '0.8em', color: '#555' }}>{u.password_hash}</td>
<td style={{ fontSize: '0.8em', color: '#555' }}>{u.salt}</td>
<td>{u.passwordLastChanged ?? '—'}</td>

<td>
  {editingUser?.id === u.id ? (
    <>
      {/* Поле для установки пароля админом */}
      <div style={{ marginBottom: 6 }}>
        <input
          type="password"
          placeholder="Новый пароль (админ)"
          value={editingUser.newPassword ?? ""}
          onChange={(e) =>
            setEditingUser({ ...editingUser, newPassword: e.target.value })
          }
          style={{ width: 180 }}
        />
      </div>

      <div>
        <button onClick={() => handleSaveEdit(u.id)}>💾</button>
        <button onClick={() => setEditingUser(null)}>🗑️</button>
      </div>
    </>
  ) : (
    <>
      <button onClick={() => setEditingUser(u)}>↔️</button>
      <button onClick={() => handleDeleteUser(u.email)}>🗑️</button>
    </>
  )}
</td>
</tr>
  )}
</tbody>
</table>
</div>
);
};

export default UserManager;

```

ResetPasswordPage.tsx – смена пароля пользователя

```

import React, { useState } from 'react';
import { authService } from '../api/clients';
import { useNavigate } from 'react-router-dom';

```

```

const ResetPasswordPage: React.FC = () => {
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [oldPassword, setOldPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState<string | null>(null);
  const [success, setSuccess] = useState(false);

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    // === Локальные проверки ===
    if (!email || !oldPassword || !newPassword || !confirmPassword) {
      setMessage('❑ Все поля обязательны для заполнения.');
```

return;

```
    }

    if (newPassword !== confirmPassword) {
      setMessage('❑ Пароли не совпадают.');
```

return;

```
    }

    if (oldPassword === newPassword) {
      setMessage('❑ Новый пароль не должен совпадать со старым.');
```

return;

```
    }

    setLoading(true);
    setMessage(null);

    try {
      // === Отправка запроса на бэк ===
      const response = await authService.resetPassword({
        email,
        oldPassword,
        newPassword,
        confirmPassword,
      });

      // Успех
      setSuccess(true);
      setMessage('❑ Пароль успешно изменён! Перенаправление...');
      setTimeout(() => navigate('/login'), 2000);
    } catch (err: any) {
      console.error('Ошибка сброса пароля:', err);

      // Берём сообщение прямо с бэка, если есть
      let backendMsg = 'Не удалось изменить пароль. Проверьте введённые данные.';
      if (err?.response?.data) {
        const data = err.response.data;
        if (typeof data === 'string') backendMsg = data;
        else if (data?.message) backendMsg = data.message;
        else if (data?.error) backendMsg = data.error;
      }

      setMessage(`❑ ${backendMsg}`);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div
      style={{
        maxWidth: 420,
        margin: '80px auto',
        background: '#fff',

```

```

padding: '30px 25px',
borderRadius: 12,
boxShadow: '0 2px 10px rgba(0,0,0,0.1)',
}}
>
<h2 style={{ textAlign: 'center', marginBottom: 20 }}>🔒 Смена пароля</h2>

{message && (
  <p
    style={{
      textAlign: 'center',
      color: success ? 'green' : 'red',
      fontWeight: 500,
    }}
  >
    {message}
  </p>
)}

<form onSubmit={handleSubmit}>
  <div style={{ marginBottom: 12 }}>
    <label>Email:</label>
    <input
      type="email"
      value={email}
      placeholder="Введите ваш email"
      onChange={(e) => setEmail(e.target.value)}
      style={{
        width: '100%',
        padding: 10,
        borderRadius: 8,
        border: '1px solid #ccc',
        marginTop: 5,
      }}
    />
  </div>

  <div style={{ marginBottom: 12 }}>
    <label>Старый пароль:</label>
    <input
      type="password"
      value={oldPassword}
      placeholder="Введите текущий пароль"
      onChange={(e) => setOldPassword(e.target.value)}
      style={{
        width: '100%',
        padding: 10,
        borderRadius: 8,
        border: '1px solid #ccc',
        marginTop: 5,
      }}
    />
  </div>

  <div style={{ marginBottom: 12 }}>
    <label>Новый пароль:</label>
    <input
      type="password"
      value={newPassword}
      placeholder="Введите новый пароль"
      onChange={(e) => setNewPassword(e.target.value)}
      style={{
        width: '100%',
        padding: 10,
        borderRadius: 8,
        border: '1px solid #ccc',
        marginTop: 5,
      }}
    />
  </div>

```

```

<div style={{ marginBottom: 20 }}>
  <label>Подтверждение нового пароля:</label>
  <input
    type="password"
    value={confirmPassword}
    placeholder="Повторите новый пароль"
    onChange={(e) => setConfirmPassword(e.target.value)}
    style={{
      width: '100%',
      padding: 10,
      borderRadius: 8,
      border: '1px solid #ccc',
      marginTop: 5,
    }}
  />
</div>

<button
  type="submit"
  disabled={loading}
  style={{
    width: '100%',
    padding: 10,
    background: loading ? '#aaa' : '#007bff',
    color: '#fff',
    border: 'none',
    borderRadius: 8,
    cursor: loading ? 'not-allowed' : 'pointer',
    fontSize: 16,
    fontWeight: 600,
  }}
>
  {loading ? 'Обработка...' : 'Изменить пароль'}
</button>

<button
  type="button"
  onClick={() => navigate('/login')}
  style={{
    marginTop: 10,
    width: '100%',
    padding: 10,
    background: '#f0f0f0',
    color: '#333',
    border: 'none',
    borderRadius: 8,
    cursor: 'pointer',
    fontSize: 15,
  }}
>
  Назад ко входу
</button>
</form>
</div>
);
};

export default ResetPasswordPage;

```

Заключение

В ходе выполнения лабораторной работы удалось реализовать программную систему, которая обеспечивает работу криптографических методов и разграничение прав доступа для пользователей. Было реализовано

веб-приложение и веб-сервер, которые изолированы друг от друга. Вся бизнес-логика системы, включая аутентификацию, управления пользователями, применение политик паролей, генерацию кода подтверждения и разграничение прав реализованы исключительно на стороне Backend-сервера. Frontend выполняет роль отображения данных и обработку пользовательского ввода

Такое принципиальное архитектурное решение обеспечивает максимальную защищённость. Даже при компроментации клиентского кода, злоумышленник не получит доступ к механизмам управления системой. Все критически важные операции защищены на сервере, где применяются проверки подлинности, авторизации и валидации. Так, нам удалось скрыть логику безопасности от внешних атакующих и случайных действий пользователя.

База данных тоже не хранится на локальной машине, разворачивается в случае работы с системой, а затем переносится в локальный файл и шифруется. Возможно данное решение не очень легковесное в случае масштабирования, но очень безопасно и необходимо в рамках выполнения технического задания.

Изучив основные проблемы и важность безопасности при разработке программного обеспечения стало очевидно, что сейчас безопасности – это основной фундамент проектирования и создания программ. Учётные данные пользователей — один из самых ценных активов системы, и их компрометация может привести к полной потере доверия, юридическим последствиям и финансовым убыткам. Поэтому применение криптографически стойких методов (хэширование с солью, использование проверенных алгоритмов), строгое разграничение прав по принципу наименьших привилегий, защита каналов связи (HTTPS), а также

централизация логики на сервере – это очень важные и нужные принципы и меры информационной безопасности на сегодняшний день.

В ходе разработки программного обеспечения были получены новые навыки и применены интересные технологии, которые в дальнейшем стоит использовать при создании новых технических решений. Таким образом, система, построенная на представленных выше принципах сможет противостоять атакам со стороны злоумышленников и не допустить ошибок со стороны обычных пользователей. Всё это надёжно сохранит конфиденциальность, доступность и целостность данных пользователей и самой программы.

Приложение. Листинг кода

В данном разделе представлен весь код, основные и ключевые моменты были описаны выше.

```
Backend-core

Build.gradle
plugins {
    id 'java'
    id 'org.springframework.boot'
version '3.5.6'
    id 'io.spring.dependency-
management' version '1.1.7'
}

group = 'com.cryptoauth'
version = '0.0.1-SNAPSHOT'
description = 'Demo project for Spring Boot'

java {
    toolchain {
        languageVersion =
JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation
'org.springframework.boot:spring-boot-starter'
    implementation
'org.springframework.boot:spring-boot-starter-data-
jpa'
    implementation
'org.springframework.boot:spring-boot-starter-
security'
    implementation
'org.springframework.boot:spring-boot-starter-
validation'
    implementation
'org.springframework.boot:spring-boot-starter-web'

    // --- DATABASE ---
    implementation
'org.postgresql:postgresql'
    implementation
'org.flywaydb:flyway-core'
    implementation
'org.flywaydb:flyway-database-postgresql'

    // --- JWT ---
    implementation 'io.jsonwebtoken:jjwt-
api:0.12.5'
    runtimeOnly 'io.jsonwebtoken:jjwt-
impl:0.12.5'

    runtimeOnly 'io.jsonwebtoken:jjwt-
jackson:0.12.5'

    // --- OpenApi ---
    implementation
'org.springdoc:springdoc-openapi-starter-webmvc-
ui:2.7.0'

    // --- Lombok ---
    compileOnly
'org.projectlombok:lombok'
    annotationProcessor
'org.projectlombok:lombok'

    // --- Mapper ---
    implementation
'org.mapstruct:mapstruct:1.6.3'
    annotationProcessor
'org.mapstruct:mapstruct-processor:1.6.3'

    // --- Jackson ---
    implementation
'com.fasterxml.jackson.core:jackson-databind'

    // --- Mail Service ---
    implementation
'org.springframework.boot:spring-boot-starter-mail'

    testImplementation
'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly
'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}

Application.yml
jwt:
  secret:
VGhpcy1pc0Etc2VjcmV0S2V5LXlvdS1zaG91bGQtdXNlIQ==

server:
  port: 8988

spring:
  datasource:
    url:
jdbc:postgresql://localhost:5432/infosec
    username: postgres
    password: postgres
    driver-class-name: org.postgresql.Driver
```



```

jpa:
  hibernate:
    ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    database-platform:
org.hibernate.dialect.PostgreSQLDialect

flyway:
  enabled: true
  locations: classpath:db/migration
  baseline-on-migrate: true
  validate-on-migrate: true

mail:
  host: smtp.gmail.com
  port: 587
  # username: furniturecoup73@gmail.com
  # password: hwba frfa mmbo bpji
  username: clinicapayment@gmail.com
  password: tpwj qnut rehj kfif
  from: clinicapayment@gmail.com

properties:
  mail:
    smtp:
      auth: true
      starttls:
        enable: true

# H2 console (опционально, если нужно
для разработки)
h2:
  console:
    enabled: true
    path: /h2-console

# Swagger UI (если используется
springdoc-openapi)
springdoc:
  api-docs:
    path: /v3/api-docs
  swagger-ui:
    path: /swagger-ui.html

app:
  mode: file
  encryption:
    key: MySuperSecretKey123
  filedb:
    path: ./data/users.enc

migration-V1__init.sql
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT
gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT
NULL,
  password_hash VARCHAR(512),
  role VARCHAR(50) NOT NULL,
  account_locked BOOLEAN NOT NULL,
  password_restrictions BOOLEAN NOT
NULL,
  password_expiration_months INT NOT
NULL,
  password_last_changed TIMESTAMP,
  min_password_length INT,
  failed_attempts INT NOT NULL,
  first_login BOOLEAN,
  email_confirmed BOOLEAN NOT
NULL
);

InfosecurityApplication.java
package com.cryptoauth.infosecurity;

import
org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class InfosecurityApplication {

    public static void main(String[]
args) {

        SpringApplication.run(InfosecurityApplicati
on.class, args);
    }
}

user-utils
PasswordHashUtil.java
package
com.cryptoauth.infosecurity.user.utils;

import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.Base64;

public class PasswordHashUtil {

    private PasswordHashUtil() {}

    // Генерация соли (16 байт)
    public static String generateSalt() {
        byte[] salt = new byte[16];
        new SecureRandom().nextBytes(salt);
        return
Base64.getEncoder().encodeToString(salt);
    }

    // Хэширование с солью
    public static String sha1Hex(String input,
String salt) {
        try {
            // "SHA" - SHA-1

```

```

        MessageDigest messageDigest =
MessageDigest.getInstance("SHA");

messageDigest.update(salt.getBytes(java.nio.charset.
StandardCharsets.UTF_8));
        byte[] digest =
messageDigest.digest(input.getBytes(java.nio.charset
.StandardCharsets.UTF_8));

        StringBuilder stringBuilder = new
StringBuilder();
        for (byte b : digest) {

stringBuilder.append(String.format("%02x", b &
0xff));
        }

        return stringBuilder.toString();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

public static boolean matches(String raw,
String salt, String hashed) {
    if (raw == null && hashed == null)
return true;
    if (raw == null || hashed == null) return
false;
    return sha1Hex(raw,
salt).equals(hashed);
}
}

DesFileCipher.java
package
com.cryptoauth.infosecurity.user.utils;

import
java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.MessageDigest;
import
java.security.NoSuchAlgorithmException;

import javax.crypto.*;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

import java.io.*;

public class DesFileCipher {

    private static final String
TRANSFORMATION =
"DES/CBC/PKCS5Padding";

    // фиксированный IV (8 байт),
выполняем по тз.
    private static final byte[] FIXED_IV =
new byte[] { 0, 0, 0, 0, 0, 0, 0, 1 };

```

```

private DesFileCipher() {}

// Генерация ключа DES из парольной
фразы (берём первые 8 байт SHA-1 хэша)
public static SecretKeySpec
keyFromPassphrase(String passphrase) {
    try {
        MessageDigest messageDigest =
MessageDigest.getInstance("SHA");
        byte[] digest =
messageDigest.digest(passphrase.getBytes(java.nio.c
harset.StandardCharsets.UTF_8));

        // DES требует 8-байтный ключ ->
берём первые 8 байт хэша
        byte[] keyBytes = new byte[8];
        System.arraycopy(digest, 0,
keyBytes, 0, 8);

        return new SecretKeySpec(keyBytes,
"DES");
    } catch (Exception e) {
        throw new
RuntimeException("Cannot generate key from
passphrase", e);
    }
}

public static void processFile(int mode,
File inFile, File outFile, String passphrase) {
    try {
        Cipher cipher =
Cipher.getInstance(TRANSFORMATION);

        cipher.init(mode,
keyFromPassphrase(passphrase), new
IvParameterSpec(FIXED_IV));

        if (!inFile.exists() && mode ==
Cipher.DECRYPT_MODE)
            throw new
FileNotFoundException("Encrypted file not found: "
+ inFile.getAbsolutePath());

        try (FileInputStream fis = new
FileInputStream(inFile);
             FileOutputStream fos = new
FileOutputStream(outFile);
             CipherOutputStream cos = new
CipherOutputStream(fos, cipher)) {

            byte[] buffer = new byte[8192];
            int r;

            while ((r = fis.read(buffer)) != -1)
                cos.write(buffer, 0, r);
        } catch (IOException |
NoSuchAlgorithmException |
NoSuchPaddingException |
InvalidKeyException |
InvalidAlgorithmParameterException e) {

```

```

        throw new RuntimeException((mode
== Cipher.ENCRYPT_MODE ? "Encryption" :
"Decryption") + " failed", e);
    }
}

    public static void encryptFile(File inFile,
File outFile, String passphrase) {

processFile(Cipher.ENCRYPT_MODE,      inFile,
outFile, passphrase);
    }

    public static void decryptFile(File inFile,
File outFile, String passphrase) {

processFile(Cipher.DECRYPT_MODE,      inFile,
outFile, passphrase);
    }

    public          static          void
overwriteAndDelete(File file) {
    if (file == null || !file.exists()) return;

    try (RandomAccessFile raf = new
RandomAccessFile(file, "rw")) {
        long size = raf.length();
        raf.seek(0);
        byte[] zeros = new byte[8192];
        long written = 0;

        while (written < size) {
            int          toWrite          =
(int)Math.min(zeros.length, size - written);
            raf.write(zeros, 0, toWrite);
            written += toWrite;
        }
    } catch (IOException ignored) {}
    file.delete();
}

    Config
DatabaseCryptoSyncManager.java
package
com.cryptoauth.infosecurity.user.config;

import
com.cryptoauth.infosecurity.user.entity.Role;
import
com.cryptoauth.infosecurity.user.entity.User;
import
com.cryptoauth.infosecurity.user.repository.UserRep
ository;
import
com.cryptoauth.infosecurity.user.utils.DesFileCipher;
import
com.cryptoauth.infosecurity.user.utils.PasswordHash
Util;
import
com.fasterxml.jackson.databind.ObjectMapper;

```

```

import
com.fasterxml.jackson.databind.SerializationFeature;
import
com.fasterxml.jackson.datatype.jsr310.JavaTimeMod
ule;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import
org.springframework.boot.CommandLineRunner;
import
org.springframework.context.event.ContextClosedEv
ent;

import
org.springframework.stereotype.Component;
import
org.springframework.jdbc.core.JdbcTemplate;
import
org.springframework.context.event.EventListener;
import
org.springframework.transaction.annotation.Transacti
onal;

import java.io.File;
import java.nio.file.Files;
import java.time.LocalDateTime;
import java.util.*;

@Component
@RequiredArgsConstructor
@Slf4j
public class DatabaseCryptoSyncManager
implements CommandLineRunner {

    private          final          UserRepository
userRepository;

    private final JdbcTemplate jdbcTemplate;

    // Сохраним passphrase при старте,
чтобы использовать при закрытии
    private volatile String passphrase;

    private final ObjectMapper objectMapper
= new ObjectMapper().registerModule(new
JavaTimeModule())

.disable(SerializationFeature.WRITE_DATES_AS_T
IMESTAMPS);

    private static final String JSON_FILE =
"userdata.json";
    private static final String ENC_FILE =
"userdata.enc";

    private final File jsonFile = new
File(JSON_FILE);
    private final File encFile = new
File(ENC_FILE);

    @Override
    public void run(String... args) throws
Exception {
        File jsonFile = new File(JSON_FILE);

```

```

        File encFile = new File(ENC_FILE);

        // === Запрос парольной фразы ===
        Scanner scanner = new
Scanner(System.in);
        System.out.print("Введите парольную
фразу для доступа к данным (Enter = Init!): ");
        this.passphrase =
scanner.nextLine().trim();

        // Расшифровка или создание нового
файла ===
        if (encFile.exists()) {
            try {

DesFileCipher.decryptFile(encFile, jsonFile,
passphrase);
                log.info("Файл userdata.enc
расшифрован для работы");
            } catch (Exception e) {
                log.error("Ошибка расшифровки
userdata.enc: неверный пароль или повреждён
файл", e);
                return;
            }
        } else {
            log.warn("Файл userdata.enc не
найден. Создаётся новый JSON с
администратором по умолчанию");
            createDefaultAdmin(jsonFile);
            DesFileCipher.encryptFile(jsonFile,
encFile, passphrase);

Files.deleteIfExists(jsonFile.toPath());
            log.info("Создан новый
зашифрованный файл userdata.enc");
        }

        // === Синхронизация JSON →
PostgreSQL === //
        if (jsonFile.exists() &&
Files.size(jsonFile.toPath()) > 0) {
            try {
                User[] usersFromFile =
objectMapper.readValue(jsonFile, User[].class);

                for (User u : usersFromFile) {
                    u.setId(null); // JPA
сгенерирует новый UUID
                }

                // Полностью перезаписываем
таблицу
                userRepository.deleteAll();

userRepository.saveAll(Arrays.asList(usersFromFile)
);
                log.info("Все пользователи из
JSON перезаписаны в базе данных");
            } catch (Exception e) {

```

```

                log.error("Ошибка загрузки
пользователей из JSON", e);
            }
        } else {
            log.warn("JSON-файл отсутствует
или пуст. Тогда создаётся администратор по
умолчанию");
            if (userRepository.count() == 0) {
                createDefaultAdmin(jsonFile);

DesFileCipher.encryptFile(jsonFile, encFile,
passphrase);

Files.deleteIfExists(jsonFile.toPath());
                log.info("Создан администратор
по умолчанию и зашифрованный файл.");
            }
        }

        // === Экспорт PostgreSQL -> JSON -
> ENC при завершении === //

Runtime.getRuntime().addShutdownHook(new
Thread(() -> {
            try {
                List<User> allUsers =
userRepository.findAll();

objectMapper.writerWithDefaultPrettyPrinter().write
Value(jsonFile, allUsers);

DesFileCipher.encryptFile(jsonFile, encFile,
passphrase);

Files.deleteIfExists(jsonFile.toPath());
                log.info("Данные PostgreSQL
экспортированы и зашифрованы в userdata.enc");
            } catch (Exception e) {
                log.error("Ошибка при экспорте
PostgreSQL -> ENC", e);
            }
        }));

@EntityListener(ContextClosedEvent.class)
@Transactional
public void
onContextClosed(ContextClosedEvent ev) {
    // безопасность, если passphrase не
установлен — ничего не делаем
    if (this.passphrase == null) {
        log.warn("Passphrase отсутствует
— пропускаем экспорт/очистку.");
        return;
    }

    try {
        // Экспортируем snapshot из БД в
JSON
        List<User> allUsers =
userRepository.findAll();

```

```

objectMapper.writerWithDefaultPrettyPrinter().write
Value(jsonFile, allUsers);
        log.info("Экспортировано {}
пользователей в {}", allUsers.size(),
jsonFile.getAbsolutePath());

        // Шифруем JSON -> userdata.enc
        DesFileCipher.encryptFile(jsonFile,
encFile, this.passphrase);

Files.deleteIfExists(jsonFile.toPath());
        log.info("JSON зашифрован в {}",
encFile.getAbsolutePath());

        // Очищаем таблицу через JPA
        userRepository.deleteAll();
        log.warn("Все записи
пользователей удалены");

        clearDatabase();
        log.warn("Таблица users
сброшена");

    } catch (Exception ex) {
        log.error("Ошибка при
экспорте/шифровании/очистке при закрытии
контекста", ex);
    }
}

// === Очистка базы данных === //
private void clearDatabase() {
    try {
        // Очистка таблицы пользователей
        userRepository.deleteAll();
        log.warn("Все пользователи в базе
данных удалены (выполнена очистка
PostgreSQL)");

        jdbcTemplate.execute("DROP
TABLE IF EXISTS users CASCADE");
        //
        jdbcTemplate.execute("TRUNCATE TABLE users
RESTART IDENTITY CASCADE");
        log.warn("Таблица 'users' удалена и
очищена из базы данных.");
    } catch (Exception e) {
        log.error("Ошибка при очистке
базы данных", e);
    }
}

// === Вспомогательный метод для
создания админа === //
private void createDefaultAdmin(File
jsonFile) {
    try {
        String salt =
PasswordHashUtil.generateSalt();
        String hash =
PasswordHashUtil.sha1Hex("admin", salt);

```

```

        User admin = User.builder()
            .email("admin@example.com")
            .password_hash(hash)
            .salt(salt)
            .role(Role.ADMIN)
            .accountLocked(false)
            .failedAttempts(0)
            .passwordRestrictions(false)
            .passwordExpirationMonths(0)
            .minPasswordLength(0)
            .emailConfirmed(false)
            .firstLogin(false)

            .passwordLastChanged(LocalDateTime.now())
            .build();

        objectMapper.writerWithDefaultPrettyPrinter().write
        Value(jsonFile, List.of(admin));
        log.info("Создан JSON с
администратором по умолчанию");
    } catch (Exception e) {
        log.error("Ошибка при создании
администратора по умолчанию", e);
    }
}

EncryptionService.java
package
com.cryptoauth.infosecurity.user.config;

import
org.springframework.beans.factory.annotation.Value;
import
org.springframework.stereotype.Service;

import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.nio.file.*;
import java.security.Key;
import java.util.Base64;

@Service
public class EncryptionService {

    private final String secret;

    public
    EncryptionService(@Value("${app.encryption.key}"
    ) String secret) {
        this.secret = secret;
    }

    private Cipher initCipher(int mode)
throws Exception {
        Key aesKey = new
        SecretKeySpec(secret.getBytes(), "AES");
        Cipher cipher =
        Cipher.getInstance("AES");
        cipher.init(mode, aesKey);
        return cipher;
    }
}

```

```

    }

    public void encryptToFile(String data,
Path path) throws Exception {
        Cipher cipher =
initCipher(Cipher.ENCRYPT_MODE);
        byte[] encrypted =
cipher.doFinal(data.getBytes());

Files.createDirectories(path.getParent());
        Files.write(path,
Base64.getEncoder().encode(encrypted));
    }

    public String decryptFromFile(Path path)
throws Exception {
        if (!Files.exists(path)) return "";
        byte[] content =
Files.readAllBytes(path);
        byte[] decoded =
Base64.getDecoder().decode(content);
        Cipher cipher =
initCipher(Cipher.DECRYPT_MODE);
        byte[] decrypted =
cipher.doFinal(decoded);
        return new String(decrypted);
    }
}

```

```

FlywayConfig.java
package
com.cryptoauth.infosecurity.user.config;

import org.flywaydb.core.Flyway;
import
org.springframework.beans.factory.annotation.Value;
import
org.springframework.context.annotation.Configuratio
n;
import jakarta.annotation.PostConstruct;

@Configuration
public class FlywayConfig {

    @Value("${spring.datasource.url}")
    private String dbUrl;

    @Value("${spring.datasource.username}")
    private String dbUser;

    @Value("${spring.datasource.password}")
    private String dbPassword;

    @PostConstruct
    public void migrate() {
        System.out.println("Running Flyway
migrations...");
        Flyway.configure()
            .dataSource(dbUrl, dbUser,
dbPassword)
            .baselineOnMigrate(true)
            .load()
    }
}

```

```

        .migrate();
    }
}

controller
package
com.cryptoauth.infosecurity.user.controller;

import java.util.UUID;

import
org.springframework.data.domain.Page;
import
org.springframework.data.domain.Pageable;
import
org.springframework.http.ResponseEntity;
import
org.springframework.web.bind.annotation.DeleteMap
ping;
import
org.springframework.web.bind.annotation.GetMappi
ng;
import
org.springframework.web.bind.annotation.PathVaria
ble;
import
org.springframework.web.bind.annotation.PostMappi
ng;
import
org.springframework.web.bind.annotation.PutMappin
g;
import
org.springframework.web.bind.annotation.RequestBo
dy;
import
org.springframework.web.bind.annotation.RequestM
apping;
import
org.springframework.web.bind.annotation.RequestPa
ram;
import
org.springframework.web.bind.annotation.RestContr
oller;
import
org.springframework.security.core.Authentication;

import
com.cryptoauth.infosecurity.user.dto.UserCreate;
import
com.cryptoauth.infosecurity.user.dto.UserGet;
import
com.cryptoauth.infosecurity.user.dto.UserResponsePr
ofile;
import
com.cryptoauth.infosecurity.user.dto.UserUpdate;
import
com.cryptoauth.infosecurity.user.service.UserService
;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

```

```

@Slf4j
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1/user")
public class UserController {

    private final UserService userService;

    @GetMapping("/users")
    public Page<UserGet>
    getAllUsers(@RequestParam(value = "search",
        required = false) String search, Pageable pageable) {
        return userService.getAllUsers(search,
            pageable);
    }

    @PostMapping("/register")
    public UserGet
    registerUser(@RequestBody UserCreate
        userCreate){
        return
        userService.createUser(userCreate);
    }

    @GetMapping("/users/{email}")
    public UserGet findUser(@PathVariable
        String email) {
        return
        userService.getUserByEmail(email);
    }

    @GetMapping("/users/user/{email}/id")
    public UserGet getIdUser(@PathVariable
        UUID id) {
        return userService.getUser(id);
    }

    @PutMapping("/users/{id}")
    public ResponseEntity<UserGet>
    updateUser(@PathVariable("id") UUID id,
        @RequestBody UserUpdate userUpdate) {
        return
        ResponseEntity.ok(userService.updateUser(id,
            userUpdate));
    }

    @DeleteMapping("/users/{email}")
    public ResponseEntity<Void>
    deleteUser(@PathVariable String email) {
        userService.deleteUser(email);
        return
        ResponseEntity.noContent().build();
    }

    @GetMapping("/me")
    public UserResponseProfile
    getMe(Authentication authentication) {
        // Вытаскивается из токена
        String username =
        authentication.getName();
        return userService.getMe(username);
    }
}

```

```

Dto
PageResponse.java
package
com.cryptoauth.infosecurity.user.dto;

import java.util.List;

public record PageResponse<T>(List<T>
    items, int page, int size, long totalElements,
    int totalPages, boolean last) {}

UserChangePassword.java
package
com.cryptoauth.infosecurity.user.dto;

public record UserChangePassword(String
    oldPassword, String newPassword, String
    confirmPassword) {}

UserCreate.java
package
com.cryptoauth.infosecurity.user.dto;

public record UserCreate(String email) {}

UserGet.java
package
com.cryptoauth.infosecurity.user.dto;

import java.time.LocalDateTime;
import java.util.UUID;

import
com.cryptoauth.infosecurity.user.entity.Role;

public record UserGet(UUID id,
    String email,
    Role role,
    String password_hash,
    String salt,
    Boolean accountLocked,
    Boolean passwordRestrictions,
    Integer passwordExpirationMonths,
    LocalDateTime passwordLastChanged,
    Integer minPasswordLength,
    Integer failedAttempts,
    Boolean firstLogin,
    Boolean emailConfirmed
    ) {}

UserResponseProfile.java
package
com.cryptoauth.infosecurity.user.dto;

import java.util.UUID;

```

```
import
com.cryptoauth.infosecurity.user.entity.Role;

public record UserResponseProfile(UUID
id, String email, Role role) {}
```

```
UserUpdate.java
package
com.cryptoauth.infosecurity.user.dto;

import java.time.LocalDateTime;

import
com.cryptoauth.infosecurity.user.entity.Role;

public record UserUpdate(
    String email,
    Role role,
    String password_hash,
    String salt,
    Boolean accountLocked,
    Boolean passwordRestrictions,
    Integer passwordExpirationMonths,
    LocalDateTime passwordLastChanged,
    Integer minPasswordLength,
    Integer failedAttempts,
    Boolean firstLogin,
    Boolean emailConfirmed
) {}
```

```
UserUpdatePolicy.java
package
com.cryptoauth.infosecurity.user.dto;

public record UserUpdatePolicy(boolean
accountLocked, boolean passwordRestricted,
    Integer minPasswordLength, Integer
passwordExpiryMonths) {}
```

```
entity

Role.java
package
com.cryptoauth.infosecurity.user.entity;

public enum Role {
    ADMIN,
    USER,
}
```

```
User.java
package
com.cryptoauth.infosecurity.user.entity;

import java.time.LocalDateTime;
import java.util.UUID;

import jakarta.persistence.*;
import lombok.*;
```

```
@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy =
GenerationType.UUID)
    @Column(updatable = false, nullable =
false)
    private UUID id;

    @Column(unique = true, nullable = false)
    private String email;

    // По ТЗ: пароль в базе хранится как
хэш (алгоритм — SHA)
    @Column(name = "password_hash",
length = 512)
    private String password_hash;

    private String salt;

    @Column(nullable = false)
    @Enumerated(EnumType.STRING)
    private Role role;

    // блокировка учётки
    @Column(nullable = false)
    private boolean accountLocked;

    // включены ли ограничения
    @Column(nullable = false)
    private boolean passwordRestrictions;

    // срок действия пароля в месяцах (0 =
бессрочно)
    @Column(nullable = false)
    private int passwordExpirationMonths;

    // дата смены пароля
    private LocalDateTime
passwordLastChanged;

    // минимальная длина пароля (0 =
разрешён пустой пароль)
    private int minPasswordLength;

    // счётчик неуспешных попыток входа
    @Column(nullable = false)
    private int failedAttempts;

    // первый вход
    @Column(nullable = false)
    private boolean firstLogin;

    // нужно ли подтверждение из почты
    @Column(nullable = false)
    private boolean emailConfirmed;
}
```



```

Exception
UserAlreadyExistsException.java
package
com.cryptoauth.infosecurity.user.exception;

    public class UserAlreadyExistsException
extends RuntimeException {
    public
UserAlreadyExistsException(String email) {
        super("User already exists: " + email);
    }
}

UserNotFoundException.java
package
com.cryptoauth.infosecurity.user.exception;

    public class UserNotFoundException
extends RuntimeException {
    public UserNotFoundException(String
email) {
        super("User not found: " + email);
    }
}

Mapper
UserMapper.java
package
com.cryptoauth.infosecurity.user.mapper;

import java.util.List;

import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;
import org.mapstruct.MappingConstants;

import
com.cryptoauth.infosecurity.user.dto.UserCreate;
import
com.cryptoauth.infosecurity.user.dto.UserGet;
import
com.cryptoauth.infosecurity.user.dto.UserUpdate;
import
com.cryptoauth.infosecurity.user.entity.User;

    @Mapper(componentModel
MappingConstants.ComponentModel.SPRING)
    public interface UserMapper {
        @Mapping(target
"passwordRestrictions", source
"passwordRestrictions") // исправление имени
        @Mapping(target = "firstLogin", source =
"firstLogin")
        @Mapping(target = "accountLocked",
source = "accountLocked")

        @Mapping(target
"minPasswordLength", source
"minPasswordLength")
        @Mapping(target
"passwordExpirationMonths", source
"passwordExpirationMonths")
        @Mapping(target
"passwordLastChanged", source
"passwordLastChanged")
        @Mapping(target = "role", source =
"role")
        @Mapping(target = "id", source = "id")
        @Mapping(target = "email", source =
"email")
        UserGet toDto(User user);

        List<UserGet>
toDtoList(List<User>
users);

        @Mapping(target
"passwordRestrictions", ignore = true)
        @Mapping(target = "firstLogin", ignore =
true)
        @Mapping(target = "accountLocked",
ignore = true)
        @Mapping(target
"minPasswordLength", ignore = true)
        @Mapping(target
"passwordExpirationMonths", ignore = true)
        @Mapping(target
"passwordLastChanged", ignore = true)
        @Mapping(target = "role", ignore = true)
        @Mapping(target = "id", ignore = true)
        @Mapping(target = "password_hash",
ignore = true)
        @Mapping(target = "salt", ignore = true)
        @Mapping(target = "failedAttempts",
ignore = true)
        @Mapping(target = "emailConfirmed",
ignore = true)
        User toEntity(UserCreate userCreate);

        @Mapping(target = "id", ignore = true)
        @Mapping(target
"passwordRestrictions", ignore = true)
        void updateEntity(@MappingTarget User
user, UserUpdate dto);
    }

Repository
UserRepository.java
package
com.cryptoauth.infosecurity.user.repository;

import java.util.Optional;
import java.util.UUID;

import
org.springframework.data.jpa.repository.JpaReposito
ry;
import
org.springframework.stereotype.Repository;

```

```

import
org.springframework.data.domain.Page;
import
org.springframework.data.domain.Pageable;

import
com.cryptoauth.infosecurity.user.entity.User;

@Repository
public interface UserRepository extends
JpaRepository<User, UUID> {
    Optional<User> findById(UUID id);

    Optional<User> findByEmail(String
email);

    Page<User> findAll(Pageable pageable);

    void deleteByEmail(String email);

    boolean existsByEmail(String email);

    Page<User>
findByEmailContainingIgnoreCase(String
emailFragment, Pageable pageable);
}

Service
package
com.cryptoauth.infosecurity.user.service;

import java.time.LocalDateTime;
import java.util.UUID;

import
org.springframework.data.domain.Page;
import
org.springframework.data.domain.Pageable;
import
org.springframework.stereotype.Service;

import
com.cryptoauth.infosecurity.user.dto.UserCreate;
import
com.cryptoauth.infosecurity.user.dto.UserGet;
import
com.cryptoauth.infosecurity.user.dto.UserResponsePr
ofile;
import
com.cryptoauth.infosecurity.user.dto.UserUpdate;
import
com.cryptoauth.infosecurity.user.entity.Role;
import
com.cryptoauth.infosecurity.user.entity.User;
import
com.cryptoauth.infosecurity.user.exception.UserNotF
oundException;
import
com.cryptoauth.infosecurity.user.mapper.UserMappe
r;

```

```

import
com.cryptoauth.infosecurity.user.repository.UserRep
ository;
import
com.cryptoauth.infosecurity.user.utils.PasswordHash
Util;

import
org.springframework.transaction.annotation.Transacti
onal;
import lombok.RequiredArgsConstructor;

@Service
@RequiredArgsConstructor
public class UserService {

    private final UserRepository
userRepository;

    private final UserMapper userMapper;

    // --- создание нового пользователя ---
    @Transactional()
    public UserGet createUser(UserCreate
userCreate) {
        String email =
userCreate.email().toLowerCase().trim();

        if (email.isBlank())
            throw new
IllegalArgumentException("Email is required");

        if ("admin".equalsIgnoreCase(email) ||
"admin@example.com".equalsIgnoreCase(email)) {
            throw new
IllegalArgumentException("Cannot create reserved
ADMIN user");
        }

        if
(userRepository.existsByEmail(email)) {
            throw new
IllegalArgumentException("User already exists");
        }

        User user =
userMapper.toEntity(userCreate);
        user.setRole(Role.USER);
        user.setFailedAttempts(0);
        user.setAccountLocked(false);
        user.setPasswordExpirationMonths(0);
        user.setMinPasswordLength(0);
        user.setPasswordRestrictions(false);

        return
userMapper.toDto(userRepository.save(user));
    }

    @Transactional(readOnly = true)
    public Page<UserGet> getAllUsers(String
search, Pageable pageable) {
        Page<User> page = (search == null ||
search.isBlank())

```

```

        ?
userRepository.findAll(pageable)
        :
userRepository.findByEmailContainingIgnoreCase(s
earch, pageable);

        return page.map(userMapper::toDto);
    }

    @Transactional(readOnly = true)
    public UserGet getUser(UUID id) {
        User user =
userRepository.findById(id)
        .orElseThrow(() -> new
IllegalArgumentException("User not found"));
        return userMapper.toDto(user);
    }

    @Transactional(readOnly = true)
    public UserGet getUserByEmail(String
email) {
        return
userMapper.toDto(userRepository.findByEmail(email)
        .orElseThrow(() -> new
UserNotFoundException(email)));
    }

    @Transactional
    public UserGet updateUser(UUID id,
UserUpdate userUpdate) {
        User user =
userRepository.findById(id)
        .orElseThrow(() -> new
RuntimeException("User not found: " + id));

        // Обновление email (если задан)
        if (userUpdate.email() != null &&
!userUpdate.email().isBlank()) {
            String email =
userUpdate.email().toLowerCase().trim();
            if (!email.equals(user.getEmail())
&& userRepository.existsByEmail(email)) {
                throw new
IllegalArgumentException("Email already in use: " +
email);
            }
            user.setEmail(email);
        }

        // Обновление роли (если задана)
        if (userUpdate.role() != null) {
            user.setRole(userUpdate.role());
        }

        // Обновление пароля (если задан)
        if (userUpdate.password_hash() != null
&& !userUpdate.password_hash().isBlank()) {
            String password =
userUpdate.password_hash();

            // Проверка ограничений на
пароль. Если включены ограничения на пароль

```

```

        if (user.isPasswordRestrictions()) {

            // Если включены ограничения
на пароль
            if (user.isPasswordRestrictions())
            {
                // Проверяем отсутствие
повторяющихся символов
                for (int i = 0; i <
password.length(); i++) {
                    for (int j = i + 1; j <
password.length(); j++) {
                        if (password.charAt(i) ==
password.charAt(j)) {
                            throw new
IllegalArgumentException(
                                "Пароль содержит
повторяющиеся символы. Задайте более сложный
пароль.");
                        }
                    }
                }
            }

            // Хешируем новый пароль
            String salt =
PasswordHashUtil.generateSalt();
            String hashedPassword =
PasswordHashUtil.sha1Hex(password, salt);

            user.setSalt(salt);

            user.setPassword_hash(hashedPassword);

            user.setPasswordLastChanged(LocalDate.now()
        );

        }

        // Прочие обновления
        if (userUpdate.accountLocked() != null)
        {
            user.setAccountLocked(userUpdate.accountLocked()
        );

        }

        if (userUpdate.failedAttempts() != null)
        {
            user.setFailedAttempts(userUpdate.failedAttempts());
        }

        if
(userUpdate.passwordExpirationMonths() != null) {
            user.setPasswordExpirationMonths(userUpdate.pass
wordExpirationMonths());
        }

        if (userUpdate.minPasswordLength() !=
null) {

```



```

        public Page<UserGet>
getAllUsers(@RequestParam(value = "search",
required = false) String search, Pageable pageable) {
    return userService.getAllUsers(search,
pageable);
}

@PostMapping("/register")
public UserGet registerUser(@RequestBody UserCreate
userCreate){
    return
userService.createUser(userCreate);
}

@GetMapping("/users/{email}")
public UserGet findUser(@PathVariable
String email) {
    return
userService.getUserByEmail(email);
}

@GetMapping("/users/user/{email}/id")
public UserGet getIdUser(@PathVariable
UUID id) {
    return userService.getUser(id);
}

@PutMapping("/users/{id}")
public ResponseEntity<UserGet>
updateUser(@PathVariable("id") UUID id,
@RequestBody UserUpdate userUpdate) {
    return
ResponseEntity.ok(userService.updateUser(id,
userUpdate));
}

@DeleteMapping("/users/{email}")
public ResponseEntity<Void>
deleteUser(@PathVariable String email) {
    userService.deleteUser(email);
    return
ResponseEntity.noContent().build();
}

@GetMapping("/me")
public UserResponseProfile
getMe(Authentication authentication) {
    // Вытаскивается из токена
    String username =
authentication.getName();
    return userService.getMe(username);
}
}

```

```

Info
InfoController.java
package com.cryptoauth.infosecurity.info;

import
org.springframework.http.ResponseEntity;

```

```

import
org.springframework.web.bind.annotation.GetMapping;

import
org.springframework.web.bind.annotation.RequestMapping;

import
org.springframework.web.bind.annotation.RestController;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;

@Slf4j
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1/info")
public class InfoController {

    @GetMapping("/about")
    public ResponseEntity<InfoResponse>
about() {
        return
ResponseEntity.ok(new
InfoResponse(
            "Беянин Никита Николаевич",
            "ПИБд-41",
            "Лабораторная работа №1",
            "Разграничение полномочий
пользователя с криптографической защитой",
            "DES (режим CBC), SHA",
            "Отсутствие повторяющихся
символов в пароле"));
    }
}

```

```

InfoResponse.java
package com.cryptoauth.infosecurity.info;

```

```

public record InfoResponse(
    String author,
    String group,
    String labNumber,
    String topic,
    String algorithms,
    String passwordPolicy) {}

```

```

auth
AuthController.java
package
com.cryptoauth.infosecurity.auth.controller;

```

```

import
com.cryptoauth.infosecurity.auth.dto.*;
import
com.cryptoauth.infosecurity.auth.service.AuthService
;

```

```

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import
org.springframework.http.ResponseEntity;

```

```

import
org.springframework.web.bind.annotation.*;

@Slf4j
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1/auth")
public class AuthController {
    private final AuthService authService;

    @PostMapping("/login")
    public ResponseEntity<LoginResult>
loginAndSendOtp(@RequestBody UserLogin
userLogin) {
    LoginResult result =
authService.login(userLogin);
    return ResponseEntity.ok(result);
}

    @PostMapping("/resend-otp")
    public ResponseEntity<String>
repeatOtp(@RequestBody ResendOtpRequest
request) {
    authService.repeatOtpCode(request);
    return ResponseEntity.ok("OTP
repeat");
}

    @PostMapping("/forgot-password")
    public ResponseEntity<String>
forgotPassword(@RequestBody
ForgotPasswordRequest request) {
    authService.forgotPassword(request);
    return ResponseEntity.ok("OTP sent to
email");
}

    @PostMapping("/verify-otp-forgot-
password")
    public ResponseEntity<String>
verifyOtpForgotPassword(@RequestBody
OtpVerifyRequest request) {
    authService.verifyOtpForgotPassword(request);
    return ResponseEntity.ok("OTP verify
success!");
}

    @PostMapping("/reset-password")
    public ResponseEntity<String>
resetPassword(@RequestBody
ResetPasswordRequest request) {
    authService.resetPassword(request);
    return ResponseEntity.ok("Password
has been reset successfully");
}

    @PostMapping("/token")
    public ResponseEntity<String>
getToken(@RequestBody OtpVerifyRequest request)
{
    String token =
authService.verifyAndGenerateToken(request);

```

```

        return ResponseEntity.ok(token);
    }

    @GetMapping("/valid-token")
    public ResponseEntity<String>
validToken(@RequestParam("token") String token) {
    authService.validToken(token);
    return ResponseEntity.ok("Token is
valid");
}

    dto
ForgotPasswordRequest.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record ForgotPasswordRequest(String
email) {}

    LoginResult.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record LoginResult(boolean
requiresOtp, String email, String token) {}

    OtpVerifyRequest.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record OtpVerifyRequest(String
email, String otp) {}

    ResendOtpRequest.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record ResendOtpRequest(String
email) {}

    ResetPasswordRequest.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record ResetPasswordRequest(String
email, String newPassword) {}

    UserLogin.java
package
com.cryptoauth.infosecurity.auth.dto;

    public record UserLogin(String email, String
password) {}

    error
AdviceController.java

```

```

package
com.cryptoauth.infosecurity.auth.error;

import java.io.IOException;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import
org.springframework.http.HttpStatusCode;
import
org.springframework.http.MediaType;
import
org.springframework.http.ResponseEntity;
import
org.springframework.security.access.AccessDeniedE
xception;
import
org.springframework.security.core.AuthenticationExc
eption;
import
org.springframework.security.web.AuthenticationEnt
ryPoint;
import
org.springframework.web.bind.MethodArgumentNot
ValidException;
import
org.springframework.web.bind.annotation.Exception
Handler;
import
org.springframework.web.bind.annotation.ResponseS
tatus;
import
org.springframework.web.bind.annotation.RestContr
ollerAdvice;

import
com.fasterxml.jackson.databind.ObjectMapper;

import jakarta.servlet.ServletException;
import
jakarta.servlet.http.HttpServletRequest;
import
jakarta.servlet.http.HttpServletResponse;

@RestControllerAdvice
public class AdviceController implements
AuthenticationEntryPoint {
    private final Logger log =
    LoggerFactory.getLogger(AdviceController.class);

    public static ErrorCauseDto
    getRootCause(Throwable throwable) {
        Throwable rootCause = throwable;
        while (rootCause.getCause() != null
        && rootCause.getCause() != rootCause) {
            rootCause = rootCause.getCause();
        }
        final StackTraceElement firstError =
        rootCause.getStackTrace()[0];
        return new ErrorCauseDto(
            rootCause.getClass().getName(),
            firstError.getMethodName(),

```

```

            firstError.getFileName(),
            firstError.getLineNumber());
        }

        private ResponseEntity<ErrorDto>
        handleException(Throwable throwable,
        HttpStatusCode httpCode) {
            log.error("{} ",
            throwable.getMessage());
            throwable.printStackTrace();
            final ErrorDto errorDto = new
            ErrorDto(throwable.getMessage(),
            AdviceController.getRootCause(throwable));
            return new ResponseEntity<>(errorDto,
            httpCode);
        }

        @Override
        public void commence(
            HttpServletRequest request,
            HttpServletResponse response,
            AuthenticationException
            authException) throws IOException,
            ServletException {
            final ResponseEntity<ErrorDto> body
            =
            handleException(authException,
            HttpStatus.UNAUTHORIZED);

            response.setContentType(MediaType.APPLICATIO
            N_JSON_VALUE);

            response.setStatus(body.getStatusCode().value());
            response.getWriter().write(new
            ObjectMapper().writeValueAsString(body.getBody()
            ));
        }

        @ExceptionHandler(AccessDeniedException.class)

        @ResponseStatus(HttpStatus.FORBIDDEN)
        public ResponseEntity<ErrorDto>
        handleAccessDeniedException(Throwable
        throwable) {
            return handleException(throwable,
            HttpStatus.FORBIDDEN);
        }

        @ExceptionHandler(NotFoundException.class)

        @ResponseStatus(HttpStatus.NOT_FOUND)
        public ResponseEntity<ErrorDto>
        handleNotFoundException(Throwable throwable) {
            return handleException(throwable,
            HttpStatus.NOT_FOUND);
        }

        @ExceptionHandler(MethodArgumentNotValidExce
        ption.class)

        @ResponseStatus(HttpStatus.BAD_REQUEST)

```

```

        public ResponseEntity<ErrorDto>
handleDataIntegrityViolationException(Throwable
throwable) {
        return handleException(throwable,
HttpStatus.BAD_REQUEST);
    }

```

```

    @ExceptionHandler(Exception.class)

```

```

    @ResponseStatus(HttpStatus.INTERNAL_SERVER
_ERROR)
    public ResponseEntity<ErrorDto>
handleAnyException(Throwable throwable) {
        return handleException(throwable,
HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```

```

ErrorCauseDto.java
package
com.cryptoauth.infosecurity.auth.error;

```

```

public class ErrorCauseDto {
    private String exception;
    private String methodName;
    private String fineName;
    private int lineNumber;

```

```

    ErrorCauseDto(String exception, String
methodName, String fineName, int lineNumber) {
        this.exception = exception;
        this.methodName = methodName;
        this.fineName = fineName;
        this.lineNumber = lineNumber;
    }

```

```

    public String getException() {
        return exception;
    }

```

```

    public String getMethodName() {
        return methodName;
    }

```

```

    public String getFineName() {
        return fineName;
    }

```

```

    public int getLineNumber() {
        return lineNumber;
    }
}

```

```

ErrorDto.java
package
com.cryptoauth.infosecurity.auth.error;

```

```

public class ErrorDto {
    private String error;
    private ErrorCauseDto rootCause;

```

```

        public ErrorDto(String error,
ErrorCauseDto rootCause) {
            this.error = error;
            this.rootCause = rootCause;
        }

```

```

    public String getError() {
        return error;
    }

```

```

    public ErrorCauseDto getRootCause() {
        return rootCause;
    }
}

```

```

NotFoundException.java
package
com.cryptoauth.infosecurity.auth.error;

```

```

    public class NotFoundException extends
RuntimeException {
        public <T>
NotFoundException(Class<T> clazz, Long id) {
            super(String.format("%s with id [%s] is
not found or not exists", clazz.getSimpleName(), id));
        }
    }

```

```

Exceptions
AccountLockedException.java
package
com.cryptoauth.infosecurity.auth.exceptions;

```

```

    public class AccountLockedException
extends RuntimeException {
        public AccountLockedException(String
email) {
            super("Account locked: " + email);
        }
    }

```

```

InvalidPasswordException.java
package
com.cryptoauth.infosecurity.auth.exceptions;

```

```

    public class InvalidPasswordException
extends RuntimeException {
        public InvalidPasswordException(String
message) {
            super(message);
        }
    }
}

```

```

Model
OtpInfo.java

```



```

package
com.cryptoauth.infosecurity.auth.model;

public class OtpInfo {

    private final String email;

    private final String otp;
    private final long expirationTime;

    public OtpInfo(String email, String otp,
long expirationTime) {
        this.email = email;
        this.otp = otp;
        this.expirationTime = expirationTime;
    }

    public String email() { return email; }
    public String otp() { return otp; }
    public long expirationTime() { return
expirationTime; }
}

```

```

Jwt
JwtException.java
package
com.cryptoauth.infosecurity.auth.jwt;

public class JwtException extends
RuntimeException {
    public JwtException(Throwable
throwable) {
        super(throwable);
    }

    public JwtException(String message) {
        super(message);
    }
}

```

```

JwtFilter.java
package
com.cryptoauth.infosecurity.auth.jwt;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import
jakarta.servlet.http.HttpServletRequest;
import
jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.lang.NonNull;
import
org.springframework.security.authentication.Username
PasswordAuthenticationToken;
import
org.springframework.security.core.context.SecurityC
ontextHolder;

```

```

import
org.springframework.security.core.userdetails.UserD
etails;
import
org.springframework.security.web.authentication.We
bAuthenticationDetailsSource;
import
org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import
org.springframework.web.filter.OncePerRequestFilte
r;

import
com.cryptoauth.infosecurity.auth.service.MyUserDet
ailsService;

import java.io.IOException;
import java.util.List;

@Slf4j
@Component
@RequiredArgsConstructor
public class JwtFilter extends
OncePerRequestFilter {

    private static final String
AUTHORIZATION = "Authorization";
    private static final String
TOKEN_PREFIX = "Bearer ";

    private final JwtService jwtService;
    private final MyUserDetailsService
userDetailsService;

    // Пути, которые не нужно фильтровать
    private static final List<String>
EXCLUDE_URLS = List.of(
        "/swagger-ui", "/swagger-ui.html",
"/v3/api-docs", "/v3/api-docs/swagger-config",
"/h2-console", "/api/auth/initialize",
"/api/auth/login"
    );

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest
request,
        @NonNull HttpServletResponse
response,
        @NonNull FilterChain filterChain
    ) throws ServletException, IOException {

        String path = request.getRequestURI();

        // Если путь публичный —
пропускаем JWT фильтр
        if
(EXCLUDE_URLS.stream().anyMatch(path::startsW
ith) ||
"OPTIONS".equalsIgnoreCase(request.getMethod()))
{

```

```

        filterChain.doFilter(request,
response);
        return;
    }

    try {
        String token =
getTokenFromRequest(request);

        if (StringUtils.hasText(token) &&
SecurityContextHolder.getContext().getAuthenticatio
n() == null) {

            String username =
jwtService.extractEmail(token);

            if (username != null) {
                UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

                if (userDetails != null &&
jwtService.validateToken(token, userDetails)) {

                    UsernamePasswordAuthenticationToken authToken =
=
                        new
UsernamePasswordAuthenticationToken(
                            userDetails,
                            null,

userDetails.getAuthorities()
                );

                authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(requ
est));

                SecurityContextHolder.getContext().setAuthenticatio
n(authToken);

                log.debug("Пользователь {}
аутентифицирован", username);
            } else {
                log.warn("JWT токен не
прошёл валидацию");
            }
        }
    } catch (Exception ex) {
        log.error("Ошибка при обработке
JWT токена: {}", ex.getMessage());
    }

    filterChain.doFilter(request, response);
}

private String
getTokenFromRequest(HttpServletRequest request) {
    String bearer =
request.getHeader(AUTHORIZATION);
    if (StringUtils.hasText(bearer) &&
bearer.startsWith(TOKEN_PREFIX)) {

```

```

        return
bearer.substring(TOKEN_PREFIX.length());
    }
    return null;
}

JwtProperties.java
package
com.cryptoauth.infosecurity.auth.jwt;

import
org.springframework.boot.context.properties.Configu
rationProperties;
import
org.springframework.context.annotation.Configuratio
n;

import lombok.Getter;
import lombok.Setter;

@Configuration
@ConfigurationProperties(prefix = "jwt",
ignoreInvalidFields = true)
@Getter
@Setter
public class JwtProperties {
    private String key = "";

    // Base64 encoded secret (256-bit min for
HS256)
    private String secret;

    // Время истечения срока действия
токена в миллисекундах
    private long expirationMs = 1000 * 60 *
60 * 10; // 10 часов
}

JwtService.java
package
com.cryptoauth.infosecurity.auth.jwt;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;

import
org.springframework.security.core.userdetails.UserD
etails;
import
org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

```

```

import java.util.function.Function;

@Service
@RequiredArgsConstructor
public class JwtService {
    private final JwtProperties jwtProperties;

    private SecretKey secretKey;

    @PostConstruct
    public void init() {
        byte[] keyBytes =
Decoders.BASE64.decode(jwtProperties.getSecret());
        this.secretKey =
Keys.hmacShaKeyFor(keyBytes);
    }

    public String generateToken(String email,
String role){
        Map<String, Object> claims = new
HashMap<>();
        claims.put("role", role);
        claims.put("jti",
UUID.randomUUID().toString());

        return Jwts.builder()
            .claims()
            .add(claims)
            .subject(email)

            .id(UUID.randomUUID().toString())
            .issuedAt(new
Date(System.currentTimeMillis()))
            .expiration(new
Date(System.currentTimeMillis() +
jwtProperties.getExpirationMs()))
            .and()
            .signWith(secretKey)
            .compact();
    }

    private <T> T extractClaim(String token,
Function<Claims, T> claimResolver){
        final Claims claims =
extractAllClaim(token);
        return claimResolver.apply(claims);
    }

    public Claims extractAllClaim(String
token){
        return Jwts.parser()
            .verifyWith(secretKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public boolean validateToken(String
token, UserDetails userDetails) {
        final String email =
extractEmail(token);

```

```

        return
(email.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
    }

    private boolean isTokenExpired(String
token) {
        return
extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String
token) {
        return
extractClaim(token,
Claims::getExpiration);
    }

    public String extractEmail(String token) {
        return
extractClaim(token,
Claims::getSubject);
    }
}

```

```

SecurityConfiguration.java
package
com.cryptoauth.infosecurity.auth.security;

import
com.cryptoauth.infosecurity.auth.jwt.JwtFilter;
import lombok.RequiredArgsConstructor;
import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuratio
n;
import
org.springframework.http.HttpMethod;
import
org.springframework.security.authentication.Authenti
cationManager;
import
org.springframework.security.config.annotation.auth
entication.configuration.AuthenticationConfiguration;
import
org.springframework.security.config.annotation.meth
od.configuration.EnableMethodSecurity;
import
org.springframework.security.config.annotation.web.
builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.
configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.
configurers.AbstractHttpConfigurer;
import
org.springframework.security.config.http.SessionCre
ationPolicy;
import
org.springframework.security.web.SecurityFilterChai
n;

```

```

import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import
org.springframework.web.cors.CorsConfiguration;
import
org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import
org.springframework.web.filter.CorsFilter;

import java.util.List;

/**
 * Конфигурация Spring Security для
 системы управления доступом
 */
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(securedEnabled =
true)
@RequiredArgsConstructor
public class SecurityConfiguration {

    private final JwtFilter jwtFilter;

    @Bean
    SecurityFilterChain
filterChain(HttpSecurity httpSecurity) throws
Exception {

        // Отключаем CSRF (используем
JWT)

httpSecurity.csrf(AbstractHttpConfigurer::disable);

        // H2 Console frames
httpSecurity.headers(headers ->
headers.frameOptions(frame ->
frame.sameOrigin())
);

        // CORS конфигурация
httpSecurity.cors(cors ->
cors.configurationSource(request -> {
    CorsConfiguration config = new
CorsConfiguration();
    config.setAllowCredentials(true);

    config.setAllowedOrigins(List.of("http://localhost:30
00", "http://localhost:5173"));

    config.setAllowedHeaders(List.of("*"));

    config.setAllowedMethods(List.of("GET", "POST",
"PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setMaxAge(3600L);
    return config;
}));

        // Stateless сессии (JWT)

httpSecurity.sessionManagement(session ->

```

```

session.sessionCreationPolicy(SessionCreationPolicy
.STATELESS)
);

// Правила авторизации

httpSecurity.authorizeHttpRequests(requests ->
requests

// Preflight запросы

.requestMatchers(HttpMethod.OPTIONS,
"/**").permitAll()

// Swagger и API Docs публичны
.requestMatchers(
"/swagger-ui/**",
"/v3/api-docs/**",
"/swagger-ui.html"
).permitAll()

// Публичные эндпоинты
(инициализация и логин)
.requestMatchers(
"/api/v1/auth/**",
"/api/v1/info/about",
"/h2-console/**"
).permitAll()

// Все остальные требуют
аутентификации
.anyRequest().authenticated()
);

// Добавляем JWT фильтр
httpSecurity.addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class);

return httpSecurity.build();
}

@Bean
public CorsFilter corsFilter() {
    CorsConfiguration config = new
CorsConfiguration();

    config.setAllowedOrigins(List.of("http://localhost:30
00", "http://localhost:5173"));

    config.setAllowedMethods(List.of("GET", "POST",
"PUT", "PATCH", "DELETE", "OPTIONS"));

    config.setAllowedHeaders(List.of("*"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L);

    UrlBasedCorsConfigurationSource
source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**",
config);

    return new CorsFilter(source);
}

```

```

@Bean
public AuthenticationManager
authenticationManager(AuthenticationConfiguration
config) throws Exception {
    return
    config.getAuthenticationManager();
}

service
AuthService.java
package
com.cryptoauth.infosecurity.auth.service;

import
com.cryptoauth.infosecurity.auth.dto.*;
import
com.cryptoauth.infosecurity.auth.jwt.JwtService;
import
com.cryptoauth.infosecurity.user.entity.User;
import
com.cryptoauth.infosecurity.user.repository.UserRep
ository;
import
com.cryptoauth.infosecurity.user.utils.PasswordHash
Util;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import
org.springframework.stereotype.Service;

import java.time.LocalDateTime;
import java.util.Date;

@Slf4j
@Service
@RequiredArgsConstructor
public class AuthService {

    private final JwtService jwtService;
    private final OtpService otpService;
    private final UserRepository
userRepository;

    public LoginResult login(UserLogin
userLogin) {

        // Проверка email и существования
        пользователя
        if (userLogin.email() == null ||
userLogin.email().isBlank()) {
            throw new
RuntimeException("Email cannot be empty");
        }

        User user =
userRepository.findByEmail(userLogin.email())
                .orElseThrow(() -> new
RuntimeException("User not found"));

        if (user.isAccountLocked()) {

```

```

            throw new
RuntimeException("Account is locked");
        }

        log.info("Попытка входа
пользователя: {}", user.getEmail());

        // Проверка срока действия пароля
        (опционально)
        if (user.getPasswordLastChanged() !=
null && user.getPasswordExpirationMonths() > 0) {
            long monthsSinceChange =
java.time.temporal.ChronoUnit.MONTHS.between(
user.getPasswordLastChanged(),
LocalDateTime.now());
            if (monthsSinceChange >=
user.getPasswordExpirationMonths()) {
                log.warn("Срок действия пароля
истёк для: {}", user.getEmail());
                // Можно отправлять
                уведомление или требовать сброс пароля
            }
        }

        // Первый вход — пароль не
        проверяется, сразу token
        if (user.isFirstLogin()) {
            user.setFirstLogin(false);
            userRepository.save(user);
            String token =
jwtService.generateToken(user.getEmail(),
user.getRole().name());
            return new LoginResult(false,
user.getEmail(), token);
        }

        // Проверка пароля
        boolean passwordOk =
PasswordHashUtil.matches(
            userLogin.password(),
            user.getSalt(),
            user.getPassword_hash());

        if (!passwordOk) {
            int failedAttempts =
user.getFailedAttempts() + 1;

            user.setFailedAttempts(failedAttempts);
            if (failedAttempts >= 3) {
                user.setAccountLocked(true);
                log.warn("Аккаунт
заблокирован: {}", user.getEmail());
            }
            userRepository.save(user);
            throw new
RuntimeException("Invalid login or password");
        }

        // Сбрасываем счётчик неудачных
        попыток при успешном входе
        if (user.getFailedAttempts() > 0) {
            user.setFailedAttempts(0);
        }
    }
}

```

```

    }

    userRepository.save(user);

    // Определяем логику OTP
    if (user.isEmailConfirmed()) {
        // Генерируем OTP и сообщаем
        // контроллеру, что нужен OTP

        otpService.generateAndSendOtp(user.getEmail());
        log.info("OTP отправлен
        пользователю {}", user.getEmail());
        return new LoginResult(true,
        user.getEmail(), null);
    } else {
        // Email не подтверждён — сразу
        // выдаём токен

        String token =
        jwtService.generateToken(user.getEmail(),
        user.getRole().name());
        return new LoginResult(false,
        user.getEmail(), token);
    }
}

public String
verifyAndGenerateToken(OtpVerifyRequest request)
{
    User user =
    userRepository.findByEmail(request.email())
    .orElseThrow(() -> new
    RuntimeException("User not found"));

    if (!user.isEmailConfirmed()) {
        log.info("Email не подтверждён —
        OTP не требуется");
        return
        jwtService.generateToken(user.getEmail(),
        user.getRole().name());
    }

    boolean verified =
    otpService.verifyOtp(request.email(), request.otp());
    if (!verified)
        throw new
        RuntimeException("Invalid or expired OTP");

    log.info("OTP успешно подтверждён
    — выдаём токен для {}", user.getEmail());
    return
    jwtService.generateToken(user.getEmail(),
    user.getRole().name());
}

public void
resetPassword(ResetPasswordRequest request) {
    User user =
    userRepository.findByEmail(request.email())
    .orElseThrow(() -> new
    RuntimeException("User not found"));

    String newSalt =
    PasswordHashUtil.generateSalt();

```

```

    String newHash =
    PasswordHashUtil.sha1Hex(request.newPassword(),
    newSalt);

    user.setSalt(newSalt);
    user.setPassword_hash(newHash);

    user.setPasswordLastChanged(LocalDateTime.now()
    );

    user.setPasswordLastChanged(LocalDateTime.now()
    );

    user.setFailedAttempts(0);
    user.setAccountLocked(false);
    userRepository.save(user);

    log.info("Пароль успешно обновлён
    для пользователя {}", user.getEmail());
}

public void validToken(String token) {
    try {
        var claims =
        jwtService.extractAllClaim(token);
        if (claims.getExpiration().before(new
        Date())) {
            throw new
            RuntimeException("Token expired");
        }
        log.info("Токен действителен для
        {}", claims.getSubject());
    } catch (Exception e) {
        throw new
        RuntimeException("Invalid token", e);
    }

    // ---- Дополнительные методы для
    // контроллера ----

    public void
    repeatOtpCode(ResendOtpRequest request) {
        otpService.generateAndSendOtp(request.email());
        log.info("Повтор OTP для {}",
        request.email());
    }

    public void
    forgotPassword(ForgotPasswordRequest request) {
        otpService.generateAndSendOtp(request.email());
        log.info("Отправлен OTP для
        восстановления пароля {}", request.email());
    }

    public void
    verifyOtpForgotPassword(OtpVerifyRequest request)
    {
        boolean verified =
        otpService.verifyOtp(request.email(), request.otp());
        if (!verified)

```

```

        throw new
RuntimeException("Invalid OTP for password
reset");
        log.info("OTP успешно подтверждён
для восстановления {}", request.email());
    }
}

```

```

EmailService.java
package
com.cryptoauth.infosecurity.auth.service;

import java.time.LocalDateTime;

import
org.springframework.mail.javamail.JavaMailSender;
import
org.springframework.mail.javamail.MimeMessageHe
lper;
import
org.springframework.stereotype.Service;
import lombok.RequiredArgsConstructor;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;

@Service
@RequiredArgsConstructor
public class EmailService {
    private final JavaMailSender mailSender;

    public void sendOtpEmail(String toEmail,
String otpCode) {
        try {
            MimeMessage message =
mailSender.createMimeMessage();
            MimeMessageHelper helper = new
MimeMessageHelper(message, true, "UTF-8");

            // От кого письмо (имя + email)
            helper.setFrom("Информационная
безопасность! <infosecurity@gmail.com>");
            // Почта поддержки

            helper.setReplyTo("infosecurity@gmail.com");
            // Кому
            helper.setTo(toEmail);

            // Тема письма — информативная
и ненавязчивая
            helper.setSubject("Ваш уникальный
код подтверждения входа");

            // Текст письма — с приветствием,
объяснением, деталями и подписью
            StringBuilder emailText = new
StringBuilder();

            emailText.append("Здравствуйте!

```

```

👋 \n\n");

```

```

        emailText.append(

```

```

        "Мы получили запрос на
отправку кода подтверждения для доступа к
вашему аккаунту. 📧\n");
        emailText.append("Пожалуйста,
используйте следующий код для подтверждения
вашей личности:\n\n");

```

```

        emailText.append("🔑 >>>
").append(otpCode).append(" <<< 🔑 \n\n");

```

```

        emailText.append("🕒 Этот код
действителен в течение 5 минут. Пожалуйста, не
сообщайте его никому.\n\n");

```

```

        emailText.append(
            "Если вы не запрашивали этот
код, пожалуйста, проигнорируйте это письмо —
это может быть угрозой вашей
безопасности.\n\n");

```

```

        emailText.append(
            "🔒 Для вашей безопасности
не пересылайте этот код и не вводите его на
подозрительных сайтах.\n\n");

```

```

        emailText.append(
            "Если у вас возникли вопросы
или проблемы, пожалуйста, свяжитесь с нашей
службой поддержки:\n");

```

```

        emailText.append("✉️
support@infosecurity@gmail.com\n\n");

```

```

        emailText.append("С
уважением,\n");
        emailText.append("Команда
infosecurity@gmail.com 🤖\n\n");

```

```

        emailText.append("🕒 Дата и время
отправки:
").append(LocalDateTime.now()).append("\n");

```

```

        helper.setText(emailText.toString(),
false);

```

```

        mailSender.send(message);
    } catch (MessagingException e) {
        throw new RuntimeException("Не
удалось отправить email: " + e.getMessage(), e);
    }
}
}

```

```

MyUserDetailsService.java
package
com.cryptoauth.infosecurity.auth.service;

```

```

import java.util.List;

```

```

import
org.springframework.security.core.authority.SimpleG
rantedAuthority;

```

```

import
org.springframework.security.core.userdetails.UserD
etails;
import
org.springframework.security.core.userdetails.UserD
etailsService;
import
org.springframework.security.core.userdetails.Userna
meNotFoundException;
import
org.springframework.stereotype.Service;

import
com.cryptoauth.infosecurity.user.entity.User;
import
com.cryptoauth.infosecurity.user.repository.UserRep
ository;

@Service
public class MyUserDetailsService
implements UserDetailsService {
    private final UserRepository userRepository;

    public
MyUserDetailsService(UserRepository
userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails
loadUserByUsername(String email) throws
UsernameNotFoundException {
        User user =
userRepository.findByEmail(email)
                .orElseThrow(() -> new
UsernameNotFoundException("User not found"));

        return new
org.springframework.security.core.userdetails.User(
            user.getEmail(),
            user.getPassword_hash(),
            List.of(new
SimpleGrantedAuthority("ROLE_"
+ user.getRole().name()))
        );
    }
}

```

```

OtpService.java
package
com.cryptoauth.infosecurity.auth.service;

import
org.springframework.stereotype.Service;
import
com.cryptoauth.infosecurity.auth.model.OtpInfo;

import lombok.RequiredArgsConstructor;

import java.security.SecureRandom;
import java.time.Instant;

```

```

import java.util.Map;
import
java.util.concurrent.ConcurrentHashMap;

@Service
@RequiredArgsConstructor
public class OtpService {

    private final EmailService emailService;

    private final Map<String, OtpInfo>
otpStorage = new ConcurrentHashMap<>();
    private static final long
EXPIRATION_TIME_MS = 5 * 60 * 1000;
    private static final SecureRandom random
= new SecureRandom();

    public void generateAndSendOtp(String
email) {
        String otp = String.format("%06d",
random.nextInt(1_000_000));
        long expiration =
Instant.now().toEpochMilli()
+ EXPIRATION_TIME_MS;

        otpStorage.put(email, new
OtpInfo(email, otp, expiration));
        emailService.sendOtpEmail(email,
otp);
    }

    public boolean verifyOtp(String email,
String otp) {
        OtpInfo info = otpStorage.get(email);

        if (info == null) return false;

        boolean valid = info.otp().equals(otp)
&& Instant.now().toEpochMilli()
< info.expirationTime();

        if (valid) {
            otpStorage.remove(email);
        }

        return valid;
    }
}

```

```

utils
OpenApiConfig.java
package
com.cryptoauth.infosecurity.auth.utils;

import
org.springframework.context.annotation.Bean;
import
org.springframework.context.annotation.Configuratio
n;

import
io.swagger.v3.oas.models.Components;

```



```

import io.swagger.v3.oas.models.OpenAPI;
import
io.swagger.v3.oas.models.security.SecurityRequirement;
import
io.swagger.v3.oas.models.security.SecurityScheme;

@Configuration
public class OpenApiConfig {

    private static final String
SCHEME_NAME = "JWT";
    private static final String SCHEME =
"bearer";

    private SecurityScheme
createBearerScheme() {
        return new SecurityScheme()

.type(SecurityScheme.Type.HTTP)
        .scheme(SCHEME);

    }

    @Bean
    OpenAPI customOpenApi() {
        return new OpenAPI()
            .components(new Components())

.addSecuritySchemes(SCHEME_NAME,
createBearerScheme()))
        .addSecurityItem(new
SecurityRequirement().addList(SCHEME_NAME));
    }

}

```

```

Frontend (REACT)
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml"
href="/vite.svg" />
    <meta name="viewport"
content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
</head>
<body>
    <div id="root"></div>
    <script type="module"
src="/src/main.tsx"></script>
</body>
</html>

```

```

Vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

```

```

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})

```

```

App.tsx
import { Routes, Route } from 'react-router-
dom'

import WelcomePage from
'./pages/WelcomePage'
import LoginPage from './pages/LoginPage'
import './App.css'
import UserManager from
'./pages/UserManager'
import ResetPasswordPage from
'./pages/ResetPasswordPage'

function App() {
  return (
    <Routes>
      <Route path="/"
element={<WelcomePage />} />
      <Route path="/login"
element={<LoginPage />} />
      <Route path="/usermanager"
element={<UserManager />} />
      <Route path="/changepassword"
element={<ResetPasswordPage />} />
    </Routes>
  )
}

export default App

```

```

main.tsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import { BrowserRouter } from 'react-
router-dom';
import './index.css'
import App from './App.tsx'

```

```

createRoot(document.getElementById('root')
!).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>,
)

```

```

Login.tsx
import React, { useState } from 'react';
import { authService } from '../api/clients';
import type { UserLogin, OtpVerifyRequest,
LoginResult } from '../types/auth';

```

```

const LoginPage: React.FC = () => {
  const [email, setEmail] = useState("");

```

```

const [password, setPassword] =
useState("");
const [otp, setOtp] = useState("");
const [loginResult, setLoginResult] =
useState<LoginResult | null>(null);
const [error, setError] = useState<string |
null>(null);

// === Универсальный обработчик
ошибок от Spring AdviceController ===
const extractErrorMessage = (err: any):
string => {
  if (!err) return 'Неизвестная ошибка';

  const backendError = err.response?.data
  ?? err.message ?? err.toString();

  if (typeof backendError === 'string')
return backendError;

  if (backendError.message) return
backendError.message;

  if (backendError.error) return
backendError.error;

  if (backendError.cause &&
backendError.cause.className)
return `Ошибка:
${backendError.cause.className} →
${backendError.cause.method}`;

  try {
    return JSON.stringify(backendError);
  } catch {
    return 'Не удалось обработать
ошибку';
  }
};

// === Авторизация по email + password
===

const handleLogin = async (e:
React.FormEvent) => {
  e.preventDefault();
  setError(null);

  try {
    const result = await authService.login({
email, password } as UserLogin);
    console.log('Login response:', result);

    setLoginResult({
      email: result.email,
      requiresOtp: result.requiresOtp,
      token: result.token ?? null,
    });

    // если OTP не требуется → сразу
заходим
    if (!result.requiresOtp && result.token) {
      localStorage.setItem('authToken',
result.token);

```

```

      localStorage.setItem('user',
JSON.stringify({ email: result.email }));
      window.location.href = '/userManager';
    }
  } catch (err: any) {
    console.error('Login error:', err);
    setError(extractErrorMessage(err));
  }
};

// === Проверка OTP ===
const handleVerifyOtp = async () => {
  if (!loginResult) return;
  setError(null);

  const request: OtpVerifyRequest = {
email: loginResult.email, otp };

  try {
    const response = await
authService.getToken(request);
    const token = response.token;

    if (!token) throw new Error("Token not
received");

    localStorage.setItem('authToken', token);
    localStorage.setItem('user',
JSON.stringify({ email: loginResult.email }));
    window.location.href =
'/changepassword';
  } catch (err: any) {
    console.error('OTP error:', err);
    setError(extractErrorMessage(err));
  }
};

return (
  <div style={{ maxWidth: 400, margin:
'50px auto', textAlign: 'center' }}>
    <h2>Login</h2>

    { /* === Форма email + password ===
*/ }

    { !loginResult?.requiresOtp && (
      <>
        <form onSubmit={handleLogin}>
          <input
            type="email"
            placeholder="Email"
            value={email}
            onChange={(e) =>
setEmail(e.target.value)}
            required
            style={{ marginBottom: 8, width:
'100%', padding: 6 }}
          />
          <br />
          <input
            type="password"
            placeholder="Password"
            value={password}

```

```

        onChange={e} =>
setPassword(e.target.value)}
        required
        style={{ marginBottom: 8, width:
'100%', padding: 6 }}
      />
      <br />
      <button type="submit" style={{
padding: 8, width: '100%' }}>
        Login
      </button>
    </form>
  </>
)}

{ /* === Форма для OTP === */ }
{loginResult?.requiresOtp && (
  <>
    <p>OTP sent to
{loginResult.email}</p>
    <input
      type="text"
      placeholder="Enter OTP"
      value={otp}
      onChange={e} =>
setOtp(e.target.value)}
      style={{ marginBottom: 8, width:
'100%', padding: 6 }}
    />
    <br />
    <button onClick={handleVerifyOtp}
style={{ padding: 8, width: '100%' }}>
      Verify OTP
    </button>
  </>
)}

{error && (
  <p
    style={{
      color: 'red',
      backgroundColor: '#ffecec',
      border: '1px solid #f5aca6',
      padding: '6px 10px',
      borderRadius: 4,
      marginTop: 10,
    }}
  >
    {error}
  </p>
)}
</div>
);
};

export default LoginPage;

UserManager.tsx
import React, { useEffect, useState } from
'react';
import { userService } from '../api/clients';

```

```

import { useNavigate } from 'react-router-
dom';

const UserManager: React.FC = () => {
  const navigate = useNavigate();
  const [user, setUser] =
useState<any>(null);
  const [loading, setLoading] =
useState(true);
  const [users, setUsers] =
useState<any[]>([]);
  const [search, setSearch] = useState("");
  const [newEmail, setNewEmail] =
useState("");
  const [editingUser, setEditingUser] =
useState<any | null>(null);

  useEffect(() => {
    const token =
localStorage.getItem('authToken');
    if (!token) {
      navigate('/login');
      return;
    }

    userService
      .getMe(token)
      .then((userData) => {
        if (userData.role !== 'ADMIN') {
          navigate('/login');
          return;
        }
        if (userData.role === 'USER') {
          navigate('/changepassword');
          return;
        }
        setUser(userData);
        loadUsers();
      })
      .catch((err) => {
        console.error('Ошибка получения
профиля:', err);
        localStorage.removeItem('authToken');
        localStorage.removeItem('user');
        navigate('/login');
      })
      .finally(() => setLoading(false));
  }, [navigate]);

  const loadUsers = async () => {
    try {
      const data = await
userService.getAllUsers(search);
      setUsers(Array.isArray(data) ? data :
data.content || data.users || []);
    } catch (e) {
      console.error('Ошибка загрузки
пользователей:', e);
    }
  };

  const handleAddUser = async () => {
    if (!newEmail.trim()) return;

```

```

    try {
      await userService.registerUser({ email:
newEmail });
      setNewEmail("");
      await loadUsers();
    } catch (e) {
      console.error('Ошибка добавления
пользователя:', e);
      alert('Ошибка при добавлении
пользователя');
    }
  };

  /**
   * СБРОС/УСТАНОВКА пароля
   админом + обновление остальных полей
   * Если у пользователя включены
passwordRestrictions, проверяем правило:
   * — все символы пароля должны быть
уникальны (отсутствие повторяющихся
символов).
   */
  const handleSaveEdit = async (id: string)
=> {
    try {
      if (!editingUser) return;

      const {
        email,
        role,
        accountLocked,
        passwordRestrictions,
        passwordExpirationMonths,
        minPasswordLength,
        failedAttempts,
        firstLogin,
        emailConfirmed,
        newPassword, // новое поле пароля
      } = editingUser;

      // Проверки на фронте
      if (newPassword && newPassword.length
> 0) {
        if (passwordRestrictions) {
          const chars = newPassword.split("");
          const unique = new Set(chars);
          if (unique.size !== chars.length) {
            alert('Пароль не должен содержать
повторяющихся символов.');
```

```

            await userService.updateUser(id, {
              email,
              role,
              accountLocked,
              passwordRestrictions,
              passwordExpirationMonths,
              minPasswordLength,
              failedAttempts,
              firstLogin,
              emailConfirmed,
              password_hash: newPassword &&
newPassword.length > 0 ? newPassword : undefined,
              // только если ввели
            });

            setEditingUser(null);
            await loadUsers();
          } catch (e) {
            console.error('Ошибка обновления
пользователя:', e);
            alert('Ошибка при обновлении
пользователя');
          }
        };

        const handleDeleteUser = async (email:
string) => {
          if (!window.confirm('Удалить
пользователя ' + email + '?')) return;
          try {
            await userService.deleteUser(email);
            await loadUsers();
          } catch (e) {
            console.error('Ошибка удаления
пользователя:', e);
            alert('Ошибка при удалении
пользователя');
          }
        };

        if (loading) return <p>Загрузка...</p>;
        if (!user) return null;

        return (
          <div>
            <h2>Панель управления
пользователями</h2>
            <p>Привет, {user.email}</p>
            <p>Роль: {user.role}</p>

            <hr />
            <h3>Список пользователей</h3>

            <div style={{ marginBottom: 10 }}>
              <input
                type="text"
                placeholder="Поиск по email..."
                value={search}
                onChange={(e) =>
setSearch(e.target.value)}
              />

```

```

        <button      onClick={loadUsers}>🔍
Найти</button>
    </div>

```

```

    <div style={{ marginBottom: 10 }}>
        <input
            type="email"
            placeholder="Введите email нового
пользователя"
            value={newEmail}
            onChange={(e) =>
setNewEmail(e.target.value)}
        />
        <button onClick={handleAddUser}>□
Добавить</button>
    </div>

```

```

    <table border={1} cellPadding={6}
style={{ borderCollapse: 'collapse', width: '100%'
}}>

```

```

    <thead>
    <tr>
        <th>Email</th>
        <th>Роль</th>
        <th>Заблокирован</th>
        <th>Огр. пароля</th>
        <th>Мин. длина</th>
        <th>Истечение пароля (мес)</th>
        <th>Попытки</th>
        <th>Первый вход</th>
        <th>Email подтверждён</th>
        <th>Хэш пароля</th>
        <th>Соль</th>
        <th>Дата смены пароля</th>
        <th>Действия</th>
    </tr>
    </thead>
    <tbody>
        {users.length === 0 && (
            <tr>
                <td colspan={13} style={{
textAlign: 'center' }}>Нет пользователей</td>
            </tr>
        )}
    </tbody>

```

```

    <tbody>
        {users.map((u) => (
            <tr key={u.id}>
                <td>
                    {editingUser?.id === u.id ? (
                        <input
                            type="email"
                            value={editingUser.email}
                            onChange={(e) =>
                                setEditingUser({
...editingUser, email: e.target.value })
                            />
                    ) : (
                        u.email
                    )}
                </td>

```

```

                <td>
                    {editingUser?.id === u.id ? (

```

```

        <select
            value={editingUser.role}
            onChange={(e) =>
                setEditingUser({
...editingUser, role: e.target.value })
            }
        >
            <option
value="ADMIN">ADMIN</option>
            <option
value="USER">USER</option>
        </select>
    ) : (
        u.role
    )}
</td>

```

```

    <td>
        {editingUser?.id === u.id ? (
            <input
                type="checkbox"

checked={!editingUser.accountLocked}
                onChange={(e) =>
                    setEditingUser({
...editingUser, accountLocked: e.target.checked })
                />
        ) : (
            u.accountLocked ? '🔒' : '🔓'
        )}
    </td>

```

```

    <td>
        {editingUser?.id === u.id ? (
            <input
                type="checkbox"

checked={!editingUser.passwordRestrictions}
                onChange={(e) =>
                    setEditingUser({
...editingUser, passwordRestrictions: e.target.checked
})
                />
        ) : (
            u.passwordRestrictions ? 'Да' :
'Нет'
        )}
    </td>

```

```

    <td>
        {editingUser?.id === u.id ? (
            <input
                type="number"

value={editingUser.minPasswordLength ?? 0}
                onChange={(e) =>
                    setEditingUser({
...editingUser, minPasswordLength: +e.target.value
})
                />
        ) : (
            u.minPasswordLength
        )}
    </td>

```

```

        />
      ) : (
        u.minPasswordLength
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="number"

value={editingUser.passwordExpirationMonths ?? 0}
        onChange={(e) =>
          setEditingUser({
            ...editingUser,
            passwordExpirationMonths:
+e.target.value,
          })
        }
        style={{ width: 60 }}
      ) />
      ) : (
        u.passwordExpirationMonths
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="number"

value={editingUser.failedAttempts ?? 0}
        onChange={(e) =>
          setEditingUser({
            ...editingUser, failedAttempts: +e.target.value })
          }
        style={{ width: 60 }}
      ) />
      ) : (
        u.failedAttempts
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="checkbox"

checked={!editingUser.firstLogin}
        onChange={(e) =>
          setEditingUser({
            ...editingUser, firstLogin: e.target.checked })
          }
      ) />
      ) : (
        u.firstLogin ? 'Да' : 'Нет'
      )}
    </td>

    <td>
      {editingUser?.id === u.id ? (
        <input
          type="checkbox"

```

```

checked={!editingUser.emailConfirmed}
        onChange={(e) =>
          setEditingUser({
            ...editingUser, emailConfirmed: e.target.checked })
          }
      ) />
      ) : (
        u.emailConfirmed ? 'Да' : 'Нет'
      )}
    </td>

    <td style={{ fontSize: '0.8em',
color: '#555' }}>{u.password_hash}</td>
    <td style={{ fontSize: '0.8em',
color: '#555' }}>{u.salt}</td>
    <td>{u.passwordLastChanged ??
'—'}</td>

    <td>
      {editingUser?.id === u.id ? (
        <div style={{ margin: 10px 0 0 0 }}>
        {/* Поле для установки
        пароля админом */}
        <div style={{ marginBottom: 6
        }}>
        <input
          type="password"
          placeholder="Новый пароль
(админ)"

value={editingUser.newPassword ?? ""}
        onChange={(e) =>
          setEditingUser({
            ...editingUser, newPassword: e.target.value })
          }
        style={{ width: 180 }}
      ) />
    </div>

    <div>
      <button onClick={() =>
handleSaveEdit(u.id)}>Сохранить</button>
      <button onClick={() =>
setEditingUser(null)}>Отмена</button>
    </div>
    </td>
      ) : (
        <div>
          <button onClick={() =>
setEditingUser(u)}>Восстановить</button>
          <button onClick={() =>
handleDeleteUser(u.email)}>Удалить</button>
        </div>
      )}
    </td>
  </tr>
</tbody>
</table>
</div>
);

```

```

    };

    export default UserManager;

    WelcomePage.tsx
    import React, { useState, useEffect } from
'react';
    import { useNavigate } from 'react-router-
dom';
    import type { InfoResponse } from
'../../types/info';
    import { infoService } from '../api/clients';
    import './WelcomePage.css';

    const WelcomePage: React.FC = () => {
        const [info, setInfo] =
useState<InfoResponse | null>(null);
        const [loading, setLoading] =
useState<boolean>(true);
        const [error, setError] =
useState<string>("");
        const navigate = useNavigate();

        useEffect(() => {
            loadInfo();
        }, []);

        const loadInfo = async () => {
            try {
                setLoading(true);
                const data = await
infoService.getAbout();
                setInfo(data);
            } catch (err: any) {
                setError(err.response?.data?.message ||
err.message || 'Ошибка загрузки');
            } finally {
                setLoading(false);
            }
        };

        const handleNext = () => {
            navigate('/login');
        };

        if (loading) {
            return (
                <div className="welcome-container">
                    <div className="loading">Загрузка
информации...</div>
                </div>
            );
        }

        if (error) {
            return (
                <div className="welcome-container">
                    <div className="error">
                        <h3>Ошибка загрузки</h3>
                        <p>{error}</p>
                        <button
                            onClick={loadInfo}
className="retry-button">

```

```

                            Повторить попытку
                        </button>
                    </div>
                </div>
            );
        }

        return (
            <div className="welcome-container">
                <div className="welcome-header">
                    <h1>Система криптографической
защиты</h1>
                    <p>Добро пожаловать в систему
управления пользователями</p>
                </div>

                {info && (
                    <div className="info-panel">
                        <InfoRow label="👤 Автор:"
value={` ${info.author} (${info.group})` } />
                        <InfoRow label="🏢 Работа:"
value={info.labNumber} />
                        <InfoRow label="🔒 Тема:"
value={info.topic} />
                        <InfoRow label="📋 Алгоритмы:"
value={info.algorithms} />
                        <InfoRow label="🔐 Политика
паролей:" value={info.passwordPolicy} />
                    </div>
                )}

                <div className="actions">
                    <button
                        className="next-button"
                        onClick={handleNext}
                        disabled={!info}
                    >
                        Перейти к системе →
                    </button>
                </div>
            </div>
        );
    };

    const InfoRow: React.FC<{ label: string;
value: string }> = ({ label, value }) => (
        <div className="info-row">
            <div className="info-
label">{label}</div>
            <div className="info-
value">{value}</div>
        </div>
    );

    export default WelcomePage;

    ResetPasswordPage.tsx
    import React, { useState } from 'react';
    import { authService } from '../api/clients';
    import { useNavigate } from 'react-router-
dom';

```

```

const ResetPasswordPage: React.FC = () =>
{
  const navigate = useNavigate();

  const [email, setEmail] = useState("");
  const [newPassword, setNewPassword] =
  useState("");
  const [confirmPassword,
  setConfirmPassword] = useState("");
  const [loading, setLoading] =
  useState(false);
  const [message, setMessage] =
  useState<string | null>(null);
  const [success, setSuccess] =
  useState(false);

  const handleSubmit = async (e:
  React.FormEvent) => {
    e.preventDefault();

    // Проверка заполнения
    if (!email || !newPassword ||
    !confirmPassword) {
      setMessage('❑ Все поля обязательны
      для заполнения.');
```

return;

```

    }

    // Проверка совпадения паролей
    if (newPassword !== confirmPassword) {
      setMessage('❑ Пароли не совпадают.');
```

return;

```

    }

    // Проверка длины пароля
    if (newPassword.length < 6) {
      setMessage('Пароль должен содержать
      минимум 6 символов.');
```

return;

```

    }

    setLoading(true);
    setMessage(null);

    try {
      const response = await
      authService.resetPassword({ email, newPassword });

      if (response?.status === 200 ||
      response?.success) {
        setSuccess(true);
        setMessage('❑ Пароль успешно
        изменён! Перенаправление...');
```

setTimeout(() => navigate('/login'),

```

        2000);
      } else {
        setMessage('❑ Ошибка: сервер
        вернул неожиданный ответ.');
```

}

```

      } catch (err: any) {
        console.error('Ошибка сброса пароля:',
        err);

```

```

const backendMsg =
  err?.response?.data?.message ||
  err?.response?.data ||
  'Не удалось изменить пароль.
  Проверьте email.';
  setMessage(`❑ ${backendMsg}`);
} finally {
  setLoading(false);
}
};

return (
  <div
    style={{
      maxWidth: 420,
      margin: '80px auto',
      background: '#fff',
      padding: '30px 25px',
      borderRadius: 12,
      boxShadow: '0 2px 10px
      rgba(0,0,0,0.1)',
    }}
  >
    <h2 style={{ textAlign: 'center',
    marginBottom: 20 }}>🔒 Сброс пароля</h2>

    {message && (
      <p
        style={{
          textAlign: 'center',
          color: success ? 'green' : 'red',
          fontWeight: 500,
        }}
      >
        {message}
      </p>
    )}

    <form onSubmit={handleSubmit}>
      <div style={{ marginBottom: 12 }}>
        <label>Email:</label>
        <input
          type="email"
          value={email}
          placeholder="Введите ваш email"
          onChange={(e) =>
            setEmail(e.target.value)}
          style={{
            width: '100%',
            padding: 10,
            borderRadius: 8,
            border: '1px solid #ccc',
            marginTop: 5,
          }}
        />
      </div>

      <div style={{ marginBottom: 12 }}>
        <label>Новый пароль:</label>
        <input
          type="password"
          value={newPassword}

```



```

        placeholder="Введите          НОВЫЙ
пароль"
        onChange={ (e) =>
setNewPassword(e.target.value)}
        style={{
            width: '100%',
            padding: 10,
            borderRadius: 8,
            border: '1px solid #ccc',
            marginTop: 5,
        }}
    />
</div>

<div style={{ marginBottom: 20 }}>
<label>Повторите пароль:</label>
<input
    type="password"
    value={confirmPassword}
    placeholder="Повторите пароль"
    onChange={ (e) =>
setConfirmPassword(e.target.value)}
    style={{
        width: '100%',
        padding: 10,
        borderRadius: 8,
        border: '1px solid #ccc',
        marginTop: 5,
    }}
    />
</div>

<button
    type="submit"
    disabled={loading}
    style={{
        width: '100%',
        padding: 10,
        background: loading ? '#aaa' :
'#007bff',
        color: '#fff',
        border: 'none',
        borderRadius: 8,
        cursor: loading ? 'not-allowed' :
'pointer',
        fontSize: 16,
        fontWeight: 600,
    }}
    />
    {loading ? 'Обработка...' : 'Сбросить
пароль'}
</button>

<button
    type="button"
    onClick={() => navigate('/login')}
    style={{
        marginTop: 10,
        width: '100%',
        padding: 10,
        background: '#f0f0f0',
        color: '#333',
        border: 'none',

```

```

        borderRadius: 8,
        cursor: 'pointer',
        fontSize: 15,
    }}
    />
    Назад ко входу
</button>
</form>
</div>
);
};

export default ResetPasswordPage;

api
clients.ts
import axios, { type AxiosInstance } from
'axios';
import type { InfoResponse } from
'../types/info';
import type { UserCreate, UserGet,
UserUpdate } from '../types/user';
import type { AuthResponse, LoginResult,
OtpVerifyRequest, UserLogin } from '../types/auth';

const API_BASE =
'http://localhost:8988/api/v1';

// === Axios-инстансы ===
export const infoApi = axios.create({
  baseURL: `${API_BASE}/info`, timeout: 10000 });
export const userApi = axios.create({
  baseURL: `${API_BASE}/user`, timeout: 10000 });
export const authApi = axios.create({
  baseURL: `${API_BASE}/auth`, timeout: 10000 });

// === INFO SERVICE ===
export const infoService = {
  getAbout: async () :
Promise<InfoResponse> => {
    const response = await
infoApi.get('/about');
    return response.data;
  },
};

// === USER SERVICE ===
export const userService = {
  getAllUsers: async (search?: string,
pageable?: any): Promise<any> => {
    const params: any = {};
    if (search) params.search = search;
    if (pageable) Object.assign(params,
pageable);
    const response = await
userApi.get('/users', { params });
    return response.data;
  },
  registerUser: async (userData: UserCreate):
Promise<UserGet> => {

```

```

        const response = await
userApi.post('/register', userData);
        return response.data;
    },

    getUserByEmail: async (email: string):
Promise<UserGet> => {
        const response = await
userApi.get(`/users/${email}`);
        return response.data;
    },

    getUserById: async (id: string):
Promise<UserGet> => {
        const response = await
userApi.get(`/users/user/${id}/id`);
        return response.data;
    },

    updateUser: async (id: string, userData:
UserUpdate): Promise<UserGet> => {
        const response = await
userApi.put(`/users/${id}`, userData);
        return response.data;
    },

    // Добавляем этот метод
    getMe: async (token?: string) => {
        const authToken = token ||
localStorage.getItem('authToken');
        if (!authToken) throw new Error('Token
not found');

        const response = await userApi.get('/me', {
            headers: {
                Authorization: `Bearer ${authToken}`,
            },
        });

        return response.data; // возвращаем
данные пользователя
    },

    deleteUser: async (email: string):
Promise<void> => {
        await userApi.delete(`/users/${email}`);
    },
};

// === AUTH SERVICE ===
export const authService = {
    login: async (credentials: UserLogin):
Promise<LoginResult> => {
        const response = await
authApi.post('/login', credentials);
        // □ мапим backend-ответ точно под
LoginResult
        return {
            email: response.data.email,
            requiresOtp: response.data.requiresOtp,
            token: response.data.token ?? null,
        };
    },

```

```

    },

    // Получение токена через OTP
    getToken: async (otpData:
OtpVerifyRequest): Promise<AuthResponse> => {
        const response = await
authApi.post('/token', otpData);
        const token = response.data;
        const userStr =
localStorage.getItem('user');
        const user = userStr ?
JSON.parse(userStr) : null;

        if (token && user) {
            localStorage.setItem('authToken', token);
            localStorage.setItem('user',
JSON.stringify(user));
            return { token, user };
        }
        throw new Error('Token or user data
missing');
    },

    // Проверка токена на валидность
    validateToken: async (token?: string):
Promise<boolean> => {
        const authToken = token ||
localStorage.getItem('authToken');
        if (!authToken) throw new Error('Token
not found');
        const response = await
authApi.get(`/valid-token?token=${authToken}`);
        return response.data === 'Token is valid';
    },

    // Регистрация
    register: async (userData: UserCreate):
Promise<AuthResponse> => {
        const response = await
authApi.post('/register', userData);
        return response.data;
    },

    resetPassword: async (payload: { email:
string; newPassword: string }) => {
        const response = await
authApi.post('/reset-password', payload);
        return response.data;
    },

    //Изменение пароля
    changePassword: async (data: {
oldPassword: string; newPassword: string }, token?:
string): Promise<any> => {
        const authToken = token ||
localStorage.getItem('authToken');
        const response = await
authApi.post('/change-password', data, { headers: {
            Authorization: `Bearer ${authToken}` } });
        return response.data;
    },

```

```

        resetPasswordForAdmin: async (email:
string, newPassword: string, token?: string):
Promise<any> => {
            const authToken = token ||
localStorage.getItem('authToken');
            const response = await authApi.post(
                `/auth/reset-password`,
                { email, newPassword },
                { headers: { Authorization: `Bearer
${authToken}` } }
            );
            return response.data;
        },

        // Выход
        logout: async (): Promise<void> => {
            try { await authApi.post('/logout'); }
        finally {
            localStorage.removeItem('authToken');
            localStorage.removeItem('user');
            window.location.href = '/login';
        }
    },
};

// === Интерцепторы ===
const setupAuthInterceptor = (instance:
AxiosInstance) => {
    instance.interceptors.request.use((config:
any) => {
        const token =
localStorage.getItem('authToken');
        if (token) config.headers.Authorization =
`Bearer ${token}`;
        return config;
    });
};

const setupErrorInterceptor = (instance:
AxiosInstance) => {
    instance.interceptors.response.use(
        (response) => response,
        (error) => {
            if (error.response?.status === 401) {
                localStorage.removeItem('authToken');
                localStorage.removeItem('user');
                window.location.href = '/login';
            }
            return Promise.reject(error);
        }
    );
};

// === Применяем интерцепторы ===
[infoApi, userApi, authApi].forEach(api =>
{
    setupAuthInterceptor(api);
    setupErrorInterceptor(api);
});

types
auth.ts

```

```

import type { UserGet } from './user';

export interface UserLogin {
    email: string;
    password: string;
}

export interface LoginResult {
    email: string;
    requiresOtp: boolean; // точно как с
бекенда
    token?: string | null; // можно оставить,
если токен может прийти
}

export interface OtpVerifyRequest {
    email: string;
    otp: string;
}

export interface AuthResponse {
    token: string;
    user: UserGet;
    requiresOtp?: boolean;
}

export interface OtpResponse {
    message: string;
    otpSent: boolean;
}

Info.ts
export interface InfoResponse {
    author: string;
    group: string;
    labNumber: string;
    topic: string;
    algorithms: string;
    passwordPolicy: string;
}

User.ts
export interface UserCreate {
    email: string;
}

export interface UserGet {
    id: string;
    email: string;
    role: string;
    accountLocked: boolean;
    passwordRestrictions: boolean;
    minPasswordLength: number;
    passwordExpirationMonths: number;
    passwordLastChanged: string;
    firstLogin: boolean;
}

export interface UserUpdate {
    email?: string;
    role?: string;
}

```

```
password_hash?: string;  
salt?: string;  
accountLocked?: boolean;  
passwordRestrictions?: boolean;  
passwordExpirationMonths?: number;  
passwordLastChanged?: string;  
minPasswordLength?: number;
```

```
failedAttempts?: number;  
firstLogin?: boolean;  
emailConfirmed?: boolean;  
}
```